System Verification and Validation Plan for Squigglr

Team #17, Squigglr
Rayan Nasrallah
Ahmed Elzaria
Muhammad Huzaifah
Shamil Canbolat
Moustafa Moustafa

November 1, 2026

Revision History

Date	Developer(s)	Change
29 October 2026	Rayan Nasrallah	Wrote Section 3.6, Automated Testing and Verification Tools (#222).
29 October 2026	Rayan Nasrallah	Wrote the performance tests in Section 4.2, T-PERF-001 to T-PERF-007 (#223).
29 October 2026	Rayan Nasrallah	Wrote the safety-critical tests in Section 4.2, T-SAF-001 to T-SAF-017, and the safety inspection (#226).
29 October 2026	Rayan Nasrallah	Wrote the operational, maintainability, security, cultural and compliance tests in Section 4.2, the traceability table in Section 4.3 and the Symbolic Parameters in Section 6.1 (#227).
29 October 2026	Rayan Nasrallah	Added his reflection answers (#228).
29 October 2026	Rayan Nasrallah	Added his generative AI use disclosure (#229).
30 October 2026	Moustafa Moustafa	Wrote Section 3.1, Verification and Validation Team (#232).
30 October 2026	Moustafa Moustafa	Wrote Sections 2.1 and 2.2, Summary and Objectives (#233).
30 October 2026	Moustafa Moustafa	Wrote Section 2.4, Relevant Documentation, and added bibliography entries for the project documents (#234).
30 October 2026	Moustafa Moustafa	Wrote Section 3.4, Verification and Validation Plan Verification (#235).
30 October 2026	Moustafa Moustafa	Wrote the revision history, Section 1, Symbols, Abbreviations, and Acronyms, and the document roadmap; added Rayan's revision history rows to the SRS and Hazard Analysis (#236).
30 October 2026	Moustafa Moustafa	Wrote the introductions to Sections 3, 4, 4.1, 4.2 and 4.3 (#237).

Date	Developer(s)	Change
30 October 2026	Moustafa Moustafa	Rewrote reflection Questions 1 and 2 as team answers and added Moustafa's Questions 3 and 4 (#242).

Contents

1 Symbols, Abbreviations, and Acronyms **vi**

2 General Information **1**
- 2.1 Summary . . . 1
- 2.2 Objectives . . . 2
- 2.3 Custom Documents . . . 3
- 2.4 Relevant Documentation . . . 4

3 Plan **5**
- 3.1 Verification and Validation Team . . . 5
- 3.2 SRS Verification . . . 7
- 3.3 Design Verification . . . 8
- 3.4 Verification and Validation Plan Verification . . . 14
- 3.5 Implementation Verification . . . 17
- 3.6 Automated Testing and Verification Tools . . . 25
- 3.7 Software Validation . . . 27

4 System Tests **28**
- 4.1 Tests for Functional Requirements . . . 28
  - 4.1.1 Guided Lessons . . . 29
  - 4.1.2 Creative Drawing: Editing, Saving and Separation from Lessons . . . 58
  - 4.1.3 Creative Drawing: Refinement and Restoring the Original . . . 69
  - 4.1.4 Attempt Records . . . 84
  - 4.1.5 Progress Summaries . . . 99
- 4.2 Tests for Nonfunctional Requirements . . . 112
  - 4.2.1 Performance . . . 112
  - 4.2.2 Safety-Critical Requirements . . . 118
  - 4.2.3 Operational and Environmental . . . 131
  - 4.2.4 Maintainability and Support . . . 132
  - 4.2.5 Security . . . 135
  - 4.2.6 Cultural . . . 138
  - 4.2.7 Compliance . . . 138
- 4.3 Traceability Between Test Cases and Requirements . . . 140

5 Unit Test Description **144**
5.1 Unit Testing Scope . . . 146
5.2 Tests for Functional Requirements . . . 147
5.3 Tests for Nonfunctional Requirements . . . 147
5.4 Traceability Between Test Cases and Modules . . . 148

6 Appendix **150**
6.1 Symbolic Parameters . . . 150
6.2 Usability Survey Questions? . . . 158

7 Appendix — Generative AI Use Disclosure **159**
7.1 AI Tools Used . . . 159
7.2 How and Where Used . . . 159
7.3 Verification of AI-Generated Content . . . 159

8 Appendix — Reflection **162**

List of Tables

1 Abbreviations and terms added by this plan . . . vi
2 Verification and validation roles of the team members . . . 6
3 Reviews of the design documents . . . 8
4 Design checklist items owned by each member . . . 10
5 Reviews of this plan . . . 14
6 Static verification of the implementation . . . 18
7 Code Checklist items and how reviewers apply them . . . 20
8 Automated testing and verification tools . . . 26
9 Cases for T-LRN-002 (FR-LRN-002 and FR-LRN-016) . . . 33
10 Cases for T-LRN-003 (FR-LRN-003) . . . 33
11 Cases for T-LRN-004 (FR-LRN-004 to FR-LRN-006) . . . 34
12 Cases for T-LRN-005 (FR-LRN-007) . . . 35
13 Cases for T-LRN-006 (FR-LRN-008) . . . 36
14 Cases for T-LRN-007 (FR-LRN-009) . . . 37
15 Cases for T-LRN-008 (FR-LRN-010) . . . 39
16 Cases for T-LRN-009 (FR-LRN-011) . . . 40
17 Cases for T-LRN-010 (FR-LRN-012) . . . 41
18 Cases for T-LRN-011 (FR-LRN-013) . . . 42
19 Cases for T-LRN-012 (FR-LRN-014) . . . 44

20 Cases for T-LRN-013 (FR-LRN-015) . . . 45
21 Cases for T-LRN-014 (FR-LRN-017) . . . 46
22 Cases for T-LRN-015 (FR-LRN-018) . . . 48
23 Cases for T-LRN-016 (FR-LRN-019) . . . 50
24 Cases for T-LRN-017 (FR-LRN-020) . . . 52
25 Cases for T-LRN-018 (FR-LRN-021) . . . 54
26 Cases for T-LRN-020 (FR-LRN-023) . . . 56
27 Cases for T-REC-001 (FR-REC-001) . . . 86
28 Cases for T-REC-002 (FR-REC-002) . . . 88
29 Cases for T-REC-003 (FR-REC-003) . . . 89
30 Cases for T-REC-004 (FR-REC-004) . . . 90
31 Cases for T-REC-005 (FR-REC-005) . . . 91
32 Cases for T-REC-006 (FR-REC-006) . . . 93
33 Cases for T-REC-007 (FR-REC-007) . . . 94
34 Cases for T-REC-008 (FR-REC-008) . . . 95
35 Cases for T-REC-009 (FR-REC-009) . . . 97
36 Cases for T-REC-010 (FR-REC-010) . . . 98
37 Learner A's lesson records and the expected summary for T-PRG-002 (FR-PRG-002) . . . 101
38 Expected attempt counts for T-PRG-003 (FR-PRG-003) . . . 102
39 Cases for T-PRG-004 (FR-PRG-004) . . . 103
40 Cases for T-PRG-007 (FR-PRG-007) . . . 106
41 Reviewer tasks for T-PRG-013 (FR-PRG-003, FR-PRG-005) . . . 111
42 Traceability for nonfunctional requirements . . . 141
43 Traceability for functional requirements . . . 142
44 Symbolic parameters for nonfunctional tests . . . 150
45 Symbolic parameters for functional tests . . . 152

1 Symbols, Abbreviations, and Acronyms

We use the terms and acronyms in the glossary of the Software Requirements Specification (SRS Section 4.1) and the symbolic constants in SRS Section 4.2 without defining them again here. Table 1 lists only the ones this plan adds. Constants that only our tests use are in the Symbolic Parameters appendix.

Table 1: Abbreviations and terms added by this plan

Term	Meaning
DevOps	Development and Operations: development environments, build automation and deployment.
Parameterized test	One test with a table of cases, where each case has a label and its own expected output, so it can be cited on its own, for example `T-LRN-005 (c)`.
PH-1, PH-2, PH-3	The SRS delivery phases: the proof-of-concept demo (23 November 2026), Rev 0 (1 February 2027) and the final demo (15 March 2027).
PR	Pull request: a proposed change to the repository, reviewed before it merges into `main`.
Rev 0, Rev 1	The first complete version of the product and documents, and the final revised version.
SAF	Prefix of the SRS safety-critical requirements, for example `SAF-RAY-001`.
SI	Safety inspection: the code inspection that gives each safety requirement a second, independent check (Section 4.2.2).
`T-<AREA>-###`	A system test ID. `AREA` is the requirement group it tests: `LRN`, `REC`, `CRT`, `PRG`, `UX`, `PERF`, `SAF`, `OPS`, `MAINT`, `SEC`, `CULT` or `COMP`.
VnV	Verification and validation.

This document is our plan for verifying and validating Squigglr: how we will check that it is built correctly, and that it is the right product for children and the adults supporting them. Section 2 says what we are testing and what we are focusing on. Section 3 says who does what, how we will review the SRS, the design, this plan and the code, which tools we use, and how we will validate Squigglr with the supervisor, the specialists and adult users. Section 4 gives the system tests for the functional and nonfunctional requirements and traces each test to its requirement. Section 5 explains how we will choose unit tests; the module tests are added once the Module Interface Specification (MIS) is done. The appendices hold the symbolic parameters, the usability survey questions, our generative AI use disclosure and our reflection.

2 General Information

2.1 Summary

Squigglr is a website that helps children practise drawing on a touchscreen with a stylus, and helps the adults supporting them see how that practice is going. This plan covers its four main parts:

- **Guided lessons.** An adult picks a lesson, like drawing a square, and the child works through stages that give less and less help: a demonstration, tracing a full outline, drawing between endpoints, then drawing with no guide. Squigglr moves to the next stage when an attempt passes. Squigglr can suggest going back a stage, but only an adult can make that change.
- **Creative drawing.** The child draws freely, and Squigglr can tidy up the drawing by adjusting the child's own strokes. The result stays editable and the original can always be restored.
- **Attempt records.** Each attempt keeps the stage, the difficulties the child ran into and the corrections they made, not just the final drawing.
- **Progress summaries.** Adults see how a child is doing over time, with attempts made with and without a guide kept separate.

The full requirements are in the SRS.

2.2 Objectives

Our main objective is to build confidence that Squigglr is safe for children, is actually what they and the adults supporting them need, and behaves correctly. In order of priority:

1. **Keep children safe and protect their well-being.** Squigglr is used by young children, some with autism, attention-deficit/hyperactivity disorder (ADHD) or other learning needs, so a failure that frustrates or upsets a child matters more than most bugs. Every safety-critical requirement (SRS Section 12.2) gets a test and a second, independent check in the safety inspection (Section 4.2.2).

2. **Validate that we are building the right thing.** Our teaching assistant (TA), Chris, pointed out that for this project, validation and usability matter more than verification. Correctly built lessons that the specialists disagree with, or summaries adults can't understand, are still a failure. We validate with the supervisor, the specialists we interviewed and adult usability sessions (see the Software Validation section).

3. **Show that Squigglr is usable and accessible.** Adults need to set up lessons and read summaries quickly, and the child's screens need to be easy to follow. We check this in usability sessions and against selected Web Content Accessibility Guidelines (WCAG) 2.2 success criteria.

4. **Make sure lessons and records behave correctly.** Stage changes, pass and fail decisions, attempt limits and adult control are the core of the product, and an adult who sees a wrong record or summary could make a bad decision for the child. We test these at the minimum, default and maximum of every value the adult can set, and against the formal specification in SRS Section 9.2.

5. **Protect children's data.** We keep records about children, so we check that a child is identified only by an identifier and an avatar, that no personal information is sent to the refinement service, and that only the right adults can see a child's records.

Some values are still to be determined (TBD) in the SRS, such as the performance limits and `REFINE_TOLERANCE`. For each one, this plan says how and when we will set it instead of leaving a placeholder (see the Symbolic Parameters appendix).

With five people and one academic year, we are leaving these out:

- **Testing with children.** No information is gathered from or about children and no study involves them (SRS CON-005 and CON-006), so every test and session uses adults or prepared attempts. This means the child's experience is judged by adults and the specialists rather than by children, which we accept as a limit of this plan.
- **Whether Squigglr improves children's drawing skills.** Showing that would take a long study with children, and Squigglr makes no such claim. We only check that the lessons work as specified and that the specialists think they make sense.
- **Third-party services and libraries.** We assume the authentication provider, the hosting and data storage service, the refinement model, the browsers and libraries like our testing frameworks are verified by the teams that make them. We only test how Squigglr uses them: what it sends, what it does with the result, and how it behaves when one is down. For example, we check that a refined drawing stays within `REFINE_TOLERANCE`, not how good the model's output looks.
- **Waiting Room features** (SRS Section 25), such as letter tracing, finger drawing and offline use, since they are not part of the product we are building. This includes full WCAG conformance; we only check the selected success criteria.
- **Every device and browser.** We test on the test iPad with a stylus, on the browsers in our end-to-end tests (Section 3.6), and on the other `SUPPORTED_DEVICES` combinations we have access to (T-PERF-007), not on every touchscreen device.

2.3 Custom Documents

[Summarize the CDs (Custom Documents) that were tackled by this project. CDs can include usability testing, code walkthroughs, user documentation,

formal proof, GenderMag personas, Design Thinking, etc. CDs should have already been approved by the course instructor as included in your problem statement. You can use a pull request to update your CDs (in TeamComposition.csv or Repos.csv) if your plan changes as a result of the VnV planning exercise. —SS]

2.4 Relevant Documentation

This plan builds on our other project documents. Here is how each one feeds into our VnV work:

- **Problem Statement and Goals** (Squigglr Team, 2026c). Defines the problem and the goals Squigglr has to meet. Validation checks Squigglr against this problem, not just against the SRS, so it is what we go back to when the supervisor or the specialists question whether a feature is needed.
- **Software Requirements Specification (SRS)** (Squigglr Team, 2026d). The main input to this plan. Every system test traces to an SRS requirement and uses its fit criterion as the pass condition (Section 4.3). The symbolic constants give the boundary values we test at, and the formal specification in SRS Section 9.2 gives the expected output of the lesson tests.
- **Hazard Analysis** (Squigglr Team, 2026b). Each hazard lists the requirements that mitigate it, which can be functional, usability or safety-critical (Hazard Analysis Section 6). The safety-critical ones get the safety tests and safety inspection (Section 4.2.2), and the rest are covered by their functional and usability tests. The causes of each hazard tell us what failures the tests should try to cause, like repeated failed attempts or a lost connection.
- **Development Plan** (Squigglr Team, 2026a). Names the language, coding standard, tools and continuous integration (CI) that our automated testing builds on (Section 3.6), and the lead roles behind some of our VnV roles. Its target-tablet experiment is where the first performance limits come from.
- **Module Guide (MG)** (Squigglr Team, 2027a). Due in January 2027. Splits Squigglr into modules. The unit tests are organized by these

modules, and the traceability between tests and modules checks that each one is covered. Design verification reviews it with the MG checklist.

- **Module Interface Specification (MIS)** (Squigglr Team, 2027b). Also due in January 2027. Gives each module's interface, which is what the unit tests call and check against.
- **VnV Report** (Squigglr Team, 2027c). Where we record the results of running this plan, including the measured values the performance tests report.

3 Plan

We start with who does what, then cover how we review the SRS, the design documents and this plan. After that come the plans for verifying the implementation through code walkthroughs, inspections and tests, and the tools that run those checks automatically (Section 3.6). The last part is validation: checking with the supervisor, the specialists and adult users that Squigglr is the right product.

3.1 Verification and Validation Team

Table 2 shows what each of us is responsible for verifying. Some roles come from our Development Plan leads: Huzaifah is Testing Lead, Shamil keeps CI running as DevOps Lead, and Ahmed plans the usability sessions as User Interface and User Experience (UI/UX) Lead. The rest come from the sections and tests each of us owns in this plan, and each of us still tests our own code. The teammate in the last column reviews each member's tests and pull requests, so nobody signs off on their own work.

Table 2: Verification and validation roles of the team members

Member and VnV role	**Verification responsibilities**	**Checked by**
Muhammad Huzaifah *Testing Lead and VnV coordinator*	Overall test strategy, plus the guided lesson and attempt record tests (`T-LRN`, `T-REC`). Leads code walkthroughs and the code inspections other than the safety inspection, records the safety inspection results, decides which tests run in CI, and keeps track of defects and coverage gaps.	Rayan
Rayan Nasrallah *Safety, performance and security lead*	Performance, safety-critical, operational, maintainability, security, cultural and compliance tests. Runs the safety inspection, which is the second check on every safety requirement (Section 4.2.2), and does not inspect code he wrote.	Huzaifah
Ahmed Elzaria *Design verification lead*	Creative drawing and progress summary tests (`T-CRT`, `T-PRG`), the unit test plan and the design document reviews. Plans the usability sessions with Shamil.	Moustafa
Shamil Canbolat *Requirements and validation lead*	Usability and accessibility tests (`T-UX`), SRS verification and software validation. Sets up the sessions with the supervisor and the specialists, sets up the automated testing tools, and keeps the CI build and test workflow running.	Ahmed
Moustafa Moustafa *VnV plan verification and quality assurance (QA) coordinator for this milestone*	Verifies this plan and does its final review against the VnV and Writing checklists. Turns review feedback into GitHub issues, takes notes at every review and validation session, and reviews the frontend code.	Shamil

People outside the team are also involved:

- **Dr. Christopher Anand (supervisor and client).** We give him a high-level update on the requirements, design and progress at our weekly meeting and record his feedback in the meeting issue. Shortly after Rev 0, we demo Squigglr to him so he can tell us whether it is what the project needs. His advice on children's safety and privacy goes straight into our requirements and tests, for example using an avatar instead of the child's name.

- **Domain specialists P1, P2 and P3.** Dr. Anand asked us (October 28 meeting) to go back to the specialists we interviewed for the SRS, or people with similar qualifications. They will try a working version of Squigglr and give feedback on the lesson guidance and feedback wording. They are volunteering their time, so we will keep the session short and work around their schedules.

- **Teaching assistant and peer teams.** They review the SRS, this plan and the design documents. We track each piece of their feedback as a GitHub issue with one owner.

- **Adult participants.** They take part in the usability sessions. We only test with adults, never children (SRS CON-005 and CON-006).

Our roles shift over the year. In the Fall, we write this plan and review the SRS and design documents. In the Winter, each of us runs the tests for our area as it gets built, and Shamil and Ahmed run the usability and specialist sessions. Like in the Development Plan, the QA coordinator rotates every milestone and is named in that milestone's tracking issue. If someone is unavailable or a role gets too heavy, we reassign the work at the next team meeting and update Table 2.

3.2 SRS Verification

[List any approaches you intend to use for SRS verification. This may include ad hoc feedback from reviewers, like your classmates (like your primary reviewer), or you may plan for something more rigorous/systematic. —SS]

[If you have a supervisor for the project, you shouldn't just say they will read over the SRS. You should explain your structured approach to the review. Will you have a meeting? What will you present? What questions

will you ask? Will you give them instructions for a task-based inspection? Will you use your issue tracker? —SS]

[Maybe create an SRS checklist? —SS]

3.3 Design Verification

The design is written in two documents: the Module Guide (MG), which splits Squigglr into modules, each hiding one secret, and the Module Interface Specification (MIS), which gives each module's access programs, state and behaviour. Design verification checks three things: that the MG covers every SRS requirement and hides every likely change, that each MIS module is precise enough to implement and to test from its text alone, and that the modules work together on the scenarios that matter most. Ahmed, as design verification lead, plans and moderates the reviews. The MG names the author of each module, and **no one signs off on a module they wrote**: if a reviewer's items fall on their own module, Ahmed gives those items to another reviewer for that module.

Table 3 lists the reviews. They follow the course schedule in Development Plan Section 8, and the internal reviews use the Sunday team review meeting before each deadline.

Table 3: Reviews of the design documents

Review	**Who**	**When**	**What it checks**
Internal MG review	Every member, on the checklist items they own (Table 4); Ahmed moderates	Team review on November 8, 2026, before the informal Design Revision −1 meeting (November 11)	The MG against the MG Checklist, and the traceability check below.
Informal Design Revision −1	Chris, our TA	November 11, 2026	Our questions on the module decomposition. Moustafa records the answers in the meeting issue, as for the VnV Plan.

Review	Who	When	What it checks
Internal MIS review	Every member, on their items; Ahmed moderates	Team review on November 15, 2026, before Design Revision −1 (November 16)	The MIS against the MIS Checklist, the task-based inspection of scenarios S1 and S2, and the interface check of T-MAINT-003.
Peer review	The peer team assigned by the course	November 19, 2026	The MG and MIS, read by people outside the project. We ask them to use the MG and MIS Checklists.
Supervisor review	Dr. Anand	Weekly supervisor meeting after November 16, 2026	A one-page summary of the modules, to check that the refinement and record modules match what he expects.
Second internal review	Every member, on their items; Ahmed moderates	Team review on January 10, 2027, before the informal design review (January 12)	Every module changed since Revision −1 against both checklists, the traceability check again, all four scenarios, and the interface check of T-MAINT-003.
Informal design review	Chris, our TA	January 12, 2027	Our remaining questions; answers recorded in the meeting issue.
Final check	Ahmed and Moustafa	Team review on January 17, 2027, before Design Revision 0 (January 18)	Every finding issue is closed or deferred with a reason, and the traceability check passes.
Peer module implementation	Another team, as set by the course	January 21 to 25, 2027	Whether one of our modules can be built from its MIS text alone (see below).

Review	Who	When	What it checks
Module issue check	Huzaifah	When each module issue is opened, and at the end of PH-1, PH-2 and PH-3	The interface check of T-MAINT-003 on every issue that creates a top-level module (see below).
Unit test design	Huzaifah, with each test author	While writing the unit tests in Section 5	Whether each access program's expected output can be worked out from the MIS.

Who owns which checklist items. Each member owns a part of the MG Checklist and the MIS Checklist, matched to their role, and checks it on every module they did not write (Table 4). Every item of both checklists has exactly one owner. Splitting the checklists means each item gets one careful reader instead of five quick ones.

Table 4: Design checklist items owned by each member

Member	MG Checklist	MIS Checklist
Ahmed	Module decomposition: one secret per module, secrets are nouns, the uses relation is a hierarchy and not a data flow, its arrows point from the user to the used module and down in clear layers, the three Level 1 groups, any control module at the top; MG quality (low coupling, information hiding).	MIS quality for each module: consistent, essential, general, minimal, high cohesion, right level of abstraction.

Member	MG Checklist	MIS Checklist
Shamil	The traceability matrices: every requirement satisfied by a module, every module used for a requirement; behaviour-hiding modules related to requirements; the MG follows the template.	MIS completeness: every MG module is in the MIS, every type is defined, every template section is present; each module's Uses lists only the modules needed to define its specification.
Huzaifah	Software-decision modules are concepts the requirements need but do not detail, and each is used by a behaviour-hiding module.	MIS semantics: each access program has an output or a state transition, and only rarely both; every state transition entry changes some state; outputs and exceptions are written as $out :=$ and $exc :=$; state invariants hold initially and after each call; every state and environment variable has a type; the syntax defines each interface precisely enough to implement.
Rayan	Likely changes: the anticipated changes include every SRS likely change, and each likely change maps to one module.	Opaque (information hiding) and implementation independent; modules that deal with files, the keyboard or the screen have environment variables for them; every safety requirement (SAF-) is met by an access program whose semantics a reviewer can point to.

Member	MG Checklist	MIS Checklist
Moustafa	The Writing Checklist items, and figures that can be zoomed in.	The Writing Checklist items, and each module starting on a new page; module classification (records as exported types, ADTs with **Template**, **Generic** where a module is parameterized by type, abstract objects with an initializer that is checked or assumed, libraries with no state); notation and syntax: exported constants have explicit values, types are not mixed, the notation is the course's or defined and is consistent, symbols come from the SRS, every argument and local function is used, and local functions make the specification easier to read.

Traceability check. The SRS has eight likely changes (LC-001 to LC-008, SRS Section 27). Shamil runs a script that lists every requirement ID defined in the SRS and every ID in the MG's traceability matrix, and reports any requirement with no module and any module with no requirement. Rayan checks by reading that each likely change maps to exactly one module, or that the MG explains the exception. The check passes when the script reports no gaps and every likely change is accounted for. It runs at both internal reviews and again at the final check, because requirements may change between them.

Task-based inspection. A checklist looks at one module at a time, so it can miss a gap between modules. The reviewers therefore trace prepared scenarios through the MIS, writing down each access program called, its inputs, the state it changes and its output. A step that no access program covers, a type that does not match between caller and callee, an exception

that no caller handles, or a state invariant that breaks is a finding. The scenarios cover the riskiest parts of the product:

- **S1, guided attempt.** The child draws `square-missing-side.json` at stage E with the stylus: stroke capture, evaluation, the missing-side feedback, the lesson rules (result, streak and `ATTEMPT_LIMIT`) and the saved attempt record.
- **S2, refinement and restore.** The child finishes `house.json` in the creative space: the refinement request, the `REFINE_TOLERANCE` check of the result, applying it as one change, then a restore and an undo; and the same with the connection lost.
- **S3, progress summary.** The adult opens learner A's summary (T-PRG-002), chooses the circle lesson, and opens the attempts behind a recurring difficulty.
- **S4, shared device.** The adult passes the adult check and switches learner while an attempt is still waiting to be saved (SAF-RAY-003, SAF-RAY-007).

S1 and S2 are traced at the first internal MIS review, and all four at the second. Each takes about 20 minutes, with one member reading the MIS aloud and the others checking.

Interface check for T-MAINT-003. NFR-MAINT-003 covers every issue that creates a top-level module, and most of these are opened once implementation starts after Design Revision 0. The check therefore follows the issues, not only the reviews. Each such issue is labelled `module` when it is opened. Huzaifah, as Testing Lead, compares it with that module's MIS entry within a week, and before the PR that closes it is merged, as T-MAINT-003 requires. An issue opened before its module is in the MIS is checked at the next internal MIS review. At both internal MIS reviews and at the end of PH-1, PH-2 and PH-3, he also lists every `module` issue opened since the last check, so none is missed. An issue with no interface description, or one that does not match the MIS, is a finding.

Peer module implementation. In January, another team builds one of our modules from its MIS (Development Plan Section 8). This is an outside check that the MIS can be implemented from its text. Each question

they have to ask us, and each place where their module behaves differently from ours, is recorded as a finding against the MIS, and their reflection (January 25) is read at the next team review.

Recording findings and finishing. Every finding becomes a GitHub issue labelled `review`, with the module, the checklist item or scenario step, and one owner, normally the module's author. Ahmed links it to the milestone's design tracking issue. Moustafa turns peer and TA feedback into issues in the same way, within a week. A review is complete when every module has been checked against every checklist item by someone who did not write it, the traceability check passes, and each finding issue is closed by a merged PR or deferred with a written reason that Ahmed accepts. The filled-in checklists are attached to the tracking issue, and the number of findings per review is reported in the VnV Report.

3.4 Verification and Validation Plan Verification

Each review in Table 5 checks this plan in a different way, and nobody reviews a part they wrote.

Table 5: Reviews of this plan

Review	**Who**	**When**	**What it checks**
Pull request review	The teammate who checks the author's work (see the Verification and Validation Team section)	Every PR, before it merges	The changed sections, using the review questions below. A PR needs one approval from someone other than its author.
TA review	Chris, our TA	Informal meeting on October 28, 2026, and the marked submission	Our questions on how to write the plan, then whether the submitted plan is specific and feasible. His October 28 answers are listed below.

Review	Who	When	What it checks
Supervisor review	Dr. Anand	Supervisor meeting on October 28, 2026, where we discussed validation (#201), then our weekly meetings after that	A high-level summary of how we will validate Squigglr, to check we are validating what matters to him.
Peer review	The peer teams assigned by the course	After submission, due November 5, 2026	The whole plan, read by people outside the project. We ask them to use the VnV checklist.
Mutation check	Each member, on sections they did not write	Final QA (November 1–2, 2026) and before Rev 1	Whether our tests would actually catch realistic bugs (see below).
Final QA	Moustafa, except Moustafa's own sections, which Shamil checks	November 1–2, 2026	The whole plan against the VnV and Writing checklists and the rubric (see below).
Running the plan	The owner of each test	Rev 0 and Rev 1	Any test we cannot run exactly as written is a defect in this plan.

TA feedback from October 28. In an informal meeting, we asked Chris how to handle a few parts of the plan. His answers became rules for the whole plan:

- A value can stay TBD, but we have to say how it will be decided. Every TBD constant in the Symbolic Parameters appendix has to say how and when it is set, and Final QA checks this.
- There is no minimum number of tests per safety requirement, but we

have to justify the number we pick. The safety tests explain why one test plus the safety inspection is enough (Section 4.2.2).

- One parameterized test with a table of cases counts as one test, as long as it is easy to follow. Each case gets a label and its own expected output, so it can be cited on its own.
- Full branch coverage is a good goal for safety-critical modules, and matters less for the user interface (Section 3.6).
- For this project, validation and usability matter more than verification.

Review questions. Every reviewer, in a PR review or in Final QA, asks these about each test they check:

1. Could someone else run it without asking the author? The input is specific and the expected output is explicit.
2. Does it trace to a requirement, and is it in the traceability tables (Section 4.3)?
3. Does it use symbolic constants instead of magic numbers, and does each constant have a value or a method in the Symbolic Parameters appendix?
4. Does it try to break Squigglr, with boundary values, invalid input and failures, instead of only showing the normal case?
5. For a review or inspection: does it say who does it, when, and with which checklist?
6. Can we actually do it with our team, devices and schedule?

Mutation check. Mutation testing makes a small change to a program and checks that some test fails. We apply the same idea to the plan. Each member takes five requirements whose tests they did not write, including at least two safety requirements. For each one, they write down a small, realistic bug, such as a lesson ending one attempt after `ATTEMPT_LIMIT` instead of at it, or a refined point landing just outside `REFINE_TOLERANCE`. Then they find the test that would fail because of it. If no test would fail, or the test's input

does not include that case, they open an issue for the test's owner. The check passes when every bug is caught by at least one test. Each member needs about an hour for this.

Final QA. As QA coordinator for this milestone, Moustafa:

1. checks that every SRS requirement ID appears in the traceability tables, with a script that compares the IDs in the SRS with the ones in this plan;
2. checks that every TBD constant has a method, that no template comments are left, and that the PDF builds with no errors or undefined references;
3. goes through the VnV and Writing checklists and the rubric, one item at a time;
4. sends fixes in someone else's section to them as PR comments.

Tracking findings. Small fixes the team finds, like wording, go in PR comments. Everything else, and all feedback from the TA and peer teams, becomes a GitHub issue with the `review` label, one assignee, and what closes it. An issue closes when the fix is merged or when we record why we are not changing the plan.

3.5 Implementation Verification

The implementation is checked in two ways: by running it and by reading it. Running it covers:

- the system tests in this plan: the functional tests (T-LRN, T-CRT, T-REC and T-PRG) and the nonfunctional tests (T-UX, T-PERF, T-SAF, T-OPS, T-MAINT, T-SEC, T-CULT and T-COMP), traced to the requirements in Section 4.3;
- the unit tests for each module, chosen as described in Section 5 once the MIS is done (the design documentation is due on 18 January 2027);
- the automated checks in Section 3.6, which run the automatic parts of these tests on every PR, unless a test names a narrower trigger.

Reading it uses the techniques in Table 6. Dynamic tests only show behaviour on the inputs they choose. Reading finds paths that no test reaches, and checks what no test can, such as naming, hard-coded constants and the mapping to the MG and MIS. The safety inspection (SI, Section 4.2.2) covers the code tied to the safety-critical requirements and is not repeated here.

Table 6: Static verification of the implementation

Technique	**Who**	**When**	**What is read**	**Checklist**
PR code review	One teammate other than the author, normally the one who checks the author's work in Table 2. Database schema and access-policy changes are reviewed by the Backend Lead (Rayan Nasrallah) together with a teammate (Development Plan).	Every PR, before merge.	The changed code and the PR's unit tests.	Items marked PR in Table 7. The reviewer reads the unit tests against the rules in Section 5.

Technique	Who	When	What is read	Checklist
Code walk-through	The author presents, and the rest of the team follows and asks questions. The Testing Lead (Muhammad Huzaifah) chooses the code and the scenario. The notetaker (Moustafa Moustafa) records findings in the meeting issue.	45 minutes of the Sunday 10:00 team review on 22 November 2026, 6 December 2026, 24 January 2027, 21 February 2027 and 7 March 2027.	At most two modules, the ones changed most since the last walk-through, followed along one scenario.	Items marked W.
Code inspection (INS-LRN, INS-EVAL, INS-REC, INS-CRT)	The Testing Lead moderates. Two inspectors who did not write the code read it. The author answers questions.	Completed by 20 November 2026 (PH-1), 29 January 2027 (PH-2) and 12 March 2027 (PH-3).	The lesson rules; stroke capture and evaluation; record saving; the refinement result check and drawing history.	Items marked I, plus A1 to A11.
Safety inspection (SI)	Rayan runs it and Huzaifah records.	Before Rev 0 and Rev 1.	The code tied to the SAF requirements.	The SI steps in Section 4.2.2.

Table 7 shows how reviewers apply each item of the team's Code Checklist to Squigglr. The "Used in" column names the techniques that check the item: PR review (PR), walkthrough (W) and inspection (I).

Table 7: Code Checklist items and how reviewers apply them

#	Item	Used in	What the reviewer checks in Squigglr
1	Identifier names are consistent	PR, W, I	Names match the SRS and MIS. The stage names are demonstration, full outline tracing, endpoint-only guidance and independent drawing, as in SRS FR-LRN-002. Constants are named exactly as in SRS Section 4.2 (for example `STAGE_PASS`). Record fields are named as in SRS Section 7.2.
2	Identifier names are meaningful	PR, W, I	A short name such as `nf` is allowed only with a comment citing the SRS Section 9.2 symbol.
3	Source code in the src folder	PR	Application code is under `src/`. Each unit test sits next to its module as `<module>.test.ts`. End-to-end tests and fixtures are kept in `tests/` (fixtures in `tests/fixtures/`).
4	Comments on "what" not "how"	PR, I	Each exported function of the lesson rules, evaluation, record, refinement check and drawing history code states what it guarantees and cites its SRS requirement (for example "FR-LRN-021").
5	Avoid hard-coded constants	PR, I	No literal value of an SRS constant, such as 80, 8 or 5. Values come from the learner's settings, or from one constants module that uses the SRS names.
6	Consistent indentation	PR	The Prettier check passes; this item is not checked by hand.
7	Explicit identification of coding standard	PR, once	The ESLint and Prettier configuration files are committed, and the README links the Development Plan coding standard.

#	Item	Used in	What the reviewer checks in Squigglr
8	Code standard is followed	PR, I	ESLint and `tsc` pass. The reviewer checks what the tools cannot: `camelCase` and `PascalCase`, and a justification next to every `any` or lint suppression.
9	Parameters in the same order for all functions	PR, I	Learner, lesson, stage, attempt, in that order, everywhere.
10	Descriptive file names	PR	Each file is named after what it does.
11	Mapping to the module guide is clear	W, I, from the design documentation (18 January 2027)	Each inspected file belongs to one MG module, and the mapping is kept in the repository.
12	Mapping between MIS symbols and code symbols	W, I, from the design documentation (18 January 2027)	Each MIS access program of the inspected module has a function of the same name, with the same inputs and outputs.
13	Understandable to someone new to the project	W	Someone who did not write the code can explain the scenario's path after the walkthrough.
14	Esoteric language features avoided	PR, W	A simpler construct is used wherever one would do.
15	Incomplete code flagged	PR, I	Incomplete code is marked `TODO(#issue)` with an open issue, and no `TODO` remains in inspected code at a milestone.

Items added for the inspections. The Code Checklist asks for more items, so the inspections add these:

A1 (INS-LRN) Each row of the SRS Section 9.2 transition table maps to one identifiable code path. After a finished attempt, the rules are checked in the SRS order: pass, end at limit, extension, suggestion. In three places SRS Section 9.2 disagrees with other parts of the SRS: a go-back suggestion pending during practice, the counts when a stage is entered again, and when an extended lesson ends. The inspector records which reading the code follows and raises it as an SRS feedback issue, not as a major finding. The guided lesson tests avoid these cases.

A2 (INS-LRN) Apart from pass-based advancement and the step from the demonstration to practice, only an adult action that passed the adult check changes the lesson or the stage (I1, I2 and SRS CON-007).

A3 (INS-LRN, INS-EVAL) An interrupted (not finalized) or invalid attempt cannot reach the pass branch or change a count (SRS FR-LRN-009 and FR-LRN-010). An incomplete attempt cannot pass, whatever its result, but counts as a failed attempt (I3, FR-LRN-011 and FR-LRN-013).

A4 (INS-LRN) On every path, the evaluable attempts at a stage never exceed `ATTEMPT_LIMIT_MAX`, the extension attempts used never exceed `ATTEMPT_EXTENSION`, and the extension is granted at most once (I5).

A5 (INS-EVAL) Evaluation and saving use the strokes as captured, with no smoothing or refinement, and the lesson code imports nothing from the refinement code (I4, SRS BR-004 and BR-005).

A6 (INS-EVAL) Feedback and completion text come only from the lesson catalogue data, never from code literals or a model (FR-LRN-014, FR-LRN-022 and NFR-MAINT-001).

A7 (INS-REC) An attempt record copies the adult-set values in use instead of pointing at the current settings (FR-REC-004), and no code path updates a saved attempt record or an ended lesson record (FR-REC-009).

A8 (INS-REC) A retried save cannot create a second record, and each attempt is saved under the lesson record it was made in (FR-REC-008).

A9 (INS-CRT) A refinement result is checked as a whole before any of it is applied. It is applied only if it changes nothing but the paths of existing strokes and keeps every refined stroke within `REFINE_TOLERANCE` of its original; a failed, rejected or out-of-date result leaves the drawing unchanged (SRS FR-CRT-011, FR-CRT-012, FR-CRT-020 and FR-CRT-021; H-MUH-002). A stroke that an earlier refinement changed is not refined again unless the child edited it since (FR-CRT-014).

A10 (INS-CRT) Every edit, refinement and restore is one undoable change in the drawing history. The original strokes of each refinement are kept apart and never overwritten, also when the drawing is saved and reopened, so a restore returns them exactly (FR-CRT-007, FR-CRT-017, FR-CRT-018 and FR-CRT-024; H-MUH-001).

A11 (all) Every branch of the inspected files is reached by a test in this plan or a unit test, or the gap is listed from the coverage report (Section 3.6).

What each inspection covers.

- INS-LRN: the lesson rules (FR-LRN-009, FR-LRN-010, FR-LRN-015, FR-LRN-016, FR-LRN-018 to FR-LRN-021 and FR-LRN-023; SRS Section 9.2).
- INS-EVAL: stroke capture, finishing an attempt, classification, the pass decision and the choice of feedback (FR-LRN-007 to FR-LRN-014, FR-LRN-022 and FR-REC-002).
- INS-REC: creating and saving attempt and lesson records (FR-REC-001 and FR-REC-003 to FR-REC-010).
- INS-CRT: checking refinement results, and the creative drawing history with undo, redo and restore (FR-CRT-007, FR-CRT-008, FR-CRT-011, FR-CRT-012, FR-CRT-014, FR-CRT-017 to FR-CRT-021 and FR-CRT-024).

PH-1 inspects INS-EVAL and INS-LRN for the PH-1 code. INS-REC starts in PH-2, when FR-REC-004 to FR-REC-010 are built; in PH-1, FR-REC-001 and FR-REC-003 rely on PR review and on T-REC-001 and T-REC-003. INS-CRT also starts in PH-2, when FR-CRT-014, FR-CRT-019, FR-CRT-021 and FR-CRT-024 complete refinement and restore; in PH-1, the creative drawing code relies on PR review and on the T-CRT tests. PH-2 inspects all four in full, with items 11 and 12, because the MG and MIS are done on 18 January 2027. PH-3 inspects only the code changed since the commit of the PH-2 inspection.

Inspection procedure.

1. *Planning* (moderator, two working days before the meeting): open an issue labelled `review`. The issue names the files and commit, the SRS requirements, the two inspectors and the checklist items. The inspectors are chosen from people who did not write the files, according to the commit history. For INS-REC, one of them is the Backend Lead unless he wrote the code. If the Testing Lead wrote any of the files, the milestone QA coordinator moderates instead.

2. *Preparation* (each inspector alone, at most one hour): check the code against every item, and list each finding in the issue with its file, line and item.

3. *Meeting* (at most one hour; moderator, inspectors and author): go through the findings and mark each one major or minor. A finding is major if it could change a pass decision, a count, a stage, a saved record or what the child or adult sees, or if it breaks one of A1 to A10. A disagreement inside the SRS named in A1 is not major; it becomes an SRS feedback issue. Every other finding is minor. The author answers questions but does not argue findings.

4. *Rework* (author): each major finding becomes its own issue labelled `bug`, linked from the inspection issue and fixed by a PR. Each minor finding is fixed, or closed with a reason.

5. *Follow-up* (moderator): check every major fix. If there were three or more major findings, inspect the changed code again before the milestone.

The inspection passes when no major finding is open on its date. When a file is in both an inspection and the safety inspection at the same milestone, the team holds one meeting: the SI steps first, then A1 to A11.

Walkthrough procedure. The author shares the code on screen and follows one scenario from input to saved result, for example a failing attempt from the stylus events, through evaluation and the lesson rules, to the saved attempt record. Listeners say where they cannot follow (item 13), and findings become GitHub issues. The walkthroughs on 6 December 2026 and 21 February 2027 prepare for the course code-review interviews on 9 December 2026 and 23 February 2027 (Development Plan). Findings from those interviews are recorded the same way. From 24 January 2027, walkthroughs also check items 11 and 12.

Feasibility and reporting. An inspection takes about seven person-hours: two hours of preparation for the two inspectors, a one-hour meeting for four people, and one hour of planning and follow-up. The ten inspections (two in PH-1, four in PH-2 and four smaller ones in PH-3) take about 60 person-hours, or about twelve for each member over the year. Walkthroughs use time in the existing weekly meeting. The VnV Report lists each inspection and walkthrough with its date, commit, participants, findings by checklist item and severity, and status.

3.6 Automated Testing and Verification Tools

This section names the tools that check the code automatically. They follow the expected technology and coding standard in the Development Plan. They are planned, not yet installed; the DevOps Lead (Shamil Canbolat) sets them up when the application scaffold exists, and the Testing Lead (Muhammad Huzaifah) chooses which tests run where. Table 8 lists each tool, what it checks and when it runs.

Table 8: Automated testing and verification tools

Purpose	**Tool**	**How and when it is used**
Language	TypeScript, strict mode	Next.js front end and server code. `tsc --noEmit` runs locally and on every PR; any type error fails the check.
Coding standard and linter	ESLint with the Next.js Core Web Vitals and TypeScript ESLint configurations	The coding standard in the Development Plan. Runs on every PR; any error fails the check. Used as evidence for NFR-MAINT-004.
Formatter	Prettier (check mode)	Runs on every PR; any unformatted file fails the check (NFR-MAINT-004).
Unit and component tests	Vitest with React Testing Library	Unit tests for logic such as stroke evaluation and save handling, and component tests for screens. Runs locally before review and on every PR; any failing test fails the check.
Code coverage	`@vitest/coverage-v8`	Produces a statement and branch coverage report on every PR (see below).
End-to-end tests	Playwright, on Chromium, WebKit and Firefox	Complete user flows, including the automated tests in the nonfunctional system tests (for example network loss and repeated failed attempts). Runs on every PR once a flow exists. Emulation does not replace physical stylus testing, so stylus tests are also run by hand on the test iPad.
Continuous integration	GitHub Actions	Runs all checks above on PRs to `development` and `main` and after each merge. A PR cannot be merged while a required check fails. The existing `latex-pages.yml` workflow continues to build the documents.

Purpose	Tool	How and when it is used
Performance measurement	Safari Web Inspector (Timelines), Chrome DevTools (Performance panel) and the browser Performance API (`performance.now()`)	Measures stroke latency, refinement time and page load for the performance tests. Safari Web Inspector, run on a Mac, reads timings from the test iPad; Chrome DevTools is used for runs on a laptop. Timings logged by Playwright scripts make the measurements repeatable.
Network inspection	Chrome DevTools (Network panel)	Records requests for the HTTPS and third-party data tests (NFR-OPS-001, SAF-SHA-002).

Code coverage. The coverage report is attached to every PR as a GitHub Actions artifact, and the change in coverage is noted in the PR review. The team does not set a fixed coverage target, because high coverage alone does not show that tests are good (Development Plan). Instead, reviewers check that new or changed logic has tests, and that modules tied to a safety-critical requirement (the safety-critical tests below), such as attempt saving, stroke evaluation and access checks, have every branch covered. Full branch coverage of these modules is a goal, not a guarantee: some branches (for example, a server failure in the middle of a request) may be hard to reach in a test. The actual coverage is reported honestly in the VnV Report as a summary table of statement and branch coverage per module, and any safety-critical branch left uncovered is listed with the reason and the test or inspection that covers it instead.

Limits of the tools. Passing these checks shows that the code builds, follows the coding standard and passes its tests. It does not show that the tests are complete, so the code inspections planned under Implementation Verification remain necessary.

3.7 Software Validation

[If there is any external data that can be used for validation, you should point to it here. If there are no plans for validation, you should state that

here. —SS]

[You might want to use review sessions with the stakeholder to check that the requirements document captures the right requirements. Maybe task based inspection? —SS]

[For those capstone teams with an external supervisor, the Rev 0 demo should be used as an opportunity to validate the requirements. You should plan on demonstrating your project to your supervisor shortly after the scheduled Rev 0 demo. The feedback from your supervisor will be very useful for improving your project. —SS]

[For teams without an external supervisor, user testing can serve the same purpose as a Rev 0 demo for the supervisor. —SS]

[This section might reference back to the SRS verification section. —SS]

4 System Tests

These tests check Squigglr as a whole against the requirements in the SRS. Section 4.1 covers the functional requirements and Section 4.2 the nonfunctional ones, and Section 4.3 traces every test back to its requirement. Tests of individual modules come later, once the design is done (Section 5).

Every test has an ID `T-<AREA>-###`, where `AREA` is the requirement group it tests, and the same fields: Purpose, Type, Initial State, Input, Output, Test Case Derivation and How test will be performed. Values the SRS leaves open are written as symbolic constants, and the Symbolic Parameters appendix gives each value or says how it will be set.

When one rule needs several boundary cases, like a setting at its minimum, default and maximum, we write one parameterized test instead of a separate test for each case. Its input is a table with one row per case. Each row has a letter and its own expected output, and the derivation says why each output is right, so a single case can be cited as, for example, `T-LRN-005(c)`. Our TA, Chris, agreed that this counts as one specific test as long as it is easy to follow.

4.1 Tests for Functional Requirements

These tests cover the functional requirements in SRS Section 9, grouped the same way the SRS groups them: guided lessons (FR-LRN, SRS Sections 9.1

and 9.2), creative drawing (FR-CRT, Section 9.3), attempt records (FR-REC, Section 9.4) and progress summaries (FR-PRG, Section 9.5). Each test starts from prepared input, like a stroke sequence, a drawing or an attempt history, so its expected output is known before it runs. The lesson tests use the formal specification in SRS Section 9.2 to work out the expected state after each step.

4.1.1 Guided Lessons

These tests cover the 23 guided lesson requirements in SRS Section 9.1 (FR-LRN-001 to FR-LRN-023) and the rules and invariants of the formal specification in SRS Section 9.2. The lesson flow is summarized with the safety-critical tests (Section 4.2.2). That summary follows SRS UC-005, where the child keeps practising while a suggestion waits. In SRS Section 9.2, an attempt can only be finished while no suggestion is pending, so in these tests an adult answers every suggestion before the next attempt (T-LRN-016). FR-LRN-001 to FR-LRN-007, FR-LRN-010, FR-LRN-011, FR-LRN-013, FR-LRN-015, FR-LRN-016, FR-LRN-022 and FR-LRN-023 are due in PH-1, and the rest in PH-2. Each test first runs in the phase of the requirements it covers. After that, its automatic parts run in CI on every PR, unless the test names a narrower trigger, and its manual parts run at the milestones the test names. The tests try to cause the failures in Hazard Analysis rows H-AHM-001 to H-AHM-007: lost or doubled strokes, an incomplete drawing that passes, the wrong feedback, a suggestion at the wrong time, and a stale or wrong learner's setting. H-AHM-003, a wrong coordinate mapping, is tested by T-SAF-010. Every drawing is made by an adult team member, and no child takes part (SRS CON-005 and CON-006).

Conventions. The tests below share these conventions.

- *Test lessons.* The tests use the line, circle and square lessons of the catalogue. The rules are the same for every lesson. The tests of FR-LRN-001 to FR-LRN-006 (T-LRN-001 to T-LRN-004) run on every catalogue lesson, except cases (e) and (f) of T-LRN-002, which use the three test lessons.

- *Defaults.* The adult-set values are at their SRS defaults unless a row says otherwise: `STAGE_PASS` 80, `DIFFICULTY_STREAK` 4, `ATTEMPT_LIMIT`

8, IMPROVE_WINDOW 2, ATTEMPT_EXTENSION 2 and ATTEMPT_LIMIT_MAX 10 (SRS Section 4.2). IMPROVE_MARGIN is fixed at 5.

- *Finishing and adult check.* "The finishing action" is whatever control the design gives for ending an attempt (Finalize in SRS Section 9.2). Every prepared attempt ends with the finishing action unless a row says otherwise. "The adult check" is ADULT_CHECK, as in the safety-critical tests.

- *How a sequence is written.* A number is one complete attempt with that stage result, out of 100. 95i is an incomplete attempt with result 95. inv is invalid input (a file from T-LRN-008). int is an attempt cut off by a pointercancel event halfway through its stroke, after which practice continues at the same stage. "6×40" means six attempts at 40. "[D]" after an attempt means a suggestion appears and the adult declines it at once, with the adult check. "[A→T]" means the adult accepts and chooses stage T. The stages are D, T, E and I (demonstration, tracing, endpoints only and no guide), as in SRS Section 9.2.

- *Two ways to run the rule tests.* The rule tests (T-LRN-011, T-LRN-013, T-LRN-016 to T-LRN-018 and T-LRN-020) run in two ways:

  1. in Vitest against the module that applies the lesson rules, with the completeness and result of each attempt given directly; this is the only way to set exact boundary values such as 44 or 45;
  2. end to end in Playwright on the real interface, for the rows named in each test. There, a written 40, 50 or 85 stands for a failing attempt, a higher failing attempt or a passing attempt, and is replayed as circle-fail.json, circle-near-miss.json or circle.json, unless the test says otherwise. Playwright-only tests that write 40 or 85 use the same mapping.

  The replayed files do not score the written numbers: their results lie in the bands of the CAL_* constants in Table 45 (at most 60, 70 to 79 and at least 90). These bands, not the written numbers, give the needed pass, fail and improvement results end to end.

- *Switching learner ends the lesson.* When the adult switches learner or ends the session, the current learner's lesson stops. The lesson is

recorded as unfinished (T-REC-007), and a later lesson for that learner is a new lesson record.

- *The list of results H.* In SRS Section 9.2, H holds the results of the evaluable attempts at the current stage. This is the state definition; the transition table does not append to H. The tests, and the model of T-LRN-021, follow the state definition.

1. T-LRN-001: Lesson catalogue

   Purpose: an adult can only offer agreed lessons of lines, basic shapes and simple drawings, and each lesson must work as a guided lesson.

   Type: Static (inspection) and dynamic, automatic. Covers FR-LRN-001.

   Initial State: the lesson catalogue data at the milestone release; an adult logged in with one child profile.

   Input: (a) a Vitest test loads every catalogue entry; (b) Playwright opens the adult's lesson list and starts each entry; (c) a team member who did not write the catalogue reads every entry.

   Output: (a) every entry has a category of line, basic shape or simple drawing, and the four stages D, T, E and I in that order; no entry is a letter or depends on one. (b) Every entry is listed, can be chosen and opens at its demonstration. (c) The reviewer finds no entry outside the three categories and no letter activity needed to finish the catalogue.

   Test Case Derivation: the fit criterion of FR-LRN-001. Letters are deferred (SRS ASM-010 and LC-001). The stages are those of FR-LRN-002.

   How test will be performed: (a) runs in CI on every PR that changes the catalogue. (b) runs at each milestone. (c) runs at PH-1, Rev 0 and Rev 1, and problems become GitHub issues.

2. T-LRN-002: Stage order and the adult's starting stage

   Purpose: practice must begin at the stage the adult chose, and the stages must come in the agreed order of less and less help.

Type: Dynamic, automatic and manual. Covers FR-LRN-002 and FR-LRN-016.

Initial State: learner A, plus learner B for case (f), each with no lesson records, on one device.

Input: the cases vary the adult's starting choice and the child's actions; the lesson rules and the defaults stay fixed. Cases (a) to (d) run for every catalogue lesson, and cases (e) and (f) for the line, circle and square lessons. "Good drawing" means the lesson's good-drawing fixture: `line.json`, `circle.json` or `square-4-strokes.json` for the test lessons. "Failing drawing" means `line-short.json`, `circle-fail.json` or `square-missing-side.json`, according to the lesson. The cases are in Table 9.

Output: the expected output of each case is in its row of Table 9; the test passes when every case gives it.

Test Case Derivation: (a) and (d) follow the order D, T, E, I of FR-LRN-002 and SRS UC-003. (b) and (c) follow FR-LRN-016 and SRS UC-004, with the counts at zero (SRS UC-004 step 4). Whether the demonstration comes first at a later start is not checked, because SRS Section 9.2 and SRS UC-003 disagree on it. (e) is the fit criterion's "child-only practice action does not change the configured lesson or starting stage". (f) is H-AHM-007, the wrong learner's setting.

How test will be performed: Playwright reads the visible stage and the lesson record. A tester repeats (a) to (c) with the stylus on the test iPad at each milestone.

3. T-LRN-003: The demonstration is not an attempt

Purpose: watching the example must not count as practice, or the records would overstate what the child did.

Type: Dynamic, automatic. Covers FR-LRN-003.

Initial State: learner A with no lesson records; each catalogue lesson started with the default start.

Input: the cases vary what happens during and after the demonstration; the learner and the defaults stay fixed. The cases are in Table 10.

Table 9: Cases for T-LRN-002 (FR-LRN-002 and FR-LRN-016)

Case	Adult's choice	Actions	Expected output
a	Default start	Watch the demonstration, then continue	D is shown first, then T with the full outline
b	Start at E, with the adult check	Practice begins	E is the first drawing stage shown, with its counts at zero
c	Start at I, with the adult check	Practice begins	I is the first drawing stage shown
d	Default start	Good drawing, three times	The stages appear as D, T, E, I, then the lesson is completed
e	Start at E	The child makes a failing drawing, finishes it, retries, taps the avatar and taps the stage indicator	The lesson is unchanged, the stage shown is still E, and the lesson record's starting stage is E
f	A at E, B at T, same lesson	The adult switches A, then B, then A. On each switch, the adult starts the lesson again with that learner's stage, with the adult check	The stage shown is the one just selected for that learner, and each lesson record holds its learner's starting stage (E, T, E)

Table 10: Cases for T-LRN-003 (FR-LRN-003)

Case	Actions	Expected output
a	Watch the demonstration to its end	The lesson's drawing is shown being drawn; the lesson record has zero attempts and no passed stage
b	During the demonstration, replay `circle.json` on the drawing area	No attempt is evaluated or recorded; the stage is still D
c	Continue to T, then one attempt at 40	The lesson record has exactly one attempt, at T
d	The adult ends the session during the demonstration	No attempt record

Output: the expected output of each case is in its row of Table 10; the test passes when every case gives it.

Test Case Derivation: (a) and (c) are the fit criterion. D has no scored attempts (SRS Section 9.2: "Only levels in A admit scored drawing attempts"). In (b), strokes during the demonstration try to sneak in an attempt. In (d), there is no stroke, so there is no record (FR-REC-005).

How test will be performed: Playwright, reading the lesson record from the test database after each case.

4. T-LRN-004: Guidance shown at each drawing stage

Purpose: each stage must show exactly the help it promises, so that less help really means less help.

Type: Dynamic, automatic and manual. Covers FR-LRN-004, FR-LRN-005 and FR-LRN-006.

Initial State: each catalogue lesson opened at T, E and I in turn, with the adult check. The drawing area is `TEST_CANVAS`.

Input: the cases vary the stage; the lesson's good drawing stays fixed (`line.json`, `circle.json` or `square-4-strokes.json` for the test lessons, and the good-drawing fixture of any other catalogue lesson). The cases are in Table 11.

Table 11: Cases for T-LRN-004 (FR-LRN-004 to FR-LRN-006)

Case	**Stage**	**Expected guide**	**Then**
a	T	The complete outline is visible	The lesson's good drawing is accepted and evaluated
b	E	The lesson's start and end points are visible; the outline is not	The same as (a)
c	I	Neither the outline nor the points are visible	The same as (a)

Output: the expected output of each case is in its row of Table 11; the test passes when every case gives it.

Test Case Derivation: (a), (b) and (c) are the fit criteria of FR-LRN-004, FR-LRN-005 and FR-LRN-006, each with a stylus attempt that is accepted.

How test will be performed: before drawing, Playwright compares a screenshot of the empty drawing area with a reference screenshot for each lesson and stage, approved once by the Testing Lead (Playwright `toHaveScreenshot`). Playwright also checks that the guide layers present match the stage. At each milestone, a tester draws at each stage on the test iPad with the stylus and confirms what is visible.

5. T-LRN-005: Strokes kept in order, with their boundaries

   Purpose: evaluation and records need the strokes exactly as drawn. A lost, merged or doubled stroke changes the result.

   Type: Dynamic, automatic. Covers FR-LRN-007.

   Initial State: the circle lesson at T (the square lesson for case (b)); the drawing area is `TEST_CANVAS`.

   Input: the cases vary the replayed file; the stage (T), the drawing area and the defaults stay fixed. The cases are in Table 12.

Table 12: Cases for T-LRN-005 (FR-LRN-007)

Case	**Replayed file**	**Expected captured strokes**
a	`line.json` (one stroke)	One stroke with the file's points, in order
b	`square-4-strokes.json`	Four strokes; each stroke boundary at the same point as in the file
c	`overlap-2-strokes.json`	Two strokes, not merged where they cross
d	`line-twice.json` (two separate strokes along the same line)	Two strokes; neither is removed as a duplicate
e	`circle-fast.json` (a circle drawn in under half a second)	The same number of points as the file; none dropped

   In every case, each captured point equals the file's point to within 0.01 CSS pixels, with the same pressure and time.

Output: the expected output of each case is in its row of Table 12; the test passes when every case gives it.

Test Case Derivation: (a) to (c) are the fit criterion's one-stroke and multi-stroke samples. (d) follows H-AHM-002, which asks that legitimately repeated strokes survive duplicate removal. (e) is H-AHM-001, missed pointer samples. The 0.01-pixel allowance covers floating-point storage only.

How test will be performed: Playwright replays each file and reads the captured strokes from the saved attempt record. A Vitest test sends the same event lists to the input handler directly.

6. T-LRN-006: Several strokes make one attempt

Purpose: lifting the stylus between strokes must not end the attempt, or the first side of a square would be judged as a whole square.

Type: Dynamic, automatic. Covers FR-LRN-008.

Initial State: the square lesson at T, with zero attempts.

Input: the cases vary the number of strokes and the pauses between them; the square lesson, the stage (T) and the defaults stay fixed. The cases are in Table 13.

Table 13: Cases for T-LRN-006 (FR-LRN-008)

Case	Actions	Expected output
a	`square-4-strokes.json`, with `STROKE_PAUSE` between strokes, then the finishing action	Exactly one attempt evaluated and recorded
b	`square-2-strokes.json` (two L-shaped strokes), then the finishing action	One attempt, classified complete
c	The first stroke of `square-2-strokes.json`, then three times `STROKE_PAUSE` with the stylus lifted	No outcome shown and the attempt count still zero; after the second stroke and the finishing action, one complete attempt

Output: the expected output of each case is in its row of Table 13; the test passes when every case gives it.

Test Case Derivation: the fit criterion ("one attempt is evaluated against the complete figure, not two"). In SRS Section 9.2, a lift closes a stroke, not the attempt. (c) is the H-AHM-001 cause in which "a stylus lift is mistaken for the end of an attempt".

How test will be performed: Playwright counts the attempt records and the evaluation results.

7. T-LRN-007: Interrupted attempts

Purpose: an attempt cut off before it is finished must never pass, advance the child or use up an attempt.

Type: Dynamic, automatic. Covers FR-LRN-009.

Initial State: learner A at T of the circle lesson (the square lesson for cases (b) and (c)), with zero attempts; learner B also exists.

Input: the cases vary how and when the attempt is interrupted; the stage (T) and the defaults stay fixed. The cases are in Table 14.

Table 14: Cases for T-LRN-007 (FR-LRN-009)

Case	**Interruption**	**Expected output**
a	A `pointercancel` event halfway through the stroke of `circle.json`	No outcome; stage T; attempt counts zero
b	The first two strokes of `square-4-strokes.json`, then the adult ends the session (adult check)	No outcome; stage T; counts zero; the attempt shows as interrupted, apart from finished attempts
c	The same two strokes, then the adult switches to learner B	As (b); the interrupted attempt is saved under A, and nothing under B
d	All of `circle.json` (which passes when finished) without the finishing action, then a page reload	Not passed; stage T; counts zero
e	Half of `circle.json`, then the page is hidden for `STROKE_PAUSE` and shown again	No outcome; counts zero

Output: the expected output of each case is in its row of Table 14; the test passes when every case gives it.

Test Case Derivation: every case follows SRS Section 9.2 ("Interrupted or invalid input: do not pass or change level; neither count is incremented") and the fit criterion, which interrupts both between and during strokes. (d) uses a drawing that would pass, so a pass could only come from the interruption. The interrupted mark in (b) is FR-REC-005 (T-REC-005).

How test will be performed: Playwright, reading the stage and the counts from the lesson state and the records.

8. T-LRN-008: Invalid and ambiguous input

   Purpose: input that cannot be read reliably must not pass and must not use up an attempt.

   Type: Dynamic, automatic. Covers FR-LRN-010.

   Initial State: the circle lesson at T, with zero attempts.

   Input: the cases vary the kind of unreadable input, what comes around it and `ATTEMPT_LIMIT`; the circle lesson, the stage (T) and the other settings stay fixed. The cases are in Table 15.

   Output: the expected output of each case is in its row of Table 15; the test passes when every case gives it.

   Test Case Derivation: (a) to (c) are the three inputs the fit criterion names. (d) tries to make invalid input count toward the limit (H-AHM-006). (e) checks that invalid input leaves no trace. Input that cannot be mapped to the drawing area is covered by T-SAF-010.

   How test will be performed: Vitest sends each file to the input handler and the evaluation. Playwright replays (a) to (e), with 40 replayed as `circle-fail.json`.

9. T-LRN-009: Complete or incomplete

   Purpose: a drawing with a missing part must be classified incomplete, even when the parts that are present are accurate.

   Type: Dynamic, automatic and manual. Covers FR-LRN-011.

   Initial State: the line, circle and square lessons, each evaluated at T, E and I.

Table 15: Cases for T-LRN-008 (FR-LRN-010)

Case	`ATTEMPT_LIMIT`	**Input**	**Expected output**
a	8	`empty.json` (the finishing action with no strokes)	Not passed; a status that is neither pass nor fail is shown; counts unchanged
b	8	`nan-points.json` (one point in ten has no coordinates)	As (a)
c	8	`order-unknown.json` (two strokes with identical times)	As (a)
d	6	5×40, then the inputs of (a), (b) and (c), then 40	The lesson continues through (a) to (c) and ends only after the final 40, the sixth evaluable attempt
e	8	The input of (b), then `circle.json`	`circle.json` is the first evaluable attempt (count 1) and passes

Input: the cases vary the lesson and the drawing; the defaults stay fixed, and each case runs at all three drawing stages. The cases are in Table 16.

Table 16: Cases for T-LRN-009 (FR-LRN-011)

Case	Lesson	File	Expected classification
a	Each	Its good drawing (`line.json`, `circle.json`, `square-4-strokes.json`)	Complete
b	Square	`square-missing-side.json`	Incomplete
c	Circle	`circle-gap.json` (about a quarter not drawn)	Incomplete
d	Line	`line-short.json` (stops about halfway)	Incomplete
e	Line	`line-wrong-end.json` (ends well away from the end point)	Incomplete
f	Each	One complete and one missing-part drawing made by hand on the test iPad by each of two team members	The class the drawer intended

Output: the expected output of each case is in its row of Table 16; the test passes when every case gives it.

Test Case Derivation: the fit criterion asks for complete examples and examples with a missing side, segment or required endpoint. (b) is the high-scoring incomplete example of H-AHM-004: its three accurate sides score at least `CAL_INCOMPLETE_MIN`, so its result is recorded and must not change the class. (f) is the fit criterion's "adult-made" examples.

How test will be performed: Vitest calls the evaluation function for each file, lesson and stage. Playwright replays (a) to (e) at each stage. (f) is done by hand at PH-1.

10. T-LRN-010: Assisted or independent

Purpose: adults must be able to tell work done with a guide from work done without one.

Type: Dynamic, automatic. Covers FR-LRN-012.

Initial State: learner A with the circle lesson.

Input: the cases vary the stage of each attempt and how the lesson started; the circle lesson and the defaults stay fixed. The cases are in Table 17.

Table 17: Cases for T-LRN-010 (FR-LRN-012)

Case	**Sequence**	**Expected label of each attempt**
a	One attempt at T	Assisted
b	One attempt at E	Assisted
c	One attempt at I	Independent
d	The adult starts at I; one attempt	Independent
e	At I: 4×40 [A→E], then one attempt at E	The four I attempts independent; the E attempt assisted
f	From T: 40, 85, 40, 85, 40, 85	Four assisted (the T and E attempts) and two independent (the I attempts)

Output: the expected output of each case is in its row of Table 17; the test passes when every case gives it.

Test Case Derivation: $G(l)$ in SRS Section 9.2 labels T and E assisted and I independent, from the stage of the attempt itself. So (d) and (e) check that the label follows the stage, not how the lesson started. In (f), the label belongs to each attempt, so one lesson holds both kinds (as T-SAF-015 notes).

How test will be performed: Vitest for (a) to (f). Playwright end to end for (f), with 40 replayed as `circle-fail.json` and 85 as `circle.json`.

11. T-LRN-011: Pass decision at the threshold

Purpose: a drawing passes only when it is complete and meets the learner's `STAGE_PASS`. A pass at the wrong time would move the child on too early.

Type: Dynamic, automatic. Covers FR-LRN-013.

Initial State: the circle lesson at T for one learner. The square lesson is used for case (j).

Input: the cases vary `STAGE_PASS` and the attempt; the circle lesson at T (the square lesson for case (j)) and the other settings at their defaults stay fixed. Cases (a) to (f) are each run with `STAGE_PASS` = 70, 80 and 90; cases (g) to (j) are end to end. The cases are in Table 18.

Table 18: Cases for T-LRN-011 (FR-LRN-013)

Case	`STAGE_PASS`	**Attempt**	**Expected output**
a	70, 80, 90	Complete, result `STAGE_PASS` − 1	Not passed
b	70, 80, 90	Complete, result `STAGE_PASS`	Passed
c	70, 80, 90	Complete, result `STAGE_PASS` + 1	Passed
d	70, 80, 90	Incomplete, result 100	Not passed
e	70, 80, 90	Incomplete, result `STAGE_PASS`	Not passed
f	70, 80, 90	`inv`	Not passed; not counted
g	90	`circle.json`	Passed
h	80	`circle-near-miss.json`	Not passed
i	70	`circle-near-miss.json`	Passed
j	70	`square-missing-side.json` (square lesson)	Not passed

Output: the expected output of each case is in its row of Table 18; the test passes when every case gives it.

Test Case Derivation: Pass(X, l, p) in SRS Section 9.2 requires $C =$ complete and $v \geq \theta_p$, which gives (a) to (c). These are the fit criterion's "below, at and above the adult-selected threshold", at both ends of the range and at the default. (d) and (e) follow invariant I3 and H-AHM-004. (f) follows FR-LRN-010. (g) to (i) follow from the calibration bands: `CAL_PASS_MIN` is 90, and `CAL_NEAR_MISS` is 70 to 79 with the near miss classified complete. (h) and (i) show that the learner's own value is used. In (j), the drawing scores at least `CAL_INCOMPLETE_MIN` (70), so only its incomplete class can stop the pass.

How test will be performed: Vitest runs (a) to (f) against the lesson rules. Playwright runs (g) to (j).

12. T-LRN-012: Predefined feedback for each difficulty

    Purpose: the feedback must name the difficulty that is actually in the drawing, using only the team's approved wording.

    Type: Dynamic, automatic. Covers FR-LRN-014.

    Initial State: the catalogue's approved feedback list for each lesson and stage.

    Input: the cases vary the lesson, the stage and the difficulty in the drawing; the settings stay at their defaults, and each file ends with the finishing action. The cases are in Table 19.

    In every case, the text shown equals the catalogue's text for that lesson, stage and category, and no request goes to an AI or model service during the lesson.

    Output: the expected output of each case is in its row of Table 19; the test passes when every case gives it.

    Test Case Derivation: the fit criterion asks for one prepared matching drawing for each category, and for no free-generated advice. (a) to (d) cover the three categories FR-LRN-014 names. (f) is the H-AHM-005 cause in which "the feedback lookup uses the wrong lesson or guidance stage". (e) checks that a good drawing gets no difficulty. SAF-MOU-001 (feedback does not draw) is tested by T-SAF-014.

    How test will be performed: Playwright reads the feedback text and compares it with the catalogue file. The network log is checked for requests to the refinement service or any model endpoint.

13. T-LRN-013: Retry before the limit

    Purpose: a child who does not pass must be able to try again, until the limit ends the lesson.

    Type: Dynamic, automatic. Covers FR-LRN-015.

    Initial State: the circle lesson at T with `ATTEMPT_LIMIT` = 6.

Table 19: Cases for T-LRN-012 (FR-LRN-014)

Case	Lesson, stage	File	Expected feedback
a	Circle, T	`circle-gap.json`	The circle T observation for a missed part, placed at the gap; no other observation
b	Square, E	`square-missing-side.json`	The observation for an incomplete shape
c	Line, I	`line-short.json`	The observation for an incomplete line
d	Line, E	`line-wrong-end.json`	The observation for an incorrect endpoint
e	Circle, T	`circle.json`	No difficulty observation; only the approved message for a finished attempt
f	Circle, I	`circle-gap.json`	The I-stage observation for a missing part, not the tracing one
g	Each	One prepared drawing for each other difficulty category in the catalogue, made when the category is added	That category's observation

Input: the cases vary the number and kind of non-passing attempts; the circle lesson, the stage (T) and `ATTEMPT_LIMIT` 6 stay fixed. The cases are in Table 20.

Table 20: Cases for T-LRN-013 (FR-LRN-015)

Case	Sequence	Expected output
a	40	A new attempt can start at T; the drawing area is cleared and the outline shown again
b	5×40	As (a)
c	6×40	The lesson ends, with no further attempt (T-LRN-018)
d	`95i`	As (a)
e	`inv`	As (a), with the attempt counts unchanged

Output: the expected output of each case is in its row of Table 20; the test passes when every case gives it.

Test Case Derivation: the fit criterion, before the limit and at it. (b) is the last attempt before `ATTEMPT_LIMIT`, and (c) reaches it (SRS Section 9.2, "End at limit"). In (d), an incomplete attempt is still a non-passing attempt.

How test will be performed: Vitest for every case; Playwright end to end for (a) and (c) with `circle-fail.json`.

14. T-LRN-014: Adult-set values are checked, defaulted and used

Purpose: each child's values must stay within the team's ranges, unset values must take the middle of the range, and lessons must use the values actually saved for that child.

Type: Dynamic, automatic. Covers FR-LRN-017.

Initial State: a new child profile with no values chosen; for case (l), learners A and B.

Input: in cases (a) to (f), one value is tried at a time at each of the values in its row, with the other five at their defaults. The minimum and maximum must be saved, leaving the value unset must save the value in the "Unset" column, and the values just outside the range must be refused. A refused value keeps the previous value and shows

a message to the adult. Cases (g) to (l) try combinations and the use of the saved values, each from a new child profile. The cases are in Table 21.

Table 21: Cases for T-LRN-014 (FR-LRN-017)

Case	Value	Min saved	Max saved	Unset saves	Below refused	Above refused
a	`STAGE_PASS`	70	90	80	69	91
b	`DIFFICULTY_STREAK`	3	5	4	2	6
c	`ATTEMPT_LIMIT`	6	10	8	5	11
d	`IMPROVE_WINDOW`	1	3	2	0	4
e	`ATTEMPT_EXTENSION`	1	3	2	0	4
f	`ATTEMPT_LIMIT_MAX`	8	12	10	7	13

Case	Input	Expected output
g	All six values unset	80, 4, 8, 2, 2 and 10 saved
h	`ATTEMPT_LIMIT` = 10, `ATTEMPT_LIMIT_MAX` = 9	Refused; nothing saved
i	`ATTEMPT_LIMIT` = 10, `ATTEMPT_LIMIT_MAX` = 10	Saved
j	`DIFFICULTY_STREAK` = 5, `ATTEMPT_LIMIT` = 6	Saved
k	`ATTEMPT_LIMIT` = 7.5; separately, `DIFFICULTY_STREAK` = “four”	Both refused
l	A at all minimums (70, 3, 6, 1, 1, 8) and B at all maximums (90, 5, 10, 3, 3, 12); the adult switches A, B, A, and each time the learner makes one `circle-near-miss.json` attempt at T	A’s attempts pass and B’s do not; each attempt record holds its own learner’s values

Output: the expected output of each case is in its row of Table 21; the test passes when every case gives it.

Test Case Derivation: the fit criterion asks for every setting at its minimum, default and maximum, plus one value outside each bound. The ranges are in SRS Section 4.2, and the defaults are their midpoints. (h) and (i) test `ATTEMPT_LIMIT_MAX` $\geq$ `ATTEMPT_LIMIT` at the bound-

ary. (j) is the closest pair that the ranges allow for `ATTEMPT_LIMIT` > `DIFFICULTY_STREAK`; within the ranges, that inequality cannot fail. (k) follows "Attempt count settings shall be integers" (SRS Section 4.2). (l) checks that "selected values are used", and H-AHM-007 (another learner's threshold); this works because `CAL_NEAR_MISS` lies between 70 and 90 and the near miss is classified complete. T-SAF-008 covers the adult check on changes, so that check is not repeated here.

How test will be performed: a Vitest test of the validation function runs every case. Playwright repeats (a) to (k) through the settings screen and through direct server requests, and runs (l) end to end.

15. T-LRN-015: Only an adult changes the lesson, skips or goes back

    Purpose: changing lesson, skipping a stage and going back are the adult's decisions. Only passing moves the child on by itself.

    Type: Dynamic, automatic. Covers FR-LRN-018.

    Initial State: unless a case says otherwise, learner A in the circle lesson at E with `DIFFICULTY_STREAK` = 3, just after 3×40, so a suggestion is pending. Each case starts from this state.

    Input: the cases vary who acts and through which path; the starting state and the settings stay fixed. The cases are in Table 22.

    Output: the expected output of each case is in its row of Table 22; the test passes when every case gives it.

    Test Case Derivation: the fit criterion asks for each discretionary change without and with authorization, plus pass-based advance with no adult action. The cases also follow invariants I1 and I2 in SRS Section 9.2. (d) and (e) try paths around the interface, as T-SAF-001 does for SAF-RAY-001.

    How test will be performed: Playwright. The server requests are sent with the child's session.

16. T-LRN-016: Go-back suggestion

    Purpose: the adult should get a suggestion to go back after a full run of failures, and not when the child is improving, has too few attempts, or has just declined one.

Table 22: Cases for T-LRN-015 (FR-LRN-018)

Case	Action	Expected output
a	The child picks the square lesson; at the adult check, the child leaves it, then enters a wrong answer	The circle lesson is still at E
b	The child tries to skip to I	The stage is still E
c	The child taps accept on the suggestion	The suggestion is still pending; stage E
d	Browser back, forward and reload	Lesson, stage and pending suggestion unchanged
e	The server requests behind (a) to (c), sent without a passed adult check	All refused; nothing changes
f	The adult completes the check and picks the square lesson	The square lesson starts with a new lesson record; the circle record ends as unfinished (T-REC-007)
g	The adult completes the check and skips to I	Stage I
h	The adult completes the check and accepts, choosing T	Stage T
i	The adult completes the check and declines	Stage E; the suggestion is cleared
j	The initial state is at T instead; one attempt at 85	Stage E; no adult check shown

Type: Dynamic, automatic. Covers FR-LRN-019.

Initial State: the circle lesson at the stage in each case's row.

Input: the cases vary the stage, three settings and the sequence of results; `STAGE_PASS`, `ATTEMPT_EXTENSION` and `ATTEMPT_LIMIT_MAX` stay at their defaults. The Settings column gives the three settings in the order of its header. The cases are in Table 23.

Output: the expected output of each case is in its row of Table 23; the test passes when every case gives it.

Test Case Derivation: SuggestBack in SRS Section 9.2 needs four things: the stage is E or I, the streak is at least `DIFFICULTY_STREAK`, the child is not improving, and no suggestion is pending.

- (a) to (d) are one below and at the minimum and maximum streak, at both eligible stages (the fit criterion).
- (e): T has no earlier drawing stage.
- (f): with `IMPROVE_WINDOW` 3, the improvement check needs six results, so three rising results do not delay the suggestion ("insufficient history does not delay the first suggestion").
- (g) to (j) compare the last window with the one before: in (g), $60 - 50 = +10$, then $60 - 60 = 0$; in (h), $55 - 50 = +5$, which is `IMPROVE_MARGIN`, and "at least" counts as improving; in (i), +4; in (j), $\text{mean}(45, 45) - \text{mean}(40, 40) = +5$, then $\text{mean}(45, 45) - \text{mean}(40, 45) = +2.5$.
- (k): a decline resets the streak, so a new suggestion needs a new full streak (FR-LRN-019 and SRS UC-005 step 2a). At the tenth failure, "End at limit" takes priority.
- (l) is the main flow of SRS UC-005; the furthest stage is kept.
- (m) is SRS UC-005 step 2b.
- (n) follows FR-LRN-009, FR-LRN-010 and the H-AHM-006 cause in which "invalid input is counted as failure".
- (o): a pass changes the stage and clears the streak.

In every case, the adult answers a suggestion before the next attempt, because SRS Section 9.2 and SRS UC-005 disagree on whether a pending suggestion blocks practice.

Table 23: Cases for T-LRN-016 (FR-LRN-019)

Case	Stage	DIFFICULTY_STREAK, IMPROVE_WINDOW, ATTEMPT_LIMIT	Sequence	Expected output
a	E	3, 2, 8	40, 40	No suggestion
b	E	3, 2, 8	40, 40, 40 [D]	Suggestion after the 3rd; stage E; cleared by the decline
c	I	5, 2, 8	4×40	No suggestion
d	I	5, 2, 8	5×40 [D]	Suggestion after the 5th; stage I
e	T	3, 2, 8	7×40	No suggestion at any point
f	E	3, 3, 8	40, 50, 60 [D]	Suggestion after the 3rd
g	E	3, 1, 8	40, 50, 60, 60 [D]	None after the 3rd; suggestion after the 4th
h	E	3, 1, 8	40, 50, 55, 55 [D]	None after the 3rd; suggestion after the 4th
i	E	3, 1, 8	40, 50, 54 [D]	Suggestion after the 3rd
j	E	4, 2, 8	40, 40, 45, 45, 45 [D]	None after the 4th; suggestion after the 5th
k	E	3, 2, 10	3×40 [D], 3×40 [D], 3×40 [D], 40	Suggestions after the 3rd, 6th and 9th only; the lesson ends after the 10th
l	E	3, 2, 8	3×40 [A→T], then 85	After the accept: stage T, no suggestion pending, furthest stage E; the 85 moves the child to E
m	E	3, 2, 8	3×40, then no answer and no input for STROKE_PAUSE	The suggestion is still pending; stage E
n	E	3, 2, 8	40, `inv`, 40, `int`, 40 [D]	Suggestion only after the 5th input (the 3rd evaluable attempt)
o	E	3, 2, 8	40, 40, 85, then at I 40, 40, 40 [D]	None at E; suggestion after the 3rd failure at I

How test will be performed: Vitest runs every case. Playwright repeats (b) with three `circle-fail.json`. Playwright also runs, at E with `DIFFICULTY_STREAK` 3 and `IMPROVE_WINDOW` 1, `circle-fail.json` twice and then `circle-near-miss.json`, which gives no suggestion because the improvement is at least 10, then one more `circle-near-miss.json`, which gives the suggestion.

17. T-LRN-017: Improvement extension

    Purpose: a child who is improving when the limit arrives gets a few more tries, once, and never more than the maximum.

    Type: Dynamic, automatic. Covers FR-LRN-020.

    Initial State: the circle lesson at T, where no suggestion can appear.

    Input: the cases vary four settings, given in the Settings column in the order of its header, and the sequence of results; `STAGE_PASS` stays at 80 and `DIFFICULTY_STREAK` at 4. Every case satisfies `ATTEMPT_LIMIT` + `ATTEMPT_EXTENSION` $\geq$ `ATTEMPT_LIMIT_MAX`, because SRS SAF-RAY-005 and SRS Section 9.2 give different endings otherwise. The cases are in Table 24.

    Output: the expected output of each case is in its row of Table 24; the test passes when every case gives it.

    Test Case Derivation: the rules are "Extend at base limit" and "End at limit" in SRS Section 9.2.

    - At the base limit, the improvement check compares the last `IMPROVE_WINDOW` results with the window before: +10 in (a) and (m), +5 in (c) (exactly `IMPROVE_MARGIN`, so it qualifies), +4 in (d), $\text{mean}(50, 50, 50) - \text{mean}(40, 40, 40) = +10$ in (k) and $45 - 40 = +5$ in (l).
    - After a grant, each evaluable attempt uses one extra attempt, and the lesson ends when the extra attempts run out or the total reaches `ATTEMPT_LIMIT_MAX`.
    - (e): the extension is not granted again, even while the child is improving.
    - (f): the cap (12) comes before the end of the extension (10 + 3).

Table 24: Cases for T-LRN-017 (FR-LRN-020)

Case	`ATTEMPT_LIMIT`, `ATTEMPT_EXTENSION`, `ATTEMPT_LIMIT_MAX`, `IMPROVE_WINDOW`	Sequence	Expected output
a	8, 2, 10, 2	6×40, 4×50	Extension after the 8th; ends after the 10th; extension count 1
b	8, 2, 10, 2	8×40	Ends after the 8th; no extension
c	8, 2, 10, 2	6×40, 4×45	Extension after the 8th; ends after the 10th
d	8, 2, 10, 2	6×40, 44, 44	Ends after the 8th; no extension
e	8, 2, 10, 2	6×40, 50, 50, 60, 70	Extension after the 8th; ends after the 10th, although still improving
f	10, 3, 12, 2	8×40, 4×50	Extension after the 10th; ends after the 12th (two of the three extra attempts)
g	10, 2, 10, 2	8×40, 50, 50	Ends after the 10th; no extension
h	6, 2, 8, 2	4×40, 4×50	Extension after the 6th; ends after the 8th
i	8, 2, 10, 2	6×40, 50, 50, 85	Extension after the 8th; the 9th passes and the stage becomes E
j	8, 2, 10, 2	6×40, 50, 50, `inv`, `int`, 50, 50	Extension after the 8th; ends after the 12th input (the 10th evaluable attempt)
k	8, 2, 10, 3	5×40, 5×50	Extension after the 8th; ends after the 10th
l	8, 2, 10, 1	7×40, 3×45	Extension after the 8th; ends after the 10th
m	8, 1, 9, 2	6×40, 50, 50, 50	Extension after the 8th; ends after the 9th

- (g): there is no room for an extension, because the attempt count already equals the maximum (the guard needs fewer evaluable attempts than the maximum).
- (i): a pass takes priority.
- (j): invalid and interrupted input do not use up the extension.
- (h) uses the minimum `ATTEMPT_LIMIT` and `ATTEMPT_LIMIT_MAX`, (m) the minimum `ATTEMPT_EXTENSION`, and (f) the maximum of all three, for the fit criterion's margin boundary, limit and no regrant.

How test will be performed: Vitest runs every case. Playwright repeats (a) as six `circle-fail.json` and four `circle-near-miss.json`, and (b) as eight `circle-fail.json`.

18. T-LRN-018: The attempt limit ends the lesson

    Purpose: a child must not keep repeating a stage they keep failing, and the furthest stage reached must be kept.

    Type: Dynamic, automatic. Covers FR-LRN-021.

    Initial State: learner A in the circle lesson at the start stage in each case's row.

    Input: the cases vary `ATTEMPT_LIMIT`, the start stage and the sequence; the other settings stay at their defaults. The cases are in Table 25.

    Output: the expected output of each case is in its row of Table 25; the test passes when every case gives it.

    Test Case Derivation: "End at limit" in SRS Section 9.2, at the minimum and maximum `ATTEMPT_LIMIT` in (a) and (b), at a later stage in (c) and from a later start in (d). In (e), the furthest stage is kept when the adult moves the child back ("retain the lesson's furthest stage reached"). E starts with no failures in (e), so SRS Section 9.2 and SRS BR-002 give the same ending, although they count failures at a re-entered stage differently. In (f), only evaluable attempts count. In (g), a pass takes priority over the limit. The extension cases are in T-LRN-017.

Table 25: Cases for T-LRN-018 (FR-LRN-021)

Case	`ATTEMPT_LIMIT`	Start	Sequence	Expected output
a	6	T	6 × 40, then `circle.json` replayed	Ends after the 6th; the replay is not accepted or recorded; furthest stage T
b	10	T	10 × 40	Ends after the 10th
c	8	T	85, then at E 4 × 40 [D], 4 × 40	Ends after the 8th failure at E; furthest stage E
d	8	The adult starts at I	4 × 40 [D], 4 × 40	Ends after the 8th; starting and furthest stage I
e	8	T	85, 85, then at I 4 × 40 [A→E], then at E 4 × 40 [D], 4 × 40	Ends after the 8th failure at E; furthest stage I
f	6	T	40, `inv`, 40, `int`, 40, 40, 40, 40	Ends after the 8th input (the 6th evaluable attempt)
g	6	T	5 × 40, 85	The 6th passes; stage E; the lesson continues

How test will be performed: Vitest runs every case. Playwright runs (a) and (c) with `circle-fail.json` and `circle.json`, reading the furthest stage from the lesson record.

19. T-LRN-019: Feedback and results describe the drawing, not the child

    Purpose: Squigglr is a practice tool, so its lesson messages must talk about the drawing and never label, diagnose or compare the child.

    Type: Static (inspection) and dynamic, automatic. Covers FR-LRN-022.

    Initial State: the catalogue feedback text at the revision, plus every lesson-end message: "every stage passed", "ended at `ATTEMPT_LIMIT`", "ended when a granted extension ran out or `ATTEMPT_LIMIT_MAX` was reached" and "unfinished". The message that tells the adult an extension was granted is added, if the design shows one.

    Input: (a) a team member who did not write the text reads every message, using `forbidden-terms.txt` and three questions: does the message refer to the task, stage or strokes? Does it label or diagnose the child? Does it compare the child with others? (b) Playwright collects every text shown during the runs of T-LRN-011 to T-LRN-020 and searches it for the terms.

    Output: (a) every message refers to the task, stage or strokes (for example "the top side is missing"), and none uses a listed term, an ability label, a diagnosis or a comparison with other children. (b) No match. A sentence that says a view is not an assessment or diagnosis, as SAF-RAY-004 requires for summaries, is not a finding.

    Test Case Derivation: the fit criterion (inspect all feedback and completion outcomes using prepared attempts). The terms come from:

    - Hazard Analysis H-MOU-002: "wrong", "bad", "fail" and "mistake";
    - T-SAF-004: "struggling", "behind" and "below average";
    - SRS NFR-UX-017: "diagnosis", "disorder", "delay", "score" and "rank";
    - ability and comparison words: "smart", "clever", "talented", "slow", "better than", "worse than" and "other children".

How test will be performed: (a) at PH-1, Rev 0 and Rev 1, and in PR review whenever catalogue text changes; findings become GitHub issues. (b) in CI.

20. T-LRN-020: Automatic advancement within the lesson

    Purpose: passing must move the child to the next stage with no adult step, never skip a stage, and never start another lesson.

    Type: Dynamic, automatic. Covers FR-LRN-023.

    Initial State: learner A in the circle lesson at the start stage in each case's row.

    Input: the cases vary the start stage and the results; the settings stay at their defaults. The cases are in Table 26.

Table 26: Cases for T-LRN-020 (FR-LRN-023)

Case	**Start**	**Sequence**	**Expected output**
a	T	85	Stage E; no adult prompt
b	E	85	Stage I
c	I	85	Lesson completed ("every stage passed"); furthest stage I; no other lesson starts
d	T	40	Stage T
e	T	100	Stage E, not I
f	The adult starts at E	85	Stage I
g	T	40, 40, 40, 85, then at E 4×40 [D], 3×40, then 40	Still running after seven failures at E; ends after the 8th at E
h	Default start	Watch the demonstration; the child continues	Stage T; no adult check shown

    Output: the expected output of each case is in its row of Table 26; the test passes when every case gives it.

    Test Case Derivation: invariants I6 to I8 and the rows "Passing a non-final drawing stage" and "Passing the final drawing stage" in SRS Section 9.2 give (a) to (c). (d) to (f) check that a non-passing attempt

stays and that no stage is skipped. In (g), the stage counts start again at a new stage, so the three T failures do not count at E. In (h), the "Begin practice after demonstration" transition needs no adult.

How test will be performed: Vitest runs every case. Playwright runs (a) to (c) as three `circle.json` from T.

21. T-LRN-021: The lesson rules match the formal specification

    Purpose: a second, independent check of the lesson rules over far more attempt sequences than the tables above can list.

    Type: Dynamic, automatic (Vitest with fast-check). Covers the rules and invariants of SRS Section 9.2, with FR-LRN-009, FR-LRN-010, FR-LRN-013, FR-LRN-015, FR-LRN-018 to FR-LRN-021 and FR-LRN-023.

    Initial State:

    - a reference model of SRS Section 9.2: a direct translation of the table of guided lesson state transitions and of the predicates Pass, Improving and SuggestBack, written by the Testing Lead, who does not write the lesson rules module. In the model, H holds the results of the evaluable attempts at the current stage, as in the conventions;
    - the lesson rules module, which offers a read-only view of its state for tests;
    - both start at the same chosen drawing stage with the same learner settings.

    Input: `MODEL_RUNS` random sequences of up to `SEQ_LEN_MAX` events, from the seed `MODEL_SEED`.

    - Events: a complete or incomplete attempt with result `STAGE_PASS` − 1, `STAGE_PASS`, `STAGE_PASS` + 1, 0, 100 or a random value from 0 to 100; invalid input; an interruption; an adult decline; an adult accept to an earlier stage; a child's attempt to accept, skip or change lesson; and an adult selection of a new lesson.
    - Settings: drawn from the SRS ranges, with the ends of each range drawn more often.

- Until the SRS settles where it disagrees with itself, the generator follows every suggestion with an adult accept or decline, places an accept only as the last event of a sequence, and draws only settings with `ATTEMPT_LIMIT` + `ATTEMPT_EXTENSION` $\geq$ `ATTEMPT_LIMIT_MAX`. These are three of the places where SRS Section 9.2 disagrees with SRS UC-005, BR-002 and SAF-RAY-005.

Output: after every event, the state of the module equals the state of the model, field by field. The fields are the stage, the session status, the evaluable and failed attempt counts, the streak, the pending suggestion, the extension granted, the extension attempts used and the furthest stage. Invariants I1 to I3 and I5 to I8 hold, and no run has a mismatch. I4 is about strokes and is covered by T-REC-002 and INS-EVAL.

Test Case Derivation: the model is the specification, so a mismatch is either a defect or an ambiguity in the SRS. Random orderings reach combinations the tables do not list, such as an extension granted at a stage entered after an adult selection. Random testing is used carefully:

- the seed is fixed, so every run can be repeated;
- the oracle is the model, not "it does not crash";
- fast-check shrinks a failing sequence to a shortest one;
- each shrunk failure is added as a case to the matching table test.

How test will be performed: `fc.assert` with the options `seed` = `MODEL_SEED` and `numRuns` = `MODEL_RUNS`, in CI on every PR that changes the lesson rules module. The seed changes once per milestone, and each seed and its result are recorded in the VnV Report.

4.1.2 Creative Drawing: Editing, Saving and Separation from Lessons

These tests cover the creative drawing requirements in SRS Section 9.3 that do not involve refinement: drawing and editing a creative drawing (FR-CRT-001 to FR-CRT-009), saving and reopening it under the right learner (FR-CRT-023 to FR-CRT-026), keeping it apart from guided lesson records (FR-CRT-028) and showing whether it is saved (FR-CRT-029). They follow use case UC-006 and the hazards H-MUH-001, H-MUH-003, H-MUH-004 and

H-MUH-005 in the Hazard Analysis. Refinement and restoring the original are tested in Section 4.1.3.

The tests use the words of SRS Section 9.3: a *part* is one or more strokes the child selects, and a *change* is one stroke drawn or one editing action. Unless a test says otherwise, every test starts from the same set-up. A test adult account holds two learner profiles, A and B. Learner A is active, the drawing area is `TEST_CANVAS`, and refinement is turned off for A, so finishing a drawing never changes its strokes. The prepared files the tests use are listed under *Prepared test data for the functional tests* in the Symbolic Parameters appendix. A test passes only if the drawing has exactly the expected strokes, in the expected order, with the expected points, colours and widths. Coordinates count as equal when they differ by at most `COORD_TOL`. To get the saved drawing, a script reads it through the same request Squigglr uses to reopen it, and compares it with the expected drawing, which the script computes from the fixture.

1. T-CRT-001: Every stroke drawn is added, in order

   Purpose: the drawing must hold exactly the strokes the child drew, so no stroke may be lost, merged with another or split in two.

   Type: Dynamic, automatic and manual. Covers FR-CRT-001.

   Initial State: the creative space is open for learner A with a new, empty drawing.

   Input: the five strokes of `crt-draw-5.json` are drawn: a straight line, a curve, a single tap, a closed loop, and a line that crosses the first one. The fourth and fifth strokes are drawn with the stylus lifted only briefly between them.

   Output: the drawing holds exactly five strokes, in the order drawn. Each one holds the points reported for it, in order. The tap is a stroke with one point, and the crossing line is a separate stroke from the line it crosses. Each of the five strokes can be selected on its own.

   Test Case Derivation: the fit criterion of FR-CRT-001, which asks for one new stroke per stroke drawn, in order, following the reported points. The tap, the crossing line and the short lift are the inputs most likely to make a drawing lose a stroke, merge two strokes, or split one.

How test will be performed: a Playwright script replays the pointer events of the fixture as pen input and compares the saved drawing with the fixture. A tester then draws the same five strokes by hand with the stylus on the test iPad and checks the stroke count and order.

2. T-CRT-002: Selecting a part

   Purpose: every editing action acts on the selected part, so selection must pick exactly the strokes the child chose.

   Type: Dynamic, automatic and manual. Covers FR-CRT-002.

   Initial State: `crt-three.json` is open for learner A. For case (c), `crt-cross.json` is open instead.

   Input: each case starts with nothing selected. After making the selection, the tester recolours it to a colour no stroke has yet.

   (a) Tap stroke 2.

   (b) Select strokes 1 and 3 together.

   (c) Tap exactly where the two strokes cross.

   (d) Tap an empty area of the canvas.

   Output:

   (a) Only stroke 2 is marked, and only stroke 2 changes colour.

   (b) Strokes 1 and 3 are marked, and only they change colour.

   (c) Exactly one stroke is marked, and only that stroke changes colour.

   (d) Nothing is marked, and recolouring changes no stroke.

   Test Case Derivation: the fit criterion of FR-CRT-002 checks one stroke and a group of strokes. Cases (c) and (d) try to break selection with an ambiguous tap and an empty one. The SRS does not say which stroke a tap on a crossing selects, so case (c) only checks that the selection and the change agree.

   How test will be performed: a Playwright script makes each selection and compares the drawing before and after the recolour. A tester repeats (a) and (c) with the stylus on the test iPad.

3. T-CRT-003: Moving a part

   Purpose: moving must shift the part as a whole, without changing its shape or touching any other stroke.

   Type: Dynamic, automatic. Covers FR-CRT-003.

   Initial State: `crt-three.json` is open for learner A.

   Input:

   (a) Select stroke 1 and drag it 100 CSS pixels right and 50 down.

   (b) Select strokes 2 and 3 and drag them 80 CSS pixels left and 40 up.

   (c) Select stroke 1 and drag it until half of it lies past the right edge of the canvas.

   Output: in every case, every point of the selected strokes has moved by the same offset, in the direction of the drag, and no other stroke has changed. In (a) and (b), the offset is the one dragged. In (c), the moved stroke is still in the drawing and can still be selected.

   Test Case Derivation: the fit criterion of FR-CRT-003. Case (b) checks that a group moves together. Case (c) tries to break the move at the canvas edge, where a stroke might be clipped or lost. The SRS does not say whether a part may leave the canvas, so (c) checks only that its shape is kept and the stroke is not lost.

   How test will be performed: a Playwright script drags the selection and computes each point's offset from the fixture.

4. T-CRT-004: Resizing a part

   Purpose: resizing must keep the part's proportions, so its shape stays as the child drew it.

   Type: Dynamic, automatic. Covers FR-CRT-004.

   Initial State: `crt-three.json` is open for learner A. For case (c), `crt-dot.json` is open instead.

   Input:

(a) Select stroke 1 and enlarge it to about twice its size.

(b) Select strokes 2 and 3 and shrink them to about half their size.

(c) Select the one-point stroke and try to enlarge it.

Output:

(a) There is one scale factor $s > 1$ for the whole part. For every point more than `COORD_TOL` from the centre of the part's bounding box, its new distance from that centre, divided by its old distance, differs from s by a relative error of at most `SCALE_TOL`. The centre does not move, and no other stroke changes.

(b) The same as (a), except that $s < 1$.

(c) The point stays where it was, Squigglr shows no error, and the other stroke does not change.

Test Case Derivation: the fit criterion of FR-CRT-004 multiplies every distance from the bounding-box centre by the same factor. The relative error $|d'/d - s|/s$, where d and d' are a point's distance from the centre before and after the resize, allows for floating-point rounding without hiding a distorted shape. Points very close to the centre are left out because the ratio is unstable there. Case (c) tries to break the resize with a bounding box of size zero, where a point's distance is zero and stays zero for any s.

How test will be performed: a Playwright script resizes the selection with the resize handle, works out s from the first point, and checks every other point against it.

5. T-CRT-005: Recolouring a part

Purpose: recolouring must change only the colour, and only of the selected strokes.

Type: Dynamic, automatic. Covers FR-CRT-005.

Initial State: `crt-three.json` is open for learner A. Its three strokes have three different colours.

Input:

(a) Select stroke 1 and recolour it, once to each colour Squigglr offers.

(b) Select all three strokes and recolour them to one colour.

Output: every selected stroke has the chosen colour. Its points and width are the same as before, and no stroke outside the selection has changed.

Test Case Derivation: the fit criterion of FR-CRT-005. Going through every colour in (a) catches a colour that is mapped wrongly. Case (b) checks a selection whose strokes start with different colours.

How test will be performed: a Playwright script recolours the selection and compares every stroke with the fixture.

6. T-CRT-006: Erasing a part

Purpose: erasing must remove exactly the selected strokes, and undoing the erase must bring them back exactly.

Type: Dynamic, automatic. Covers FR-CRT-006.

Initial State: `crt-three.json` is open for learner A. For case (c), `crt-cross.json` is open instead.

Input: in each case, erase the selection, check the drawing, then undo once.

(a) Select stroke 2 and erase it.

(b) Select all three strokes and erase them.

(c) Select one of the two crossing strokes and erase it.

Output: after the erase, the erased strokes are gone and every other stroke is unchanged. In (b), the canvas is empty. In (c), the other stroke is whole, including the part where the two strokes crossed. After the undo, the drawing equals the fixture, with the strokes in their original order.

Test Case Derivation: the fit criterion of FR-CRT-006. Case (b) empties the drawing. Case (c) checks that erasing one stroke does not also cut the stroke that crosses it.

How test will be performed: a Playwright script erases the selection, then undoes it, comparing the drawing with the fixture after each step.

7. T-CRT-007: Undoing and redoing changes

Purpose: a child who cannot read needs a way back from any mistake, and a way to recover from an accidental undo.

Type: Dynamic, automatic. Covers FR-CRT-007 and FR-CRT-008.

Initial State: `crt-three.json` is saved for learner A and has just been opened. Call this drawing S_0.

Input: five changes are made in order, giving drawings S_1 to S_5: C_1 draws a new stroke, C_2 moves stroke 1, C_3 resizes stroke 2, C_4 recolours stroke 3, and C_5 erases the new stroke. Then:

(a) Undo five times, then try a sixth undo.

(b) Redo five times, then try a sixth redo.

(c) Undo twice, make a new change C_6 (draw a stroke), then try to redo.

(d) Leave the creative space, reopen the drawing, then try to undo.

Output:

(a) After the k-th undo the drawing equals S_{5-k}, so the fifth undo gives back S_0. The sixth undo is not offered, or it changes nothing.

(b) After the k-th redo the drawing equals S_k. The sixth redo is not offered, or it changes nothing.

(c) The drawing is S_3 plus the new stroke, and no redo is offered.

(d) The drawing reopens exactly as it was left, and no undo is offered.

Test Case Derivation: FR-CRT-007 undoes one change at a time, most recent first, back to the drawing as it was opened. FR-CRT-008 redoes until the child makes a new change. The sequence uses one change of each kind, so each kind is undone and redone once. Case (d) checks FR-CRT-007's rule that changes made before the drawing was last opened cannot be undone. The refinement and restore steps in FR-CRT-007's fit criterion are tested in T-CRT-016 (f) and T-CRT-020 (b, e).

How test will be performed: a Playwright script saves each S_k after its change, then compares the drawing with the expected S_k after every undo and redo.

8. T-CRT-008: Starting a new drawing

   Purpose: starting a new drawing must give an empty canvas and keep the earlier drawing as it was.

   Type: Dynamic, automatic. Covers FR-CRT-009, and FR-CRT-023 and FR-CRT-024 as a second check.

   Initial State: `crt-three.json` is saved for learner A and open.

   Input:

   (a) Start a new drawing, then reopen the earlier drawing.

   (b) Move stroke 1 and, straight away, start a new drawing, without waiting for the save. Then reopen the earlier drawing.

   Output: the new drawing has no strokes in both cases. In (a), the earlier drawing reopens equal to the fixture. In (b), it reopens with stroke 1 moved. In both cases it is listed once in learner A's creative space.

   Test Case Derivation: the fit criterion of FR-CRT-009. Case (b) checks that the last change is saved with no action from the child (FR-CRT-023), even when the child moves on at once, which is one of the causes of H-MUH-005.

   How test will be performed: a Playwright script starts the new drawing, checks the canvas, then reopens the earlier drawing and compares it.

9. T-CRT-009: Saving to the active learner and reopening

   Purpose: each drawing must be saved, without any action from the child, to the learner who drew it, and reopen exactly as it was left.

   Type: Dynamic, automatic and manual. Covers FR-CRT-023 and FR-CRT-024.

   Initial State: learners A and B have no creative drawings.

   Input:

   (a) As learner A, draw `crt-draw-5.json`, move one stroke and recolour another, then leave the creative space with its own control.

The adult passes the adult check and switches to learner B, who draws `crt-three.json` and leaves. Both drawings are then reopened, each under its own learner.

(b) As learner A, open the saved drawing and erase one stroke. The test holds the erase's save request so that it is still pending. While the drawing is still open, the adult passes the adult check and switches to learner B, and only then is the held save released.

(c) Reload the page, and also sign in to the same account in a second browser, then reopen A's drawing in each.

Output:

(a) A's drawing reopens with the five strokes, the move and the recolour, and B's reopens equal to `crt-three.json`. Every stroke of both can be selected and edited.

(b) Once the released save completes, the erase is in learner A's drawing. Learner B's drawings do not change.

(c) A's drawing is identical in both browsers and equals the drawing last saved.

Test Case Derivation: the fit criteria of FR-CRT-023 (two learners, each drawing reopened as left) and FR-CRT-024 (reopened exactly as last saved, every stroke editable). Case (b) checks the cause in H-MUH-004 where a drawing is saved to the learner active when the save completes, not when it was drawn. Holding the save until after the switch makes sure that a build with this defect fails, since an unheld save would normally finish before the adult check. The refined states in FR-CRT-024's fit criterion are tested in T-CRT-027.

How test will be performed: a Playwright script runs each case and reads the saved drawings of both learners through the test database. In (b) it holds the save request with request routing (`page.route`) and releases it after the switch. A tester repeats (a) on the test iPad with the stylus.

10. T-CRT-010: Each learner sees only their own drawings

Purpose: on a shared account or device, one child must never see, or change, another child's drawing.

Type: Dynamic, automatic. Covers FR-CRT-026, and FR-CRT-023 as a second check.

Initial State: learner A has three saved drawings and learner B has two. Learner A is active.

Input:

(a) Open A's creative space and list its drawings. Then switch to B (through the adult check) and list again.

(b) While A is active, open the web address of one of B's drawings directly.

Output:

(a) A's creative space lists exactly A's three drawings, and B's lists exactly B's two.

(b) B's drawing is not shown, and B's drawing does not change.

Test Case Derivation: the fit criterion of FR-CRT-026. Case (b) tries to get round the list by going straight to another learner's drawing, which a child may do with the browser's history (H-MUH-004).

How test will be performed: a Playwright script lists the drawings for each learner and opens B's drawing address while A is active.

11. T-CRT-011: Keeping changes through a lost connection, and showing unsaved changes

Purpose: a dropped connection must not lose the child's changes, and the adult must be able to see when changes are not yet saved.

Type: Dynamic, automatic and manual. Covers FR-CRT-025 and FR-CRT-029.

Initial State: `crt-three.json` is saved for learner A and open, and the network is online.

Input:

(a) Set the network offline. Draw two strokes and move stroke 1. Set the network online, wait for the save to complete, then leave the creative space and reopen the drawing.

(b) Repeat (a), but turn the network off and on three times during the edits, making one change each time it is off.

(c) With the network online, make one change and watch the save indicator.

(d) Set the network offline and draw two strokes, then close the page. Reopen Squigglr online and open the drawing. Repeat with the page closed just after the connection returns, before the indicator shows that all changes are saved.

Output:

(a) While offline, Squigglr shows that there are unsaved changes. After the connection returns and the save completes, it shows that all changes are saved. The reopened drawing has the two new strokes and the move.

(b) The reopened drawing has every change exactly once, with no stroke saved twice, and the indicator matches the save state after each switch.

(c) The indicator shows unsaved changes until the server confirms the save, then shows that all changes are saved.

(d) The reopened drawing has every change that the indicator showed as saved before the page was closed. The changes it keeps from the two strokes are recorded for each run, and none is kept twice.

Test Case Derivation: the fit criteria of FR-CRT-025 (changes made while offline are saved once the connection returns) and FR-CRT-029 (unsaved, then saved). Case (b) tries to make Squigglr save the same change twice or skip one, as in the retried saves of H-MUH-004 and H-MUH-005. Case (d) is the Document/Test control of H-MUH-005 for cause (c), closing the page during and after a lost connection. Offline use is out of scope (SRS ASM-005), so FR-CRT-025 cannot require changes to survive a closed page. The case therefore only requires that the indicator was truthful (FR-CRT-029) and records which changes are kept, as the Hazard Analysis asks.

How test will be performed: a Playwright script switches the network with `context.setOffline()`, records the indicator after each step, and reads the saved drawing through the test database. In (d) it closes the

page and opens a new one in the same browser context. A tester repeats (a) on the test iPad by turning off Wi-Fi.

12. T-CRT-012: Creative drawings do not change practice records

    Purpose: practice records must show only guided practice, so nothing done in the creative space may add to or change them.

    Type: Dynamic, automatic. Covers FR-CRT-028.

    Initial State: learner A has the prepared history `history-basic.json` (two lesson records and eight attempt records). Its attempt records, lesson records and progress summary are exported and kept as the reference.

    Input: in the creative space, draw `crt-draw-5.json`, edit it, start a second drawing, and finish both. Then turn refinement on for A, finish a prepared drawing of the supported set so that it is refined, and restore its original.

    Output: learner A's attempt records, lesson records and progress summary are identical to the reference. No attempt is added, and no count or date changes.

    Test Case Derivation: the fit criterion of FR-CRT-028 and the cause in H-MUH-003 where creative drawings and attempts are counted together. A refined drawing counted as an attempt would make assisted work look like the child's own.

    How test will be performed: a Playwright script exports the records and the summary through the test database before and after, and compares the two.

4.1.3 Creative Drawing: Refinement and Restoring the Original

These tests cover the creative drawing requirements in SRS Section 9.3 about refinement: refining a finished drawing (FR-CRT-010 to FR-CRT-016), restoring the original (FR-CRT-017 to FR-CRT-019), failure and interruption (FR-CRT-020 to FR-CRT-022), no refinement in guided lessons (FR-CRT-027) and turning refinement off (FR-CRT-030). They also test the refinement and restore steps of FR-CRT-007 and the refined states of FR-CRT-024, which T-CRT-007 and T-CRT-009 leave to this section. They

follow UC-007 and BR-005, and try to cause the failures in Hazard Analysis rows H-MUH-001, H-MUH-002, H-MUH-003 and H-MUH-006: a restore that loses work, a refinement that changes more than it may or reaches the wrong drawing, a refined drawing that passes as the child's own, and a child who cannot get rid of a refinement they do not want. Refinement is goal G.4, the riskiest part of the product, so FR-CRT-012, FR-CRT-017 and FR-CRT-020 each have a second, independent test. As Section 2.2 says, the tests check what Squigglr does with a refinement result, not how good the result looks.

The set-up is the one in Section 4.1.2, except that refinement is on for learner A. *Finishing* a drawing means using the control the child uses to say the drawing is done. The tests also use these terms:

- *Supported set.* The SRS leaves the supported set open (OI-03). Until it is agreed, these tests use five items: circle, square, triangle, house and sun, in `ref-circle.json`, `ref-square.json`, `ref-triangle.json`, `house.json` and `sun.json`. This is the set the tests use, not a scope decision. When OI-03 is closed, each agreed item gets a prepared drawing and this list follows it; the tests themselves do not change. The drawings of items outside the set are `ref-out-scribble.json` and `ref-out-fish.json`. If the fish joins the supported set, another item outside it is recorded instead.

- *Within the tolerance.* For a point p and a stroke s, $d(p, s)$ is the shortest distance from p to the line joining the points of s in order (for a stroke of one point, the distance to that point). A refined stroke r is within `REFINE_TOLERANCE` of its original o when $d(p, o) \leq$ `REFINE_TOLERANCE` for every point p of r, and $d(q, r) \leq$ `REFINE_TOLERANCE` for every point q of o. "Every point" includes points taken every 1 CSS pixel along each segment, so a segment that bulges between two good end points also fails. Checking both ways is what FR-CRT-012 asks for: it catches a stroke that was shortened or partly dropped.

- *The refinement stub.* Except in T-CRT-014 and T-CRT-018, which use the real refinement service of the test environment, the test set-up intercepts Squigglr's calls to the refinement service and answers with a stub, so the expected drawing is known before the test runs. The *shift stub* returns each stroke it is sent with every point moved `REFINE_SHIFT` to the right. This is a valid refinement of known shape. The stub counts the calls it receives, and can hold its answer for `REFINE_DELAY` or until

the test releases it. For a drawing D, $R(D)$ is the drawing the shift stub returns. Other answers are given in the cases that use them.

- *Refined state.* What Squigglr shows about refinement (FR-CRT-016) is read from the indicator's accessible name, so the script reads what a screen reader would. How a child who cannot read sees it is a usability matter, as the SRS says.

As in Section 4.1.2, two drawings are *equal* when they have the same strokes, in the same order, with the same colours and widths, and every coordinate within `COORD_TOL`.

1. T-CRT-013: When refinement starts

   Purpose: refinement must happen when the child finishes a drawing and at no other time, so strokes never move while the child is still drawing.

   Type: Dynamic, automatic. Covers FR-CRT-010.

   Initial State: the common set-up, with the shift stub and its call count at zero.

   Input:

   (a) Draw each drawing of the supported set in turn and finish it.

   (b) Draw `house.json` without finishing it. Then select, move, resize, recolour and erase a part, undo twice and redo once.

   (c) Finish the drawing of (b), restore its original, then undo the restore.

   (d) Draw `sun.json`, leave the creative space without finishing, and reopen the drawing.

   Output:

   (a) Exactly one call per finish, five in all, and each drawing D becomes $R(D)$.

   (b) No call, and the drawing is exactly as edited.

   (c) Exactly one call, for the finish; the restore and the undo make none.

(d) No call, and the drawing reopens exactly as left.

Test Case Derivation: the fit criterion of FR-CRT-010 names finishing (one refinement) and drawing, editing, undoing, restoring and leaving (none). Each case tries an action that a mistaken implementation could tie to refinement, such as the end of a stroke, a save or a restore. This is cause (c) of H-MUH-002.

How test will be performed: a Playwright script runs each case and reads the stub's call count after each action.

2. T-CRT-014: Refinement of the supported set by the real service

Purpose: the refinement Squigglr really uses must keep the child's strokes, colours and widths, and move no point further than `REFINE_TOLERANCE`.

Type: Dynamic, automatic. Covers FR-CRT-011 and FR-CRT-012.

Initial State: the common set-up, with the real refinement service of the test environment.

Input: each drawing of the supported set is drawn and finished `REFINE_RUNS` times. Its strokes are given colours and widths in turn from those offered, so that a changed colour or width shows.

Output: for every run, the drawing after refinement has the same number of strokes as the drawing as finished, in the same order, each with the same colour and width; and each stroke is within `REFINE_TOLERANCE` of its original, both ways. The script also records the runs in which the drawing was left unchanged. A supported item left unchanged in all its runs is reported as a defect against G.4.

Test Case Derivation: the fit criteria of FR-CRT-011 and FR-CRT-012, on prepared drawings of each supported item. Only the real service shows what the deployed refinement does, and the team does not train the model, which may return excessive changes (H-MUH-002, cause (a)). The model may answer differently each time, so each drawing is run more than once. A drawing left unchanged meets both requirements, which is why those runs are counted separately.

How test will be performed: a Playwright script finishes each drawing, reads the saved drawing as in Section 4.1.2, and computes the distances

as defined above. It runs before each milestone and on every PR that changes the refinement module or the model, not on every PR, because it calls the outside service.

3. T-CRT-015: Checking refinement results before they are applied

   Purpose: a second, independent check of FR-CRT-011, FR-CRT-012 and FR-CRT-020, over far more results than the real service or the stubs return.

   Type: Dynamic, automatic (Vitest with fast-check). Covers FR-CRT-011, FR-CRT-012 and FR-CRT-020.

   Initial State: the module that applies refinement results, called directly, with no canvas or network; a reference check of the three rules above (same strokes, colours and widths, and within `REFINE_TOLERANCE` both ways), written from this section by a team member who does not write the module.

   Input: `CHECK_RUNS` random pairs of a drawing and a result, from the seed `MODEL_SEED`.

   - Drawings: 1 to 20 strokes of 1 to 200 points, in random colours and widths, including strokes of one point.
   - Results, each made from its drawing: every point moved a random distance up to `REFINE_TOLERANCE` (valid); every point moved exactly `REFINE_TOLERANCE` (valid, at the limit); one point moved `REFINE_TOLERANCE` + 0.5 CSS pixels; a stroke added, removed or two strokes swapped; one colour or width changed; the last third of a stroke removed; and malformed results (a missing field, a coordinate that is not a number or is infinite, a wrong type, an empty answer).

   Output: the module applies a result if and only if the reference check accepts it. When it applies a result, the drawing equals the result; when it does not, the drawing equals the input drawing exactly. No run ends in an uncaught error.

   Test Case Derivation: T-CRT-014 can only see what the real model happens to return. This test gives the module the results a model could return on a bad day (H-MUH-002, cause (a)), including results

just past the limit and at it. 0.5 CSS pixels is 50 times `COORD_TOL`, so a rejected result cannot be blamed on rounding. Removing the last third of a stroke breaks only the check from the original to the refined stroke. Random testing is used as in T-LRN-021: the seed is fixed, the oracle is the reference check, and each failure that fast-check shrinks is added as a case to T-CRT-023.

How test will be performed: `fc.assert` with the options `seed` = `MODEL_SEED` and `numRuns` = `CHECK_RUNS`, in CI on every PR that changes the refinement module.

4. T-CRT-016: Editing a refined drawing

   Purpose: a refined drawing must stay the child's to edit, with every stroke as editable as one the child drew.

   Type: Dynamic, automatic. Covers FR-CRT-013, and the refinement step of FR-CRT-007.

   Initial State: `house.json` is drawn and finished with the shift stub, so the open drawing is R(`house.json`).

   Input: on refined strokes,

   (a) the selections of T-CRT-002 (a) and (b);

   (b) the move of T-CRT-003 (a);

   (c) the resizes of T-CRT-004 (a) and (b);

   (d) the recolour of T-CRT-005;

   (e) the erase and undo of T-CRT-006;

   (f) after (a) to (e), undo every change, one at a time.

   Output: in (a) to (e), each action gives the output of the matching case of T-CRT-002 to T-CRT-006, applied to the refined strokes.

   (f) Undoing the edits gives R(`house.json`); one more undo gives the drawing as finished, unrefined; the undos after that remove the strokes one by one, back to the empty canvas the drawing was opened with.

Test Case Derivation: the fit criterion of FR-CRT-013 asks that FR-CRT-002 to FR-CRT-007 meet their own fit criteria on refined strokes (G.4 measurement (b)). A refined drawing returned as one picture, or with strokes merged, would fail (a). Case (f) is the refinement step of FR-CRT-007: an applied refinement is one change, so one undo reverses it completely.

How test will be performed: the Playwright scripts of T-CRT-002 to T-CRT-007, started from R(`house.json`) instead of their own drawings.

5. T-CRT-017: No second refinement of the same strokes

   Purpose: finishing the same drawing again must not refine it again, or the drawing would drift further from what the child drew each time.

   Type: Dynamic, automatic. Covers FR-CRT-014.

   Initial State: `house.json` is drawn and finished with the shift stub, so the open drawing is $R_1 = R$(`house.json`).

   Input:

   (a) Finish again without any change.

   (b) Finish five more times without any change.

   (c) Move refined stroke 2, draw one new stroke, and finish.

   Output: in (a) and (b), the drawing equals R_1 exactly after every finish.

   (c) Only stroke 2 and the new stroke differ from the drawing just before the finish, each moved `REFINE_SHIFT` to the right. Every other stroke equals its stroke in R_1.

   Test Case Derivation: the fit criterion of FR-CRT-014, and cause (c) of H-MUH-002. The shift stub moves every stroke it receives, so any second refinement would show after one finish, and (b) shows the moves do not add up. The test passes whether Squigglr sends only the strokes it may refine or sends them all and applies the result only to those strokes.

   How test will be performed: a Playwright script compares the drawing after each finish with the expected one.

6. T-CRT-018: Drawings outside the supported set

   Purpose: Squigglr must leave a drawing alone when it has nothing to refine, so it never turns a drawing into something the child did not draw.

   Type: Dynamic, automatic. Covers FR-CRT-015.

   Initial State: the common set-up, with the real refinement service of the test environment.

   Input:

   (a) Finish an empty canvas.

   (b) Draw `ref-out-scribble.json` and finish it.

   (c) Draw `ref-out-fish.json` and finish it.

   Output: for each case, the drawing equals the drawing as finished exactly; one of its strokes can be selected and moved as in T-CRT-003 (a); and, from PH-2, the refined state shows not refined. In (a) the canvas stays empty and Squigglr shows no error.

   Test Case Derivation: the fit criterion of FR-CRT-015, and cause (d) of H-MUH-002. The real service is used because whether a drawing is recognized may be decided there or in Squigglr; the test passes whichever decides, as long as the drawing is unchanged. The scribble is the drawing least like any item; the fish is a plausible picture outside the set, which is more likely to be refined by mistake.

   How test will be performed: a Playwright script, run with T-CRT-014.

7. T-CRT-019: Showing whether a drawing is refined

   Purpose: the child and the adult must be able to tell a refined drawing from the child's original.

   Type: Dynamic, automatic. Covers FR-CRT-016.

   Initial State: `house.json` is drawn; the shift stub is used, except in (g).

   Input: the drawing is brought into each state:

(a) never refined (refinement turned off, then finished);

(b) refined;

(c) refined, then a refined stroke moved;

(d) refined, then restored;

(e) refined, then the refinement undone;

(f) refined, new strokes drawn, refined again, and restored once;

(g) finished while the stub answers with an error, so refinement fails.

Output: the refined state shows refined in (b), (c) and (f), and not refined in (a), (d), (e) and (g).

Test Case Derivation: the fit criterion of FR-CRT-016 lists states (a) to (e) and (g). FR-CRT-016 asks whether the drawing "holds a refinement that has not been undone or restored": editing does not remove the refinement (c), and in (f) one refinement is still left after one restore. A drawing shown as not refined while it is refined would let an adult take assisted work as the child's own (H-MUH-003).

How test will be performed: a Playwright script reads the indicator's accessible name in each state.

8. T-CRT-020: Restoring the original

Purpose: a restore must give back exactly what the child drew, so no version of the child's work is lost.

Type: Dynamic, automatic. Covers FR-CRT-017, FR-CRT-018, and the restore step of FR-CRT-007 and FR-CRT-008.

Initial State: O_1 is `house.json`, drawn and finished with the shift stub, giving $R_1 = R(O_1)$.

Input:

(a) From the child's view, restore the original.

(b) From R_1, move stroke 1 (giving E), restore, then undo.

(c) From R_1, draw two new strokes (giving O_2), finish (giving R_2), then restore twice.

(d) Look for the restore control after (c), on a drawing never refined, and after a refinement and an undo.

(e) After (a), undo, then redo.

Output:

(a) The restore takes one action, and the drawing equals O_1.

(b) After the restore the drawing equals O_1; after the undo it equals E.

(c) After the first restore the drawing equals O_2, which still holds the refined strokes of R_1; after the second it equals O_1.

(d) No restore control is offered in any of the three.

(e) The undo gives R_1; the redo gives O_1.

Test Case Derivation: cases (a) to (d) are parts (a) to (d) of the fit criterion of FR-CRT-017 (G.4 measurement (c)), and (a) is also the one-action restore of FR-CRT-018. Case (c) checks that each restore steps back only one refinement, the case most likely to lose work (H-MUH-001). In (b) and (e), a restore is one change that can be undone and redone (FR-CRT-007, FR-CRT-008).

How test will be performed: a Playwright script finds the restore control in the child's view by its accessible name, counts the actions, and compares the drawing with the expected one after each step.

9. T-CRT-021: The adult restores a saved drawing

Purpose: an adult must be able to restore the original for a child who cannot, and only for the learners in their own account.

Type: Dynamic, automatic. Covers FR-CRT-019.

Initial State: learner A has a saved refined drawing, R(`house.json`); learner B has a saved refined drawing, R(`sun.json`); learner B is active. A second test adult account exists.

Input:

(a) The adult passes the adult check of SAF-RAY-001, opens A's drawings, and restores the original of A's drawing. Then A is made active and the drawing is reopened.

(b) The adult starts to restore A's drawing but fails the adult check, then tries again and cancels it.

(c) Signed in to the second account, the tester sends the restore request for A's drawing directly.

Output:

(a) A's reopened drawing equals `house.json`, and B's drawing still equals R(`sun.json`).

(b) No restore happens, and A's drawing still equals R(`house.json`).

(c) The request is refused, and A's drawing is unchanged.

Test Case Derivation: the fit criterion of FR-CRT-019 for (a). Cases (b) and (c) try to break the two limits in its statement: the adult check of SAF-RAY-001, which keeps a child from reaching this action, and the account limit of BR-001. A restore applied to the active learner instead of the chosen one would show in B's drawing in (a).

How test will be performed: a Playwright script for (a) and (b); for (c), the script sends the request with the second account's session.

10. T-CRT-022: Random edit, refine and restore sequences

Purpose: a second, independent check of restoring, undoing and not refining twice, over far more orders of actions than the cases above.

Type: Dynamic, automatic (Vitest with fast-check). Covers FR-CRT-017, with FR-CRT-007, FR-CRT-008 and FR-CRT-014.

Initial State: the creative drawing module's state functions, called directly, with the shift stub as a plain function; a reference model of the drawing history, written from SRS Section 9.3 by a team member who does not write the module. In the model, a change is a list entry, and a refinement entry keeps its original.

Input: `CRT_MODEL_RUNS` random sequences of up to `CRT_SEQ_LEN_MAX` events, from the seed `MODEL_SEED`. The events are: draw a stroke, move, resize, recolour, erase, finish, undo, redo, restore, and save and reopen.

Output: after every event, the module and the model agree on the drawing, on whether undo, redo and restore are offered, and on the refined state. No run has a mismatch.

Test Case Derivation: the four cases in the fit criterion of FR-CRT-017 cover a few orders; random sequences reach others, such as a restore after a reopen that follows the undo of a second refinement. Saving and reopening clears undo but keeps the original restorable (FR-CRT-007, FR-CRT-024). Random testing is used as in T-LRN-021, and each shrunk failure is added as a case to T-CRT-020.

How test will be performed: `fc.assert` with the options `seed` = `MODEL_SEED` and `numRuns` = `CRT_MODEL_RUNS`, in CI on every PR that changes the creative drawing module.

11. T-CRT-023: Failed and invalid refinement results

    Purpose: a refinement that fails, or returns a result that breaks the rules, must leave the drawing exactly as the child finished it.

    Type: Dynamic, automatic. Covers FR-CRT-020.

    Initial State: each drawing D of the supported set in turn, drawn and about to be finished.

    Input: D is finished while the stub

    (a) cannot be reached, because the connection is cut;

    (b) answers with a server error;

    (c) gives no answer within `REFINE_LATENCY_MAX`;

    (d) answers $R(D)$ with one stroke added;

    (e) answers $R(D)$ with one point moved `REFINE_TOLERANCE` + 0.5 CSS pixels from its original stroke;

    (f) answers with text that is not valid JSON;

    (g) answers $R(D)$ with one stroke removed;

    (h) sends $R(D)$ but cuts the answer off halfway.

    Then the stub is set back to the shift stub and D is finished again.

    Output: for every drawing and case, the drawing after the failure equals D exactly; one stroke can be selected and moved as in T-CRT-003 (a), and the move undone; from PH-2 the refined state shows not refined. Finishing again makes exactly one call and gives $R(D)$.

Test Case Derivation: cases (a) to (f) are the six simulated cases in the fit criterion of FR-CRT-020 (G.4 measurement (d)). Cases (g) and (h) are the "only part of a result is applied" failure of H-MUH-002. Finishing again checks the rationale of FR-CRT-020: a failure must not stop later refinement. T-CRT-015 tests the same rules on random results.

How test will be performed: a Playwright script sets the stub's answer for each case; (a) uses `context.setOffline()` and (c) a controlled clock.

12. T-CRT-024: Late results, and drawing while refinement runs

    Purpose: the child must be able to keep drawing while refinement runs, and a result that arrives after the child has moved on must not overwrite the newer drawing.

    Type: Dynamic, automatic. Covers FR-CRT-021 and FR-CRT-022.

    Initial State: the shift stub holds its answer until the test releases it. D is `house.json`, drawn and finished.

    Input: before the answer is released,

    (a) draw one new stroke n;

    (b) draw one new stroke n and move stroke 1 by a known distance;

    (c) leave the creative space; the answer is released, and the drawing is reopened;

    (d) undo the last stroke of D;

    (e) start a new drawing and draw one stroke on it;

    (f) make no change.

    Then the answer is released.

    Output:

    (a) The drawing equals D plus n, with nothing moved by `REFINE_SHIFT`.

    (b) Stroke n is kept as drawn and stroke 1 is moved by exactly the known distance, as T-CRT-001 and T-CRT-003 (a) require; nothing else changes.

(c) The drawing reopens equal to D.

(d) The drawing equals D without its last stroke.

(e) The new drawing holds only its one stroke, and D reopens equal to D.

(f) The drawing equals $R(D)$.

Test Case Derivation: the fit criteria of FR-CRT-021 ((a) and (c)) and FR-CRT-022 ((b)), and cause (b) of H-MUH-002. Undoing (d) is also a change, and (e) checks that a result never reaches another drawing. Case (f) shows that the stub's held answer is applied when nothing has changed, so in the other cases a result that was discarded can be told apart from one that never came.

How test will be performed: a Playwright script makes the changes, releases the stub, and compares the drawing with the expected one.

13. T-CRT-025: No refinement in guided lessons

Purpose: strokes drawn in a guided lesson must never be refined, so a lesson record shows only what the child drew.

Type: Dynamic, automatic. Covers FR-CRT-027.

Initial State: learner A, with refinement on; the shift stub, with its call count at zero.

Input:

(a) For each lesson in the catalogue, at stages T, E and I, the lesson's passing file is replayed and finished; at the circle lesson, `circle-fail.json` is also replayed.

(b) A creative drawing is finished while the stub holds its answer; the child then goes to the circle lesson and replays `circle.json` at T; the answer is released during the attempt.

Output: in (a), the stub receives no call. In both cases, the strokes shown during and after each attempt equal the replayed file, and the attempt record holds the same strokes. In (b), the creative drawing is left unrefined (FR-CRT-021).

Test Case Derivation: the fit criterion of FR-CRT-027, and cause (e) of H-MUH-002. Case (b) tries to make a late creative result land on lesson strokes. T-REC-002 checks the same strokes in the record in a second way.

How test will be performed: a Playwright script reads the stroke list the canvas draws during and after each attempt, and the record from the test database.

14. T-CRT-026: Turning refinement off and on

    Purpose: a child who does not want refined drawings, or the adult for them, must be able to keep drawings exactly as drawn.

    Type: Dynamic, automatic. Covers FR-CRT-030.

    Initial State: refinement is on for learners A and B, with the shift stub.

    Input:

    (a) As the child in A's creative space, turn refinement off, then draw and finish `ref-circle.json`.

    (b) Turn it on again in the creative space, then draw and finish `ref-square.json`.

    (c) As the adult, pass the adult check and turn refinement off in A's settings; back in A's creative space, draw and finish `ref-triangle.json`.

    (d) As the adult, turn it on again, then draw and finish `sun.json` as A.

    (e) With refinement off for A, make B active and finish `house.json`.

    (f) Turn refinement off for A, reload the page, and finish `ref-circle.json`.

    Output: in (a), (c) and (f), the drawing stays exactly as drawn, and the stub receives no call. In (b), (d) and (e), the drawing D becomes $R(D)$.

    Test Case Derivation: the fit criterion of FR-CRT-030: off and on again, once as the child and once as the adult. The setting is per learner profile, so (e) checks that it does not reach B. Case (f) checks

that the setting is kept, so a child who turned it off does not find the next drawing refined (H-MUH-006).

How test will be performed: a Playwright script runs the cases in order and reads the stub's call count.

15. T-CRT-027: Reopening a drawing in each refined state

    Purpose: leaving the creative space must not lose a refinement's original or the drawing's refined state.

    Type: Dynamic, automatic. Covers FR-CRT-024 (refined states), with FR-CRT-016, FR-CRT-017 and FR-CRT-023.

    Initial State: learner A has `house.json` saved in each state of T-CRT-019 (a) to (g), one drawing per state, and an eighth drawing refined twice.

    Input: the page is reloaded, and each drawing is reopened, once in the same browser and once in a second browser signed in to the same account. In each drawing, one stroke is selected and moved, then the move is undone; then the original is restored as many times as the drawing has refinements left.

    Output: each drawing reopens equal to the drawing as saved, with the same refined state; the move and its undo work as in T-CRT-003 (a); no undo of a change made before the reload is offered; each restore gives the result T-CRT-020 expects for that state; and no restore is offered in states (a), (d), (e) and (g).

    Test Case Derivation: the fit criterion of FR-CRT-024 for every state of FR-CRT-016. FR-CRT-023 saves the originals of refinements with the drawing, so a restore after reopening must still work (H-MUH-001, H-MUH-005). The second browser shows that the originals were saved, not only kept in the page.

    How test will be performed: a Playwright script prepares the drawings with the shift stub, then reopens each in a new browser context.

4.1.4 Attempt Records

These tests cover the ten attempt record requirements in SRS Section 9.4 (FR-REC-001 to FR-REC-010). FR-REC-001 and FR-REC-003 are due in

PH-1, and the rest in PH-2. Each test runs end to end in Playwright. The attempt and lesson records are read from the test database after each run, as in T-PERF-005. The tests try to cause the failures in Hazard Analysis rows H-MUH-003, H-MUH-004 and H-MUH-007: a record that hides guidance or corrections, an attempt saved twice or under the wrong lesson, and a record that changes later. Three safety requirements on records are tested with the safety-critical tests and not repeated here: SAF-RAY-003 (T-SAF-003), SAF-RAY-006 (T-SAF-006) and SAF-RAY-007 (T-SAF-007). Unless a case says otherwise, learner A uses the circle lesson with the default values. F, N and P stand for `circle-fail.json`, `circle-near-miss.json` and `circle.json`, and the notation for adult responses is that of the guided lesson tests. An unanswered suggestion is recorded as pending while the lesson runs, and as ignored once the lesson ends, or the stage changes, without an answer.

1. T-REC-001: Attempt record contents

   Purpose: each record must show which lesson and stage the attempt was at, its result, whether it passed and when it ended. The adult reads progress from these records.

   Type: Dynamic, automatic. Covers FR-REC-001.

   Initial State: learner A with no records; a controlled clock at `CLOCK_START`.

   Input: ten attempts, each finished `ATTEMPT_GAP` after the one before, except attempt 10, which is finished 0.5 seconds after attempt 9. The lesson, stage and file vary; the learner and settings stay fixed. $r(x)$ is the result the evaluation function returns for file x at that stage; the test computes it by calling the function directly. M and S stand for `square-missing-side.json` and `square-4-strokes.json`. The attempts are in Table 27.

   Output: the expected output of each case is in its row of Table 27; the test passes when every case gives it. The test database holds exactly ten attempt records, all under learner A, in the order of Table 27, each holding exactly the fields in its row.

   Test Case Derivation: the fit criterion asks for known lessons, stages, results and outcomes, with one pass and one fail at each drawing stage,

Table 27: Cases for T-REC-001 (FR-REC-001)

#	**Lesson, stage**	**File**	**Expected record**
1	Circle, T	F	Circle, T, r(F), not passed, `CLOCK_START` + 1 min
2	Circle, T	P	Circle, T, r(P), passed, + 2 min
3	Circle, E	F	Circle, E, r(F), not passed, + 3 min
4	Circle, E	P	Circle, E, r(P), passed, + 4 min
5	Circle, I	F	Circle, I, r(F), not passed, + 5 min
6	Circle, I	P	Circle, I, r(P), passed, + 6 min
7	Square, T	`square-missing-side.json`	Square, T, r(M), not passed, + 7 min
8	Square, T	`square-4-strokes.json`	Square, T, r(S), passed, + 8 min
9	Circle, T (new lesson)	F	Circle, T, r(F), not passed, + 9 min
10	Circle, T	F	Circle, T, r(F), not passed, + 9 min 0.5 s

in the order the attempts ended. The outcomes follow from calibration ($r(\mathrm{F}) \leq 60$ and $r(\mathrm{P}) \geq 90$, against `STAGE_PASS` 80). Attempt 7 is not passed because it is incomplete (SRS FR-LRN-011), although $r(\mathrm{M})$ is at least `CAL_INCOMPLETE_MIN`; attempt 8 passes because $r(\mathrm{S}) \geq 90$. Attempts 2, 4 and 8 change the stage when they pass, so they try to cause H-MUH-003 (a), where the stage is "taken from the lesson's stage when the save completes rather than when the child drew". The controlled clock makes the times exact. Attempts 9 and 10 check the order when times are close.

How test will be performed: Playwright with a controlled clock, then a database read.

2. T-REC-002: Strokes kept as drawn

   Purpose: the record must hold the child's own strokes, not a corrected or ideal shape, so the adult sees what the child really drew.

   Type: Dynamic, automatic. Covers FR-REC-002.

   Initial State: learner A; the drawing area is `TEST_CANVAS`.

   Input: the cases vary the lesson, the stage and the attempt; the learner, the drawing area and the defaults stay fixed. "Exactly" means the same number of strokes, the same order and boundaries, and each point's x and y within 0.01 CSS pixels, with the same pressure and time offset. The cases are in Table 28.

   Output: the expected output of each case is in its row of Table 28; the test passes when every case gives it. No record holds an outline, endpoint marks, feedback marks or refined points.

   Test Case Derivation: the fit criterion (point for point, and nothing that was not drawn), with SRS BR-004 and BR-005. The cases follow H-MUH-003 (b), where "Strokes are smoothed or corrected before they are saved". (d) is a near miss, the drawing most tempting to "tidy". In (e), the feedback of SAF-MOU-001 must not enter the record. (f) checks that no stroke leaks between attempts.

   How test will be performed: Playwright replays each file and compares the stored points with the file. Inspection item A5 of INS-EVAL checks the code path.

Table 28: Cases for T-REC-002 (FR-REC-002)

Case	Lesson, stage	Attempt	Expected strokes in the record
a	Line, T	`line.json`	Exactly the file's stroke
b	Square, T	`square-4-strokes.json`	Exactly the file's four strokes
c	Circle, T	`overlap-2-strokes.json`	Exactly the file's two strokes, separate
d	Circle, I	N	Exactly the file's stroke, unchanged
e	Square, E	`square-missing-side.json`, finished after the missing-side feedback is shown	Exactly the file's three strokes; nothing added for the missing side
f	Circle, T	F, then P	Record 1 holds only the stroke of F; record 2 holds only the stroke of P

3. T-REC-003: Difficulties and corrections

Purpose: a correction must not hide the difficulty, so the adult can see what the child found hard.

Type: Dynamic, automatic. Covers FR-REC-003.

Initial State: learner A with the square lesson at E (the circle lesson at T for case (d)).

Input: the cases vary the difficulties in the attempt and whether the child corrects them; the learner and the defaults stay fixed. The difficulty names are the catalogue's predefined difficulties for the square lesson at E; "missing side" is the incomplete-shape observation of T-LRN-012 (b). The cases are in Table 29.

Output: the expected output of each case is in its row of Table 29; the test passes when every case gives it. Each record lists exactly the difficulties in its row, each once, with the mark shown, and no others.

Test Case Derivation: the fit criterion (corrected and not corrected, each listed once, and no others) and the SRS Section 7.2 data dictionary

Table 29: Cases for T-REC-003 (FR-REC-003)

Case	Attempt	Expected difficulties in the record
a	`square-missing-side.json`, finished with no correction	Missing side (the incomplete-shape observation), not corrected
b	`square-missing-side-corrected.json`	Missing side, corrected
c	`square-endpoint-and-side.json`	Incorrect endpoint, corrected; missing side, not corrected
d	`circle.json` at T	None
e	`square-missing-side.json` with two pauses, each until the missing-side feedback appears, then finished	Missing side, not corrected, listed once

(an observed difficulty appears at most once in an attempt record, and a correction adds a mark without removing the difficulty). (c) mixes both marks in one attempt. (e) tries to make one difficulty appear twice. The cases follow H-MUH-007 (a), where "a later correction replaces the record of the difficulty it corrected".

How test will be performed: Playwright, pausing the replay until the feedback appears, then a database read.

4. T-REC-004: Adult-set values in use

Purpose: each record must keep the values used when the attempt was made, so a later change cannot alter what an earlier record means.

Type: Dynamic, automatic. Covers FR-REC-004.

Initial State: learner A with all eight adult-set values at their defaults. Each of cases (a) to (h) starts from a new learner profile with the defaults.

Input: in cases (a) to (h), the adult starts the circle lesson at I and replays P, which completes the lesson in one attempt (record 1). The adult then changes the one value in the row, with the adult check, and repeats the same steps (record 2). The other seven values stay at

their defaults. Cases (i) and (j) test a change during a lesson and two learners. The cases are in Table 30.

Table 30: Cases for T-REC-004 (FR-REC-004)

Case	Value changed	Record 1 holds	Record 2 holds
a	`STAGE_PASS`	80	70
b	`DIFFICULTY_STREAK`	4	3
c	`ATTEMPT_LIMIT`	8	6
d	`IMPROVE_WINDOW`	2	1
e	`ATTEMPT_EXTENSION`	2	1
f	`ATTEMPT_LIMIT_MAX`	10	8
g	`LESSON_IDLE_PROMPT`	Its default	The minimum of its range
h	`LESSON_IDLE_END`	Its default	The maximum of its range

Case	Input	Expected output
i	`STAGE_PASS` is changed from 80 to 70 during a lesson at T, then one N attempt	The record holds 70 with passed, or 80 with not passed; any other pairing fails
j	A at all minimums and B at all maximums, alternating on one device, one F each	Each record holds its own learner's eight values

Output: the expected output of each case is in its row of Table 30; the test passes when every case gives it. In (a) to (h), each record also holds the defaults for the other seven values, and record 1 is unchanged after the change.

Test Case Derivation: the fit criterion asks for records before and after a change to each adult-set value; the eight values are those of SRS BR-008. Cases (a) to (h) follow H-MUH-007 (b), where "Records refer to the learner's current adult-set values, which an adult may change at any time". The SRS does not say whether a change applies to a lesson already running, so in (i) the record must agree with the outcome it produced (H-AHM-007, stale setting). (j) is H-AHM-007, wrong learner. (g) and (h) use the ends of the `LESSON_IDLE_PROMPT` and `LESSON_IDLE_END` ranges, which are set for the test of SAF-RAY-009;

the minimums and maximums in (j) include them too.

How test will be performed: Playwright and a database read. (g), (h) and (j) are first run once those ranges are set.

5. T-REC-005: Interrupted attempt record

   Purpose: practice that the adult cuts off must still be visible, but never as a pass or a fail.

   Type: Dynamic, automatic. Covers FR-REC-005.

   Initial State: learners A and B; A in the square lesson at the stage in each case's row.

   Input: the cases vary the number of strokes before the interruption and how the adult interrupts; the square lesson and the defaults stay fixed. The cases are in Table 31.

Table 31: Cases for T-REC-005 (FR-REC-005)

Case	Actions	Expected records
a	At T, the first two strokes of `square-4-strokes.json`; the adult ends the session	One record for A: interrupted, the file's first two strokes, no outcome and no result
b	At E, the three strokes of `square-missing-side.json`; wait for the missing-side feedback; the adult switches to B	One record for A: interrupted, three strokes, missing side not corrected, no outcome; nothing for B
c	At T, one stroke of `square-4-strokes.json`; the adult ends the session	One interrupted record with one stroke
d	At T, the adult ends the session before any stroke	No attempt record

   Output: the expected output of each case is in its row of Table 31; the test passes when every case gives it.

   Test Case Derivation: the fit criterion: one or more strokes give an interrupted record with the strokes and no outcome, and no stroke gives no record. (c) is the one-stroke boundary and (d) is zero strokes. The cases follow H-MUH-004 (b).

How test will be performed: Playwright and a database read.

6. T-REC-006: Lesson record

   Purpose: the adult needs to see how far the child got in each lesson, how many attempts succeeded at each stage and how the lesson ended.

   Type: Dynamic, automatic. Covers FR-REC-006.

   Initial State: learner A with no lesson records. Each case is a new start of the circle lesson.

   Input: the cases vary the start stage and the sequence, written as results; learner A, the circle lesson and the defaults stay fixed. A start at T is the default start: the demonstration is watched, and then the attempts follow. As in the guided lesson tests, a written 40, 50 or 85 stands for a failing, a higher failing or a passing attempt and is replayed as F, N or P, whose results lie in the `CAL_*` bands, not at the written numbers. The cases are in Table 32.

   Output: the expected output of each case is in its row of Table 32; the test passes when every case gives it.

   Test Case Derivation: the fit criterion asks for a start at the first drawing stage and a later start chosen by the adult. In (a), the demonstration is watched first, so the start is T and no attempt is at D. The ending labels are the lesson endings of the SRS Section 7.2 data dictionary. The cases follow H-MUH-007 (c), whose mitigation is to "keep a lesson record for every lesson start". The counts follow the stage rules tested in T-LRN-016 to T-LRN-020. In (c), the child passes T twice, so T has two successes. In (d), the extension ends at $8 + 2 = 10$. (e) checks that each start makes a new record (SAF-RAY-006). Lessons that end through inactivity are left out, because the SRS Section 7.2 list of lesson endings has no label for them yet; T-SAF-017 covers inactivity.

   How test will be performed: Playwright and a database read.

7. T-REC-007: Unfinished lesson

   Purpose: a lesson the adult stops early must still record the stages the child already passed.

Table 32: Cases for T-REC-006 (FR-REC-006)

Case	Start	Sequence	Expected lesson record
a	T (the default start)	40, 85, 85, 40, 85	Start T, the first drawing stage, with no attempt at D; furthest I; successes T 1, E 1, I 1; ending "every stage passed"; extension count 0; five attempts
b	The adult starts at E	4×40 [D], 4×40	Start E; furthest E; successes all 0; ending "ended at `ATTEMPT_LIMIT`"; eight attempts
c	T	85, then at E 4×40 [A→T], then 85, 85, 85	Start T; furthest I; successes T 2, E 1, I 1; ending "every stage passed"; eight attempts
d	T	6×40, 4×50	Start T; furthest T; successes all 0; ending "ended when a granted extension ran out or `ATTEMPT_LIMIT_MAX` was reached"; extension count 1; ten attempts
e	T, after (b)	85, 85, 85	A second record (start T, furthest I, every stage passed); the record from (b) is unchanged

Type: Dynamic, automatic. Covers FR-REC-007.

Initial State: learners A and B; A starts the circle lesson at T.

Input: the cases vary how far A gets and how the lesson stops; the circle lesson, the start at T and the defaults stay fixed. The cases are in Table 33.

Table 33: Cases for T-REC-007 (FR-REC-007)

Case	Sequence and adult action	Expected lesson record
a	85, 40, 40; the adult stops the lesson	Ending "unfinished"; furthest E; successes T 1, E 0
b	85, 85; the adult switches to B	Ending "unfinished"; furthest I; successes T 1, E 1
c	40; the adult starts the square lesson	Circle record "unfinished"; furthest T; successes 0; square record in progress
d	85, 85, 85	Ending "every stage passed", not unfinished
e	8×40	Ending "ended at `ATTEMPT_LIMIT`", not unfinished

Output: the expected output of each case is in its row of Table 33; the test passes when every case gives it.

Test Case Derivation: the fit criterion: lessons stopped by an adult after known successes are unfinished, and a lesson whose last stage is passed is not. In (e), limit endings have their own label (SRS Section 7.2). (c) is the lesson change of T-LRN-015 (f). The cases follow H-MUH-007 (c), where "a lesson's progress is saved only when the lesson ends at its limit".

How test will be performed: Playwright and a database read.

8. T-REC-008: Each attempt saved exactly once

Purpose: a retried save or a second open session must not store an attempt twice or under another lesson, because the adult relies on the counts.

Type: Dynamic, automatic. Covers FR-REC-008.

Initial State: learner A with no records.

Input: the cases vary how the save is disturbed and how many sessions are open; learner A and the defaults stay fixed. The cases are in Table 34.

Table 34: Cases for T-REC-008 (FR-REC-008)

Case	Setup	Expected records
a	Circle lesson at T: F, F, F, F, P. For each attempt, the first save request is cut off and the network comes back after `RETRY_DELAY`	Five attempt records, one per attempt, under this lesson record; one success at T
b	Circle lesson at T: F, F, F. The server stores each save, but the response is cut off, so the browser saves again	Three attempt records; no duplicates
c	Two sessions (two browser contexts, same adult), both for A: the circle lesson in one and the square lesson in the other; attempts alternate F and `square-missing-side.json`, three each	The circle record has three attempts and the square record three; none under the other
d	Two sessions, both start the circle lesson for A; F three times in each, alternating	Two circle lesson records with three attempts each
e	The finishing action sent twice within 100 ms	One attempt record

Output: the expected output of each case is in its row of Table 34; the test passes when every case gives it. No two attempt records hold the same strokes from the same moment.

Test Case Derivation: the fit criterion cuts and restores the connection during each save, and opens two sessions at once. (b) is the case most likely to duplicate: the server saved, but the browser cannot know it. (e) is a double tap. The cases follow H-MUH-004 (a) and H-RAY-002. Offline attempts saved on reconnection are tested by T-PERF-005 and T-SAF-003.

How test will be performed: Playwright request routing. For (b), the

request reaches the server through `route.fetch()`, then the route is aborted. A database read follows.

9. T-REC-009: Saved records do not change

   Purpose: later changes to settings or lessons must never rewrite what an earlier record says.

   Type: Dynamic, automatic. Covers FR-REC-009.

   Initial State: `history-seed.json` is loaded for learner A, whose eight adult-set values are at the minimums of their ranges. The file holds three lesson records with 19 attempt records in total: (1) circle, every stage passed; (2) square, ended at `ATTEMPT_LIMIT`; (3) line, unfinished. Then, in a second browser context, the adult starts the circle lesson for A at E with the adult check, and A makes one attempt with F. This lesson stays open through step (f) and is record (4). A SHA-256 hash of the canonical JSON of every attempt record (20 in total) and of records (1) to (3) is stored.

   Input: the steps vary the change; learner A, the seeded records and the stored hashes stay fixed. The steps run in order, each followed by a comparison with the stored hashes. The steps are in Table 35.

   Output: the expected output of each step is in its row of Table 35; the test passes when every step gives it.

   Test Case Derivation: the fit criterion changes every adult-set value, a starting stage and the other settings (SRS UC-010), and adds a lesson to and removes a lesson from the catalogue. (e) is a catalogue edit (SRS LC-001) that could re-score old attempts. The edit is to the square lesson, whose record (2) has ended, so it cannot change record (4), a circle lesson. In (g), the record of a lesson that has not ended may still grow. Record (4) holds one failure before (g), so (g) gives no suggestion at either end of the `DIFFICULTY_STREAK` range. (h) tries the direct route; its audit event is tested by T-SEC-003. The steps follow H-MUH-007 (d).

   How test will be performed: a Playwright script runs the steps and recomputes the hashes from the database. Until the `LESSON_IDLE_PROMPT` and `LESSON_IDLE_END` ranges are set, those two values keep their de-

Table 35: Cases for T-REC-009 (FR-REC-009)

Step	Change	Expected output
a	The adult changes every adult-set value to the maximum of its range	Every stored hash matches
b	The adult changes the circle lesson's starting stage to E	Every stored hash matches
c	The adult turns audio off and refinement off, and changes each accessibility setting once the SRS defines them	Every stored hash matches
d	A new lesson is added to the catalogue	Every stored hash matches
e	The outline and feedback text of the square lesson are edited in the catalogue	Every stored hash matches
f	The line lesson is removed from the catalogue	Every stored hash matches; record (3) and its attempts are still shown with the lesson's name
g	In the second context, A continues record (4) with F, then P	Every stored hash matches; record (4) has two more attempts, and its earlier attempts are unchanged
h	A direct request, as the adult, to change the outcome of the first attempt record of record (1) to passed	Refused; every stored hash matches

faults in the seed and in (a). Until the SRS defines the accessibility settings, (c) changes only audio and refinement.

10. T-REC-010: Suggestions and responses

    Purpose: the record must show each go-back suggestion and what the adult did with it, so the adult can see why a later attempt was at an earlier stage.

    Type: Dynamic, automatic. Covers FR-REC-010.

    Initial State: learners A and B; A in the circle lesson at E with `DIFFICULTY_STREAK` = 3.

    Input: the cases vary the adult's response to each suggestion and how the lesson continues; the circle lesson and `DIFFICULTY_STREAK` 3 stay fixed, and every case except (g) starts at E. The cases are in Table 36.

Table 36: Cases for T-REC-010 (FR-REC-010)

Case	**Sequence**	**Expected suggestions in A's lesson record**
a	3 × F [A→T]	One: stage T, accepted
b	3 × F [D]	One: declined
c	3 × F [D], 3 × F [D]	Two, in order, both declined
d	3 × F; the record is read while the lesson runs and nobody answers	One: pending
e	3 × F; the adult stops the lesson without answering	One: ignored
f	3 × F; the child taps accept (the adult check is not completed); then the adult declines	One: declined, never accepted
g	At T, 7 × F	None
h	3 × F; the adult switches to B and starts the circle lesson for B at T; B makes one attempt	A's record: one, ignored; B's record: none

    Output: the expected output of each case is in its row of Table 36; the test passes when every case gives it. Each lesson record lists exactly the suggestions in its row, each once, with that status, and no others.

Test Case Derivation: the fit criterion asks for suggestions that are accepted, declined and not yet answered, each listed once. The statuses come from the SRS Section 7.2 data dictionary (pending, accepted, declined or ignored). A pending suggestion becomes ignored when the lesson ends, or the stage changes, without an answer, which is the team's reading of "ignored". In (f), only an adult can accept (SAF-RAY-001). In (g), T has no earlier stage. In (h), suggestions belong to their learner (SAF-RAY-007), and switching ends A's lesson as unfinished (T-REC-007).

How test will be performed: Playwright and a database read.

4.1.5 Progress Summaries

These tests cover the twelve progress summary requirements in SRS Section 9.5 (FR-PRG-001 to FR-PRG-012). FR-PRG-007 is due in PH-3 and the rest in PH-2. They follow UC-009 (review progress) and UC-010 (review history and change settings). A summary is built only from the attempt and lesson records of SRS Section 9.4, which the attempt record tests (Section 4.1.4) check, so these tests load prepared records straight into the test database and check what the summary makes of them. The tests try to cause the failures in Hazard Analysis rows H-MUH-003, H-MUH-004, H-MUH-008 and H-RAY-003: assisted work shown as the child's own, one learner's records shown under another, a summary that misleads the adult, and a summary that judges the child. Two requirements also need adult reviewers (FR-PRG-003 and FR-PRG-005); T-PRG-013 runs those tasks in the adult usability sessions instead of a separate survey. FR-PRG-012 is checked by inspection, with an automatic scan as a second approach. SAF-RAY-004 (no diagnosis or comparison) and SAF-RAY-006 (limit endings) are tested by T-SAF-004 and T-SAF-006 and not repeated here.

Set-up. One test adult account holds learners A to E, whose histories are listed under *Prepared test data for the functional tests* in the Symbolic Parameters appendix. A second test adult account holds learner F, with a history like A's. Each test loads the histories it names into an empty test database, as in T-REC-009. The adult-set values are at their defaults. "With a guide" means stages T and E, and "without a guide" stage I, as in SRS Section 9.5. Learner A's history is the main one:

- circle, three lesson records, started 2, 9 and 16 November 2026;

- square, one record, started 3 November, ended at `ATTEMPT_LIMIT` at T;
- line, one record, started 10 November, unfinished.

Its 32 attempts are listed in Table 37. The records are stored out of start order (16 November before 9 November), so a summary that lists records in storage order fails.

A summary's figures and items are read from the page by their accessible names, so each count can be matched to its lesson, stage and kind. To get the expected attempts, the script computes them from the history file, not from Squigglr.

1. T-PRG-001: One learner per summary

   Purpose: an adult must never see one learner's records in another learner's summary, especially an educator with several learners.

   Type: Dynamic, automatic. Covers FR-PRG-001.

   Initial State: learners A to E in the first account and learner F in the second, each with their history.

   Input:

   (a) Signed in to the first account, open the summary of each of A to E in turn.

   (b) Open A's summary, and switch to B's while A's is still loading (the test set-up holds the answer for A's records for 2 seconds).

   (c) Signed in to the second account, open the web address of A's summary.

   Output:

   (a) Each summary names its learner and contains exactly that learner's records: every attempt and lesson record ID in it is in the learner's history, and every ID in the history is in it.

   (b) The page ends showing B's summary with only B's records; nothing of A's appears, even briefly after A's answer arrives.

   (c) The summary is refused, and none of A's records is shown.

Test Case Derivation: the fit criterion of FR-PRG-001 asks for at least three learners with known histories; (a) uses five. Case (b) tries the cause of H-MUH-004 where a late answer fills the wrong learner's view. Case (c) checks the account limit of BR-001; F's history looks like A's, so a leak between accounts would not show as an empty page.

How test will be performed: a Playwright script collects the record IDs behind every item on the page and compares them with the history file.

2. T-PRG-002: Progress over time by lesson

Purpose: the adult must see, lesson by lesson, how far the child got each time and how each lesson ended, so change over time is visible.

Type: Dynamic, automatic. Covers FR-PRG-002.

Initial State: learner A's history.

Input: open A's summary. The lesson records in the history are in Table 37; "P/F" is the number of passed and failed attempts at each stage, and "–" means the stage was not attempted.

Table 37: Learner A's lesson records and the expected summary for T-PRG-002 (FR-PRG-002)

Lesson	**Started**	**T (P/F)**	**E (P/F)**	**I (P/F)**	**Furthest**	**Expected ending**
Circle	2 Nov	1/2	0/8	–	E	Ended at the limit
Circle	9 Nov	1/0	1/1	0/2	I	Unfinished
Circle	16 Nov	1/0	1/0	1/2	I	Every stage passed
Square	3 Nov	0/8	–	–	T	Ended at the limit
Line	10 Nov	1/1	0/1	–	E	Unfinished

Output: the summary shows three lessons. Under each, its records are listed in start order (circle: 2, 9, then 16 November), each with the starting stage T, the furthest stage, the successful attempts at each stage (the P column) and the ending in the table.

Test Case Derivation: the fit criterion of FR-PRG-002 names a history showing improvement (the three circle records), a lesson ended at `ATTEMPT_LIMIT` (the first circle record and the square) and an unfinished lesson (the second circle record and the line). An unfinished lesson shown as failed would mislead the adult (H-MUH-008). The records are stored out of order to catch a summary sorted by storage instead of by start.

How test will be performed: a Playwright script reads each record's row in the summary and compares it with the table.

3. T-PRG-003: Attempts with and without a guide kept apart

Purpose: assisted work must never be counted as the child's independent work, or an adult could move the child on too soon.

Type: Dynamic, automatic. Covers FR-PRG-003 (the counting part; the reviewer part is T-PRG-013).

Initial State: the histories of learners A and D.

Input: open each summary. The expected counts are in Table 38, computed from Table 37 for A.

Table 38: Expected attempt counts for T-PRG-003 (FR-PRG-003)

Case	**Learner, lesson**	**With a guide (passed of all)**	**Without a guide (passed of all)**
a	A, circle	5 of 16	1 of 5
b	A, square	0 of 8	None
c	A, line	1 of 3	None
d	D, circle (started at I)	None	1 of 2

Output: every count or list in each summary is labelled as with a guide or without a guide and matches its cell in Table 38. Where a kind has no attempts, the summary says so instead of showing a count of the other kind. No figure is labelled as covering both kinds, and no list mixes them.

Test Case Derivation: the fit criterion of FR-PRG-003. Case (a) has both kinds. In (b) and (c) only guided attempts exist, and in (d)

only unguided ones, where a combined figure would equal the one kind present and could pass unnoticed unless it is labelled. This is the cause in H-MUH-003 where assisted work looks independent.

How test will be performed: a Playwright script reads every count and list, with its label, from the page.

4. T-PRG-004: Recurring difficulties

   Purpose: the summary must show what keeps going wrong, using exactly the rule the SRS gives for "recurring".

   Type: Dynamic, automatic. Covers FR-PRG-004.

   Initial State: learner B's history, generated by a script from the value of `RECUR_MIN` (written r below), so the file follows the constant. All attempts are in the square lesson. The difficulties are the catalogue's predefined ones for the square, as in T-REC-003.

   Input: open B's summary. The cases are in Table 39.

Table 39: Cases for T-PRG-004 (FR-PRG-004)

Case	**Attempts in B's history**	**Expected in the summary**
a	Missing side at E in exactly r attempts, one of them corrected	Listed for square at E: r attempts, 1 corrected
b	Incorrect endpoint at E in $r-1$ attempts	Not listed
c	Missing side at T in $r-1$ attempts (as well as case (a) at E)	Not listed for T; case (a)'s count stays r
d	Missing side at I in r attempts, split across two lesson records, all corrected	Listed for square at I: r attempts, r corrected
e	After the summary is read, one more attempt with incorrect endpoint at E is added, and the summary is reopened	Case (b) is now listed: r attempts, 0 corrected

   Output: the summary lists exactly the difficulties marked "listed" in Table 39, with their counts, and nothing else.

Test Case Derivation: the fit criterion of FR-PRG-004 asks for one difficulty at exactly `RECUR_MIN` and another one fewer, (a) and (b). Case (c) checks "at the same stage": adding T to E would wrongly give $2r - 1$. Case (d) checks that records of the same lesson are counted together, and that corrections do not hide a difficulty (BR-007, FR-REC-003). Case (e) crosses the boundary from below.

How test will be performed: a Playwright script reads the list of recurring difficulties and compares it with the table.

5. T-PRG-005: Opening the attempts behind the summary

   Purpose: an adult who doubts a number must be able to see the drawings it came from, and exactly those.

   Type: Dynamic, automatic. Covers FR-PRG-005 (the automatic part; the reviewer part is T-PRG-013).

   Initial State: the histories of learners A and B.

   Input: in each summary, every figure and item is opened in turn: each count of T-PRG-002 and T-PRG-003, each recurring difficulty of T-PRG-004, and each lesson record.

   Output: each one opens exactly the attempt records it was computed from, by ID, as computed from the history file. For example, "circle, with a guide, 5 passed" opens the five passed T and E attempts in Table 37. A figure of zero opens an empty list that says no attempts match. Every opened list belongs to the summary's learner.

   Test Case Derivation: the fit criterion of FR-PRG-005, applied to every figure instead of a sample, because one wrong link is enough to mislead an adult (H-MUH-008). Zero is the case most likely to open the wrong list or fail.

   How test will be performed: a Playwright script opens each item, reads the record IDs it lists, and closes it.

6. T-PRG-006: The attempt record view

   Purpose: the adult must see everything Squigglr recorded about an attempt, with the strokes exactly as the child drew them.

Type: Dynamic, automatic. Covers FR-PRG-006.

Initial State: the histories of learners A to C, and learner H's history, in which an adult-set value changes in the middle of a lesson.

Input: open these attempt records from the summary:

(a) a passed attempt at T, and a failed attempt at I (A);

(b) an attempt at E with missing side corrected and incorrect endpoint not corrected (B);

(c) an interrupted attempt (C);

(d) the attempts just before and just after the value change (H);

(e) an attempt of 40 strokes, one of them a single point (A).

Output: each view shows the lesson, stage, outcome (or the interrupted mark in (c), with no outcome), date and time, each difficulty with its correction mark, and the adult-set values in use. In (d), the two views show the values before and after the change. The strokes drawn in each view equal the recorded strokes: the same number, order and boundaries, and every point within `COORD_TOL`. In (e), the single-point stroke is shown.

Test Case Derivation: the fit criterion of FR-PRG-006 lists the values; each case adds one that is easy to drop: a correction mark, an interrupted attempt with no outcome, values that differ between attempts, and a one-point stroke (BR-004).

How test will be performed: a Playwright script reads the fields and the stroke list the view draws, and compares them with the record in the test database.

7. T-PRG-007: The adult chooses what the summary shows

Purpose: the adult must be able to narrow the summary without changing the records or mixing guided and independent attempts.

Type: Dynamic, automatic. Covers FR-PRG-007 (PH-3).

Initial State: learner A's history. A hash of A's records is stored, as in T-REC-009.

Input: the cases are in Table 40.

Table 40: Cases for T-PRG-007 (FR-PRG-007)

Case	Choice	Expected summary
a	None (as first opened)	All five lesson records
b	Circle only	The three circle records
c	Dates 9 to 16 November	The circle records of 9 and 16 November, and the line record
d	Circle, and dates 9 to 16 November	The circle records of 9 and 16 November
e	The adult-set values hidden, then shown	The values hidden in the summary and in opened records, then shown again
f	Dates ending at the exact time of one attempt	That attempt is in a count and in the list the count opens, or in neither

Output: each case shows what its row says, with attempts with and without a guide still separate as in T-PRG-003, and after every case the stored hash of A's records still matches.

Test Case Derivation: the fit criterion of FR-PRG-007: choices change only what is shown, records stay the same, and the two kinds stay separate. The SRS says the summary covers all records until the adult chooses, (a). It does not say whether a date range includes its end, so (f) checks only that a figure and the records it opens agree (FR-PRG-005).

How test will be performed: a Playwright script makes each choice, reads the summary as in T-PRG-002, and hashes the records afterwards.

8. T-PRG-008: Empty history

Purpose: a learner with no recorded practice must not look like a learner who failed.

Type: Dynamic, automatic. Covers FR-PRG-008.

Initial State: learner E, with no records; learner A's history; a learner G, added for this test, with three saved creative drawings and no at-

tempts.

Input:

(a) Open E's summary.

(b) Open A's summary with the dates 1 to 31 December 2026 chosen (from PH-3, when FR-PRG-007 is built).

(c) Open G's summary.

Output: each summary states that no practice is recorded, and shows no stage, no count (not even 0), no difficulty, no ending and no date of a latest attempt.

Test Case Derivation: the fit criterion of FR-PRG-008 names (a) and (b). Case (c) checks that creative drawings are not counted as practice (FR-CRT-028). A summary of zeros reads as failure (H-MUH-008, H-RAY-003).

How test will be performed: a Playwright script reads the page and fails on any stage name, number or difficulty in the summary area.

9. T-PRG-009: Interrupted attempts shown apart

Purpose: an interrupted attempt has no outcome, so it must count neither as a failure nor as a success.

Type: Dynamic, automatic. Covers FR-PRG-009.

Initial State: learner C's history: three interrupted attempts at T of the circle lesson, and nothing else. Learner A's history.

Input:

(a) Open C's summary.

(b) Add four interrupted attempts to A's history, one at each stage of the circle record of 16 November and one at T of the square record, and open A's summary.

Output:

(a) The summary shows three interrupted attempts at T of the circle lesson, no successful or unsuccessful attempt, and no stage passed.

(b) Every count of T-PRG-002 and T-PRG-003 is unchanged, and the four interrupted attempts are shown separately at their lessons and stages.

Test Case Derivation: the fit criterion of FR-PRG-009, with both parts: a history of only interrupted attempts, and interrupted attempts added to a known history. (b) adds one at every stage, so a count that takes them in anywhere changes.

How test will be performed: a Playwright script compares the counts with those of T-PRG-002 and T-PRG-003.

10. T-PRG-010: Stages not attempted

    Purpose: a stage the child never tried must not look like a stage the child could not draw.

    Type: Dynamic, automatic. Covers FR-PRG-010.

    Initial State: learner D's history: a circle lesson the adult started at I, with a failed attempt at I and then a passed one, which completed the lesson; and a square lesson with three failed attempts at T, unfinished. Learner A's history.

    Input: open D's summary, then A's.

    Output: for D's circle, T and E are marked not attempted and I shows 1 successful attempt; for D's square, T shows 0 successful attempts and is marked attempted, and E and I are marked not attempted. For A, every "–" in Table 37 is marked not attempted, and every 0 in a P column whose stage was attempted is shown as 0 successful attempts.

    Test Case Derivation: the fit criterion of FR-PRG-010 asks for a lesson started at a later stage (D's circle, BR-003) and a stage attempted only without success (D's square at T). A's history adds stages not reached because the lesson ended first.

    How test will be performed: a Playwright script reads each stage's mark and count.

11. T-PRG-011: Latest attempt included

Purpose: an adult must be able to tell when recent practice is missing from a summary because it has not been saved yet.

Type: Dynamic, automatic. Covers FR-PRG-011.

Initial State: learner A's history, whose latest attempt is at 10:42:00 on 16 November 2026, and a controlled clock. Two browser contexts: the child's and the adult's.

Input:

(a) In the child's context, the connection is cut and A makes two attempts, which are held back (SAF-RAY-003). The adult opens A's summary in the other context.

(b) The connection returns, the saves are confirmed, and the adult reopens the summary.

Output:

(a) The summary states 16 November 2026, 10:42, and contains 32 attempts, none of the two held back.

(b) The summary states the time of the second new attempt, and contains 34 attempts.

Test Case Derivation: the fit criterion of FR-PRG-011. Case (b) checks that the date moves on once the attempts are saved, so the statement is not fixed when the summary is first built.

How test will be performed: a Playwright script cuts the child's connection with `context.setOffline()` and uses a controlled clock.

12. T-PRG-012: No overall score

Purpose: Squigglr does not judge a child's ability, so no summary may sum up the child as one score, grade or rating.

Type: Static (inspection), manual, with an automatic scan. Covers FR-PRG-012.

Initial State: the summaries of learners A to E at that revision.

Input:

(a) Inspection. Rayan, who owns SAF-RAY-004, and one team member who did not write the summary view read every summary, at the safety inspection before Rev 0 and Rev 1 (as for T-SAF-004). They look for any number, percentage, star, bar, colour scale, level or word (such as "overall", "average", "score", "grade" or "rating") that combines results from more than one stage.

(b) Scan. A script lists every number and percentage in every summary with its accessible name, and checks that each belongs to one stage of one lesson, or is a count of attempts or records, not a result. It also searches the summary text for the words in (a) and for the terms in `forbidden-terms.txt`.

Output: the inspectors find no such figure, and the scan finds no number that combines results of more than one stage, no percentage computed across stages and none of the words. Each finding is recorded as a GitHub issue labelled `safety`.

Test Case Derivation: the fit criterion of FR-PRG-012 is an inspection. A score can appear in forms no word list catches, like a chart, so the inspection is the main check. The scan is a second approach that runs on every PR and catches a score added between inspections. Counts of attempts are allowed: they are not results.

How test will be performed: (a) during the safety inspection, with findings recorded by the Testing Lead; (b) in CI with the Playwright tests.

13. T-PRG-013: Adults find the independent attempts and the attempts behind the summary

    Purpose: correct figures are not enough; adults must be able to read them. This is the reviewer part of the fit criteria of FR-PRG-003 and FR-PRG-005.

    Type: Dynamic, manual. Covers FR-PRG-003 and FR-PRG-005.

    Initial State: the summaries of learners A and B on the test iPad; at least `ADULT_REVIEWERS` adult reviewers, none of them on the team, who have not seen Squigglr's summaries before. No child takes part (SRS CON-005).

Input: each reviewer does the tasks in Table 41, without help, as part of the adult usability sessions.

Table 41: Reviewer tasks for T-PRG-013 (FR-PRG-003, FR-PRG-005)

Task	Question (learner)	Success	For
R1	In the circle lesson, how many attempts did the child make without a guide? (A)	Answers 5	003
R2	Which circle attempts on 16 November were made without a guide? (A)	Points to exactly the three I attempts of that record	003
R3	Did the child make any attempt without a guide in the square lesson? (A)	Answers no	003
R4	Open the attempts with a missing side at the endpoint stage of the square. (B)	Opens exactly the r attempts of T-PRG-004 (a)	005
R5	Open the successful attempts at the endpoint stage of the circle lesson. (A)	Opens exactly the 2 passed E attempts	005
R6	Open the attempts of the square lesson that ended at the limit. (A)	Opens exactly its 8 attempts	005

Output: at least `ADULT_REVIEWERS` reviewers take part, and at least `REVIEW_PASS_RATE` of the tasks succeed, counted separately for R1 to R3 (FR-PRG-003) and R4 to R6 (FR-PRG-005).

Test Case Derivation: the reviewer parts of the fit criteria of FR-PRG-003 and FR-PRG-005. Each task has one right answer, known from the history files, so success is not a matter of opinion. R3 is the trap case: a reviewer who reads a combined count would answer yes.

How test will be performed: in the adult usability sessions that Shamil and Ahmed run (Software Validation section). These six tasks are added to the session script in the Usability Survey Questions appendix (#198), so the team runs one set of sessions, not a second survey. The notetaker records each task as a success or failure, and the results go in the VnV Report. A rate below `REVIEW_PASS_RATE` is a defect in the

summary design, which is changed and tested again in the next round.

4.2 Tests for Nonfunctional Requirements

These tests cover the nonfunctional requirements in SRS Sections 10 to 17, with one subsection per group: usability and accessibility, performance, safety-critical, operational and environmental, maintainability and support, security, cultural, and compliance. Most have a pass condition. Speed tests have one too, but one run can be slow by chance, so they repeat each run and report the results as tables and plots in the VnV Report. Where a requirement can't be tested by running Squigglr, like the wording in a template, we check it with a review or an inspection and say who does it, when, and with which checklist. The usability tasks and survey questions are in the Usability Survey Questions appendix.

4.2.1 Performance

These tests cover the performance requirements in SRS Section 12 (NFR-PERF-001 to NFR-PERF-005, NFR-PERF-008 and NFR-PERF-009). Timing thresholds are written as symbolic constants. The SRS leaves their values to this plan, so each constant in the Symbolic Parameters appendix states how its value is set and when.

Test environment. Squigglr is a web app, so in principle it runs on any device with a modern browser, and on any network. Testing every device and every connection is not realistic for a team of five, and it is not needed to show that the requirements are met. The team will instead run the timing tests on one tablet, an iPad with a stylus, which is the device the app is designed around (SRS ASM-003). The tests run in a controlled setting: the iPad is charged, Low Power Mode is off, every other app and browser tab is closed, automatic updates are paused, and the iPad uses the same Wi-Fi network for every run. Keeping these conditions the same means that a change in a measured time comes from a change in Squigglr, not from the device or the network. Automatic tests that do not need a physical stylus run in Playwright on a team laptop under the same rules.

Why each test is repeated. Even in a controlled setting, one measurement can be slow by chance, for example when the browser collects garbage.

Each timing test is therefore repeated `PERF_RUNS` times, and the VnV Report gives the minimum, median, 95th percentile and maximum of the measured times next to the threshold. A test passes only when the 95th percentile and the maximum are both within the threshold. This way a single slow run is still visible, but one unlucky run does not decide the result.

Limits. Because the tests use one device and one network, they show that Squigglr meets its timing requirements on the device it is designed for. They do not show how it performs on older devices or slower networks. Other browsers and devices are checked for correct operation, not speed, in T-PERF-007.

1. T-PERF-001: Stroke feedback latency

   Purpose: a child should see each stroke appear as they draw it; a visible delay makes drawing feel broken and can frustrate a child who is still building fine motor control.

   Type: Dynamic, automatic (Playwright) and manual (physical stylus).

   Initial State: a guided lesson is open at each stage where the child draws (tracing, reduced guidance and independent drawing) with no earlier attempts; no other tabs are open.

   Input: the prepared stroke files `line.json`, `circle.json` and `square-4-strokes.json`, replayed as pointer events by Playwright, `PERF_RUNS` times each; then `PERF_RUNS` strokes drawn by hand with the stylus on the iPad.

   Output: for every stroke, the time from the last pointer event to the first frame that shows the stroke is at most `FEEDBACK_LATENCY_MAX`.

   Test Case Derivation: NFR-PERF-001 requires visual acknowledgement within `FEEDBACK_LATENCY_MAX` of the final input point. Multi-stroke shapes are included because they create the most rendering work per attempt. The replayed strokes give exactly the same input on every run, so they are the main measurement. A person cannot draw the same stroke twice, so the hand-drawn strokes are not meant to be identical: only the time to show each stroke is measured, not how accurate it is. They are there because replayed events skip the real stylus input path on the iPad, which could add its own delay.

How test will be performed: the replay script stamps `performance.now()` at the last pointer event and in the next `requestAnimationFrame` callback after the stroke is drawn. For the manual runs, the iPad is connected to a Mac and the same interval is read from a Safari Web Inspector timeline recording. Times are summarized as described above.

2. T-PERF-002: Refinement latency and canvas responsiveness

   Purpose: refinement runs on an outside service that can be slow, so this checks that the refined drawing arrives in time and that the child can keep drawing while it waits.

   Type: Dynamic, automatic.

   Initial State: the creative space is open with refinement on and the refinement service reachable.

   Input: the prepared drawings `house.json` and `sun.json` are replayed and finished, `PERF_RUNS` times each; during the wait, a further stroke is replayed every 200 ms.

   Output: the refined result appears within `REFINE_LATENCY_MAX` of finishing, and every stroke replayed during the wait is drawn and saved without an error.

   Test Case Derivation: NFR-PERF-002 requires both the time limit and an unblocked canvas while the refinement is pending.

   How test will be performed: the script records the time from the finish action to the first frame showing the refined result, and checks the canvas contents after each extra stroke.

3. T-PERF-003: Progress summary load time

   Purpose: a parent or educator with a long history of saved attempts should not be left waiting for the progress summary to load.

   Type: Dynamic, automatic.

   Initial State: an adult account with one child profile whose history holds `HISTORY_SIZE_LARGE` attempts across three lessons, logged in, with the browser cache cleared.

Input: the adult opens the progress summary ten consecutive times.

Output: each time, the summary is fully visible within `SUMMARY_LOAD_MAX`.

Test Case Derivation: the fit criterion of NFR-PERF-003 names ten consecutive requests. A large history is used because load time grows with the number of records.

How test will be performed: Playwright measures from the click to the moment the last chart element is visible. The test is repeated with histories of 10 attempts and `HISTORY_SIZE_LARGE` attempts, and the two sets of times are reported together to show how load time grows with history size.

4. T-PERF-004: Deterministic stage result

   Purpose: the adult relies on pass and fail results to choose the next level, so the same drawing must always get the same result.

   Type: Dynamic, automatic (unit test in Vitest).

   Initial State: the stage evaluation module is loaded with the criteria for each prepared stage.

   Input: the prepared stroke files from T-PERF-001, plus `circle-near-miss.json`, a drawing just outside `STAGE_PASS`, each evaluated ten consecutive times.

   Output: all ten results for each input are identical, including the pass or fail decision.

   Test Case Derivation: NFR-PERF-004 requires the same strokes and criteria to give the same result. The near miss checks that a borderline result does not change between runs.

   How test will be performed: a Vitest test calls the evaluation function directly and compares every result with the first. A second run in Playwright replays the same strokes through the user interface, to confirm that input handling does not change the result.

5. T-PERF-005: Submitting attempts after reconnection

Purpose: home and school Wi-Fi can drop; a child's work done while offline must still reach the server once the connection returns, without being lost or saved twice.

Type: Dynamic, automatic.

Initial State: a guided lesson is open; the network is online.

Input: the network is set offline in Playwright; the prepared attempt `circle.json` is completed; the network is restored. This is done three consecutive times, then repeated with three attempts completed while offline.

Output: after each reconnection, every attempt completed offline is saved on the server exactly once, in the order made, and none is lost or duplicated.

Test Case Derivation: NFR-PERF-005 names three consecutive runs. The three-attempt case checks the order of a queue and catches duplicates.

How test will be performed: after each run, the script reads the child's attempt records through the test database and compares them with the attempts made. This test is shared with SAF-RAY-003 and SAF-SHA-003 (safety-critical tests below).

6. T-PERF-006: Concurrent session capacity

Purpose: several classrooms or households may use Squigglr at once; this checks that the speed limits still hold under that load.

Type: Dynamic, automatic (load test).

Initial State: the preview deployment, with `CONCURRENT_CAPACITY` test adult accounts and one child profile each.

Input: a load script runs `CONCURRENT_CAPACITY` simulated sessions at once; each replays a lesson of ten attempts and then opens the progress summary.

Output: no request fails, and the measurements of T-PERF-001 to T-PERF-003, taken in one extra session during the load, stay within their thresholds.

Test Case Derivation: NFR-PERF-008 requires the speed limits to hold under the agreed load.

How test will be performed: the load script runs simulated sessions in Playwright; one further session takes the timing measurements. The run is repeated at a quarter, half and all of `CONCURRENT_CAPACITY`, and the times are plotted against the number of sessions in the VnV Report.

7. T-PERF-007: Squigglr keeps working until the end of the capstone

   Purpose: Squigglr must keep working on the supported devices and browsers at least until the end of the capstone, the final demo on 15 March 2027 (PH-3 in the SRS). Browsers update often, and an update can break a feature without any change to our code, so this checks every feature at each milestone.

   Type: Dynamic, manual.

   Initial State: the release build at each milestone: the proof-of-concept demo (PH-1, 23 November 2026), Rev 0 (PH-2, 1 February 2027) and the final demo (PH-3, 15 March 2027).

   Input: on the iPad and on each other combination in `SUPPORTED_DEVICES` that the team has access to, a team member follows the checklist of every feature: create an account and a child profile, run one guided lesson to the end, draw in the creative space, restore an original, and open the progress summary.

   Output: every checklist item works on every combination. Any combination that fails is removed from `SUPPORTED_DEVICES`, and OI-09 is updated.

   Test Case Derivation: the fit criterion of NFR-PERF-009.

   How test will be performed: a team member runs the checklist by hand at each milestone. Running it at the two earlier milestones, not only at the final demo, means a browser update that breaks a feature is found while there is still time to fix it. The results are kept as a table of device by feature in the VnV Report.

4.2.2 Safety-Critical Requirements

These tests cover the 18 safety-critical requirements in SRS Section 12.2, which come from the Hazard Analysis. A failure here could harm a child's well-being, privacy or records, so each requirement gets two different kinds of evidence: a test below, and the safety inspection described next. Every test below is dynamic except T-SAF-004, which checks wording in templates and so is an inspection; T-SAF-016 combines a code review with recordings on the iPad. The tests try to break each safeguard, for example by having a child tap adult-only controls, cutting the connection during an attempt, or switching between children on a shared device. All tests use test accounts and prepared data only; no child takes part (SRS CON-006). In these tests, "the adult" means the parent or educator responsible for the child, and "the adult check" means the check in `ADULT_CHECK` (for example a PIN or password) that only the adult can complete.

Safety inspection (SI). Before each revision (Rev 0 and Rev 1), the code that implements each requirement below is inspected by two team members who did not write it. The Backend Lead (Rayan Nasrallah) runs the inspection and the Testing Lead (Muhammad Huzaifah) records the results. For each requirement, the inspectors:

1. find every code path that triggers the safeguard, using the test-to-requirement traceability table;
2. check that no path skips it, for example an adult-only action reachable from a URL or a save that reports success before the server responds;
3. check that a dynamic test exercises each path, and add one if not;
4. check that errors are handled (every failed request leaves the child's record unchanged and the adult is told) and that every input from the browser is validated on the server;
5. record each finding as a GitHub issue labelled `safety`, and re-inspect after the fix.

The inspection passes when every requirement has no open `safety` issue. Results are reported in the VnV Report.

Why this is enough. Testing cannot prove that a safeguard always works; it can only raise confidence that it does. One test per requirement, each with cases chosen to break the safeguard, plus an inspection that looks for paths the test missed, gives two independent checks. A second dynamic test is added wherever the inspection finds an untested path. The inspection covers only the code tied to these 18 requirements, because a full review of every module by two people is not realistic in the time the team has. The rest of the code is covered by the code checklist and the normal PR review.

How a lesson works. Several of these tests check the limits in a guided lesson, so this paragraph summarizes the lesson flow from SRS BR-002, UC-002 to UC-005 and the state machine in SRS Section 9.2. The adult picks the child and the lesson and completes the adult check. The child then watches a demonstration and works through three stages in order: tracing a full outline, drawing between endpoints, and drawing with no guide. An attempt that meets `STAGE_PASS` moves the child to the next stage automatically, and a pass at the last stage completes the lesson. Failed attempts are counted *per stage*, and the count starts again whenever the stage changes:

- after `DIFFICULTY_STREAK` failed attempts in a row, Squigglr suggests to the adult that the child go back a stage; the stage changes only if the adult completes the adult check and accepts, and the child keeps practising while the suggestion waits;
- after `ATTEMPT_LIMIT` failed attempts at one stage, the lesson ends automatically, unless the child is improving, in which case the child gets up to `ATTEMPT_EXTENSION` more attempts, never more than `ATTEMPT_LIMIT_MAX` in total at that stage;
- after `LESSON_IDLE_PROMPT` with no input, Squigglr shows the child a check-in, and after `LESSON_IDLE_END` with no input, the lesson ends; time without input never counts as a failed attempt.

Because `ATTEMPT_LIMIT` is always larger than `DIFFICULTY_STREAK` (SAFRAY-008), the adult always receives the suggestion before the lesson ends: the suggestion is the early warning and the limit is the backstop. The adult sets these values for each child within ranges set by the team. The examples below use illustrative values (`DIFFICULTY_STREAK` = 3, `ATTEMPT_LIMIT` = 6, `ATTEMPT_EXTENSION` = 2, `ATTEMPT_LIMIT_MAX` = 8) to show the expected behaviour; the tests themselves use the ends of each range.

1. T-SAF-001: Adult-only actions need the adult check

   Purpose: choosing a lesson, choosing a starting stage, switching child and accepting a go-back suggestion belong to the adult, so a child must not be able to complete any of them.

   Type: Dynamic, manual and automatic. Covers SAF-RAY-001.

   Initial State: an adult account with two child profiles; child A is in a guided lesson, and a go-back suggestion is waiting for the adult.

   Input: on the lesson screen, the tester (acting as the child) taps every visible control, taps the screen 50 times at random positions, opens the web address of each adult-only action directly, uses the browser's back and forward buttons and reloads the page. Each time the adult check appears, the tester leaves it or enters a wrong answer. Finally, the tester (acting as the adult) completes the adult check and accepts the suggestion.

   Output: every adult-only action asks for the adult check, and none completes until the check is passed. The suggestion stays waiting and the child can keep drawing at the current stage. After the adult passes the check and accepts, the child moves back one stage.

   Example: child A fails three attempts in a row at the endpoints stage, and a go-back suggestion appears. Child A taps "accept"; Squigglr asks for the adult check, which child A cannot complete, so the child stays at the endpoints stage and keeps practising. The adult then completes the check and accepts, and child A moves back to the tracing stage.

   Test Case Derivation: SAF-RAY-001 and its hazards H-RAY-001 and H-RAY-006. Random taps imitate a child who cannot read exploring the screen.

   How test will be performed: a Playwright script performs the taps, web addresses and navigation and checks the lesson state after each one; a tester repeats the taps by hand with the stylus on the iPad.

2. T-SAF-002: Notice on an unsupported device, browser or input

   Purpose: drawing results may be unreliable on a device or browser that is not in `SUPPORTED_DEVICES`, or when the child draws with a

finger instead of a stylus, so the adult must be told and can decide whether to continue.

Type: Dynamic, automatic and manual. Covers SAF-RAY-002.

Initial State: the adult has chosen a lesson; it has not started.

Input: (a) the lesson is started once with each combination in `SUPPORTED_DEVICES`, and once with each of three combinations that are not in it: an older browser version, a phone-sized screen, and a browser without pointer event support; (b) on the iPad, a supported lesson is started and the first stroke is drawn with a finger; (c) the same, with the first stroke drawn with the stylus.

Output: (a) for each unsupported combination, a notice the adult can read appears before the first attempt; for each supported combination, no notice appears; (b) the notice appears after the first stroke; (c) no notice appears.

Test Case Derivation: the fit criterion of SAF-RAY-002 (H-RAY-002). A browser cannot tell whether a stylus is connected before it is used, so case (a) checks only what the browser reports (version, screen size and pointer event support), and cases (b) and (c) check the input type the browser reports for the first stroke (pen or touch).

How test will be performed: for (a), Playwright sets the browser version, screen size and pointer event support for each combination and checks for the notice; for (b) and (c), a tester draws on the iPad, once with a finger and once with the stylus.

3. T-SAF-003: Attempts kept until the server confirms them

   Purpose: an attempt must never look saved when it is not, or the adult would make decisions from incomplete records.

   Type: Dynamic, automatic. Covers SAF-RAY-003 and SAF-SHA-003.

   Initial State: a guided lesson is open and online.

   Input: (a) the connection is cut during an attempt; (b) the connection is cut after the attempt is finished but before the server confirms the save, and the page is then reloaded, and separately closed and reopened, while offline; (c) the server returns an error for `FAILED_SAVES` saves in a row; (d) `FAILED_SAVES` saves succeed normally.

Output: in (a) to (c), no attempt is shown as saved before the server confirms it, the adult is told which attempts are not saved, and every attempt is saved once the connection or server is back; in (d), no failure message appears.

Test Case Derivation: SAF-RAY-003 and SAF-SHA-003 (H-RAY-002, H-SHA-003). Case (d) checks that the warning does not appear when nothing failed.

How test will be performed: Playwright turns the network off and makes the server return errors, then compares the server's records with the attempts made.

4. T-SAF-004: No diagnosis or comparison in summaries and exports

   Purpose: Squigglr is a practice tool, not an assessment, so no summary or export may label, diagnose or compare a child, and each must say so.

   Type: Static (inspection), manual. Covers SAF-RAY-004.

   Initial State: the templates for every summary and export at that revision.

   Input: one team member who did not write the templates reads each one.

   Output: every template states that it describes practice only and is not an assessment or diagnosis, and none uses a diagnostic label, judging wording (for example "struggling", "behind" or "below average") or a comparison with other children.

   Test Case Derivation: the fit criterion of SAF-RAY-004 (H-RAY-003), which asks for an inspection of every summary and export template. The team does not intend to add diagnoses, so the realistic risks are a new template that leaves out the required statement and wording that judges the child; a read of each template catches both.

   How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

5. T-SAF-005: Lesson ends automatically at the attempt limit

Purpose: a child must not be left repeating a stage they keep failing, so the lesson must end by itself at the limit the adult set.

Type: Dynamic, automatic. Covers SAF-RAY-005.

Initial State: a child profile with `ATTEMPT_LIMIT` and `ATTEMPT_LIMIT_MAX` at the lowest values in their ranges; the test is then repeated with the highest values.

Input: (a) at the tracing stage, the failing attempt `circle-fail.json` is replayed until the lesson ends; (b) the child passes tracing after a few failures, and the failing attempt is replayed at the endpoints stage until the lesson ends; (c) a sequence of attempts that improves enough to earn an extension is replayed until the lesson ends.

Output: the lesson ends after exactly `ATTEMPT_LIMIT` failed attempts at one stage, or `ATTEMPT_LIMIT_MAX` when extended, with no action from the child; failures at an earlier stage do not count at the new stage; and the saved record matches the attempts made, including the furthest stage reached.

Example: the sixth failed tracing attempt ends the lesson. In case (b), a child who passed tracing after four failures still gets six attempts at the endpoints stage. In case (c), an improving child gets attempts 7 and 8, and the lesson ends after the eighth failure.

Test Case Derivation: SAF-RAY-005 (H-RAY-004). Testing both ends of the ranges follows the supervisor's advice to test at the minimum and maximum (SRC-008).

How test will be performed: Playwright counts the attempts and checks that no attempt is accepted after the lesson ends.

6. T-SAF-006: Limit endings are shown to the adult across repeated lessons

Purpose: each time a lesson is done, it is a new lesson record, so the adult must see how often the lesson ended at the limit across all of them; doing the lesson again must not hide earlier failures.

Type: Dynamic, automatic. Covers SAF-RAY-006.

Initial State: as in T-SAF-005.

Input: the adult starts the lesson and it is run until it ends at the limit; the adult then starts the same lesson again, and this is repeated so the lesson is done three times. In the third run, an improving sequence earns an extension before the lesson ends at the limit.

Output: the adult's view of that lesson shows three lesson records, each ending at the limit, one extension, and every attempt from all three records.

Example: the adult sees "ended at the limit: 3 times; extended: once" and all $6 + 6 + 8 = 20$ attempts.

Test Case Derivation: SAF-RAY-006 (H-RAY-005).

How test will be performed: the script compares the counts and the list of attempts in the adult's view with the runs made.

7. T-SAF-007: Attempts saved only to the active child

   Purpose: on a shared device, one child's attempts must never be saved to another child's record.

   Type: Dynamic, automatic and manual. Covers SAF-RAY-007.

   Initial State: one adult account with three child profiles (A, B and C) on one device.

   Input: the adult switches between the children in the order A, B, A, C, and each child makes two attempts; the connection is cut once during child B's turn.

   Output: every practice screen shows the active child's avatar, and every attempt and suggestion is saved under the child whose avatar was shown when it was made.

   Test Case Derivation: SAF-RAY-007 (H-RAY-006). The lost connection checks that a delayed save still goes to the right child.

   How test will be performed: Playwright checks each child's records; a tester checks by hand that each avatar can be recognized without reading.

8. T-SAF-008: Adult-set values stay within their ranges

Purpose: a mistyped or extreme value could give a child an impossible lesson, so each value must stay within the range the team set and only the adult may change it.

Type: Dynamic, automatic. Covers SAF-RAY-008.

Initial State: a new child profile with no values chosen.

Input: (a) no value is set; (b) each value is set to its minimum, its maximum, one below the minimum and one above the maximum; (c) `ATTEMPT_LIMIT` is set equal to `DIFFICULTY_STREAK`, and `ATTEMPT_LIMIT_MAX` is set one below `ATTEMPT_LIMIT`; (d) a change is tried without completing the adult check.

Output: (a) each value is the middle of its range; (b) in-range values are saved and out-of-range values are refused; (c) both settings are refused; (d) nothing changes. The values in use are shown with the child's records.

Example: if the team-set range for `ATTEMPT_LIMIT` were 4 to 10, an unset value would be 7, values 4 and 10 would be saved, and 3 and 11 would be refused. With `DIFFICULTY_STREAK` = 3, an `ATTEMPT_LIMIT` of 3 is refused, because the lesson would end before the adult received the go-back suggestion.

Test Case Derivation: the fit criterion of SAF-RAY-008 (H-RAY-007), using boundary values on each side of each range.

How test will be performed: a Vitest test checks the validation function with every case, and Playwright repeats cases (b) to (d) through the settings screen and through direct server requests.

9. T-SAF-009: Duplicate input events counted once

Purpose: a stylus event reported twice must not change a drawing's result, or a child could pass or fail for no real reason.

Type: Dynamic, automatic. Covers SAF-AHM-001.

Initial State: a guided lesson at the tracing stage.

Input: `square-4-strokes.json` is replayed as recorded, then with every input event sent twice; then `overlap-2-strokes.json`, two separate strokes that cross, is replayed.

Output: the recorded points, classification and pass result are the same with and without the duplicates, and both crossing strokes are kept.

Test Case Derivation: the fit criterion of SAF-AHM-001 (H-AHM-002).

How test will be performed: Playwright replays both versions and compares the saved records.

10. T-SAF-010: Drawing kept when the canvas changes size

    Purpose: rotating the iPad or resizing the window must not distort a drawing or change its result.

    Type: Dynamic, automatic. Covers SAF-AHM-002.

    Initial State: a guided lesson with an attempt partly drawn.

    Input: the window is resized and the iPad is rotated during the attempt; then input events with positions outside the canvas, missing positions (`NaN`) and very large values are sent.

    Output: the order and relative positions of the points are unchanged, and the classification is the same as without resizing; none of the invalid points is counted or makes the attempt pass.

    Test Case Derivation: the fit criterion of SAF-AHM-002 (H-AHM-003).

    How test will be performed: Playwright resizes the drawing area and compares the scaled points with the original; a Vitest test sends the invalid values to the input handler.

11. T-SAF-011: Every data request checks the account

    Purpose: one family's or school's data must never be reachable from another account.

    Type: Dynamic, automatic. Covers SAF-SHA-001, with T-SEC-001.

    Initial State: two adult accounts, A and B, each with a child profile and saved attempts.

    Input: logged in as A, the tester sends requests to read, change and delete B's child profiles, attempts, settings and summaries to every server endpoint, then repeats them with no session and with an expired session.

Output: every request is refused, and none of B's data is returned or changed.

Test Case Derivation: the fit criterion of SAF-SHA-001 (H-SHA-001).

How test will be performed: a script lists every endpoint from the code and sends each request; the list is checked in the safety inspection, so a new endpoint cannot be missed.

12. T-SAF-012: No personal information sent to outside services

    Purpose: children's and adults' personal information must not leak to outside services, including the refinement service.

    Type: Dynamic, manual. Covers SAF-SHA-002.

    Initial State: an account whose adult name and email, and the child profile's identifier, are unique marker strings.

    Input: the tester uses every feature, including refinement, while recording all network traffic.

    Output: no request to an outside service contains any marker string, and requests go only to approved destinations.

    Test Case Derivation: the fit criterion of SAF-SHA-002 (H-SHA-002). Unique markers make any leak easy to find.

    How test will be performed: the test runs in Chrome on a laptop; the DevTools Network panel is exported as a HAR file and searched for the markers and for destinations that are not approved.

13. T-SAF-013: Logout after inactivity

    Purpose: an adult who walks away from a shared device must not leave their account and their children's data open.

    Type: Dynamic, automatic. Covers SAF-SHA-006.

    Initial State: an adult is logged in.

    Input: (a) no activity for `IDLE_TIMEOUT`; (b) activity just before `IDLE_TIMEOUT`, then no activity.

    Output: the adult is logged out after `IDLE_TIMEOUT` of inactivity and not before, and any later request using that session is refused.

Test Case Derivation: SAF-SHA-006 (H-SHA-001). Case (b) checks that activity restarts the timer.

How test will be performed: Playwright uses a controlled clock to move time forward and checks the session.

14. T-SAF-014: Feedback does not draw the missing part

    Purpose: feedback should show the child what is missing without drawing it for them, so the record shows only the child's own work.

    Type: Dynamic, automatic. Covers SAF-MOU-001.

    Initial State: a guided lesson at each predefined difficulty.

    Input: `square-missing-side.json`, an attempt with one side missing.

    Output: the feedback shows where the side is missing, and the strokes on screen and in the record are the same before and after the feedback.

    Test Case Derivation: the fit criterion of SAF-MOU-001 (H-MOU-004).

    How test will be performed: Playwright compares the stroke data before and after the feedback is shown.

15. T-SAF-015: Assisted, independent and creative work kept apart

    Purpose: the adult must be able to tell what the child drew with an on-screen guide from what they drew alone, so progress is not overstated.

    Type: Dynamic and static, manual. Covers SAF-MOU-002.

    Initial State: a prepared history of one lesson record holding 10 assisted attempts, 10 independent attempts, and 5 creative space drawings made separately.

    Input: every summary and record view is opened.

    Output: assisted and independent attempts are counted and shown separately, no creative space drawing appears, and no view ranks or rates the child by the number of attempts.

    Example: a lesson record with 10 successful attempts while tracing and 2 while drawing alone is summarized as "10 with a guide, 2 without", never as "12 successful drawings".

Test Case Derivation: the fit criterion of SAF-MOU-002 (H-MOU-005). The assisted or independent label belongs to each attempt, not to the lesson: following FR-LRN-012, an attempt is assisted when it is drawn with an on-screen guide (the tracing or endpoints stage) and independent when it is drawn with no guide. One lesson record normally holds both kinds. Help from an adult outside Squigglr, such as guiding the child's hand, is not detected or recorded, and this test does not cover it.

How test will be performed: a tester checks each view against the prepared history, and a Vitest test of the summary function compares the counts.

16. T-SAF-016: No more than three flashes per second

    Purpose: rapid flashing can trigger seizures, so no child-facing screen may flash more than three times in any one second.

    Type: Static (code review) and dynamic, manual. Covers SAF-MOU-003.

    Initial State: the code for every animation and transition on child-facing screens, and Squigglr open on the iPad with a guided lesson ready.

    Input: (a) a team member who did not write it reviews every animation and transition used on child-facing screens, which are all defined in Squigglr's own code and style sheets; (b) on the iPad, the screens that animate (all feedback, the lesson-end message, the inactivity check-in and the screen transitions) are screen-recorded.

    Output: (a) no animation or transition changes brightness or colour more than three times in any one second; (b) stepping through each recording frame by frame, no one-second window has more than three flashes.

    Test Case Derivation: the fit criterion of SAF-MOU-003 and WCAG 2.2 success criterion 2.3.1 (World Wide Web Consortium, 2023). Because every animation comes from our own code, reviewing that code finds a flashing effect before it reaches a screen; the recordings then confirm the result on the device, without recording every screen.

How test will be performed: once per revision (Rev 0 and Rev 1), as part of the safety inspection; any problem is recorded as a GitHub issue labelled `safety`.

17. T-SAF-017: Check-in and lesson ending after inactivity

    Purpose: a child may walk away during a lesson, and on a shared device another child could carry on under their profile, so an idle lesson must check in with the child and then end, with a clear record for the adult.

    Type: Dynamic, automatic and manual. Covers SAF-RAY-009.

    Initial State: a guided lesson at the endpoints stage, with `LESSON_IDLE_PROMPT` and `LESSON_IDLE_END` at the lowest values in their ranges; the test is then repeated with the highest values.

    Input: (a) no input for `LESSON_IDLE_PROMPT`, then a touch; (b) no input for `LESSON_IDLE_END`; (c) two failed attempts, then no input until the lesson ends.

    Output: (a) the check-in appears at `LESSON_IDLE_PROMPT`, the touch dismisses it, and the lesson continues at the same stage; (b) the check-in appears, then the lesson ends at `LESSON_IDLE_END`, the progress is saved, and the adult's view shows the lesson ended for inactivity; (c) the record shows exactly two failed attempts, so the idle time added none.

    Example: with illustrative values of 60 seconds for the check-in and 120 seconds for the end, a child who stops drawing sees an animated character at 60 seconds; if nobody touches the screen, the lesson ends at 120 seconds and the adult sees "ended: no activity".

    Test Case Derivation: the fit criterion of SAF-RAY-009 (H-RAY-006). Case (c) checks that inactivity is never counted as failure.

    How test will be performed: Playwright uses a controlled clock to move time forward and checks the screen and the saved record; a tester checks by hand on the iPad that the check-in can be understood without reading and does not flash.

4.2.3 Operational and Environmental

These tests cover SRS Section 13 (NFR-OPS-001 to NFR-OPS-003): how Squigglr reaches users through the browser and the outside services it depends on.

1. T-OPS-001: HTTPS only

   Purpose: all data between the browser and the server must be encrypted, so children's records cannot be read in transit.

   Type: Dynamic, automatic and manual. Covers NFR-OPS-001.

   Initial State: the preview deployment.

   Input: in Chrome on a laptop, the tester uses every feature with the DevTools Network panel recording, then opens the `http://` address of each page. (DevTools cannot record traffic on the iPad, and the server's behaviour does not depend on the device.)

   Output: every request uses `https://`; every `http://` address redirects to HTTPS or is refused.

   Test Case Derivation: the fit criterion of NFR-OPS-001.

   How test will be performed: the HAR from the normal HTTPS feature run is searched for any `http://` request; the intentional `http://` probes are checked separately for a redirect or refusal, and a Playwright test checks the redirects on every release.

2. T-OPS-002: Error message when a provider is down

   Purpose: when an outside service is down, the adult should get a clear message instead of a broken screen.

   Type: Dynamic, automatic. Covers NFR-OPS-002.

   Initial State: an adult on the login page, and separately an adult in a lesson.

   Input: requests to the authentication provider, and then to the hosting and storage service, are blocked to imitate an outage, and then unblocked.

Output: an error message is shown for as long as the provider is unavailable, no data is lost, and normal use resumes when it returns. Refinement outages are covered by the FR-CRT tests.

Test Case Derivation: the fit criterion of NFR-OPS-002.

How test will be performed: Playwright blocks the provider's domain with request routing and checks the page after each step.

3. T-OPS-003: No installation

   Purpose: families and schools should be able to use Squigglr in a browser without installing anything.

   Type: Dynamic, manual. Covers NFR-OPS-003.

   Initial State: a fresh browser profile, with no extensions, on the iPad and on each other combination in `SUPPORTED_DEVICES` that the team has access to.

   Input: the tester opens the address, creates an account and runs a lesson.

   Output: everything works with no download, plugin, extension or install prompt.

   Test Case Derivation: the fit criterion of NFR-OPS-003.

   How test will be performed: run together with T-PERF-007 and recorded in the same table.

4.2.4 Maintainability and Support

These checks cover SRS Section 14 (NFR-MAINT-001 to NFR-MAINT-004). Most are about the code and the process rather than running behaviour, so most are reviews or CI checks.

1. T-MAINT-001: Lessons are data, not code

   Purpose: new lessons should be added without changing code, so the app can grow after the capstone.

   Type: Static (code review) and dynamic, manual. Covers NFR-MAINT-001.

Initial State: the lesson catalogue at Rev 0.

Input: (a) a reviewer searches the source code for lesson content (shapes, stage order, feedback wording); (b) a test lesson for a new shape is added as a lesson file, edited and removed.

Output: (a) no lesson content is found in the source code; (b) the new lesson appears, changes and disappears in the app with no code change or rebuild, and its attempts are scored by the existing scoring.

Example: a lesson file holds the target outline as a list of points, the guides for each stage (full outline and endpoints) and the feedback messages. One scoring function compares the child's strokes with whatever outline it is given (how far the drawn points are from the outline and how much of it was covered), so a triangle lesson can be added as a new file in the same way as the circle and square lessons.

Test Case Derivation: NFR-MAINT-001. This covers lessons that use the existing scoring. A lesson that needs a new kind of scoring, for example letters, where stroke order matters, needs new code and is outside this test. How maintainers update lessons is still open in the SRS, so the add, edit and remove step will follow the SRS wording once it is settled.

How test will be performed: the review is done by a team member who did not write the catalogue module.

2. T-MAINT-002: Finding the module for a lesson behaviour

   Purpose: a new developer should be able to find the code behind a lesson behaviour quickly, so future fixes are practical.

   Type: Static (walkthrough), manual. Covers NFR-MAINT-002.

   Initial State: the code and the Module Guide (MG) at Rev 0.

   Input: a team member who did not write the lesson logic gets three questions, for example "where is the go-back suggestion triggered?", and uses only the MG and the repository to find the module that answers each.

   Output: each module is found within `LOCATE_TIME_MAX`.

   Test Case Derivation: the fit criterion of NFR-MAINT-002.

How test will be performed: once, at Rev 0, taking about 15 minutes; any question that takes longer becomes an issue to improve the MG or the module names.

3. T-MAINT-003: Every module has a written interface

   Purpose: each module should state what it offers, what it takes in and what it gives back, so a team member can use another module without reading its code.

   Type: Static (document review), manual. Covers NFR-MAINT-003.

   Initial State: the GitHub issues that create a top-level module, and the Module Interface Specification (MIS).

   Input: the Testing Lead checks each of those issues.

   Output: every issue asks for an interface description (the functions the module offers, their inputs and outputs, and what must be true before and after each call), and each description matches the MIS.

   Example: the issue that creates the scoring module asks for a description such as "evaluate(strokes, target outline) returns the distance and coverage measures and whether the attempt meets STAGE_PASS", and the MIS describes the same function.

   Test Case Derivation: the fit criterion of NFR-MAINT-003, plus the match with the MIS that the requirement's title names.

   How test will be performed: done as part of the MIS review in the Design Verification plan.

4. T-MAINT-004: CI blocks style violations

   Purpose: code that breaks the coding standard should be stopped automatically before it is merged.

   Type: Dynamic, automatic. Covers NFR-MAINT-004.

   Initial State: the CI pipeline described under Automated Testing and Verification Tools.

   Input: two test PRs: one with an ESLint error (an unused variable) and one with a Prettier violation (wrong indentation).

Output: CI fails on both and blocks the merge; after the fix, it passes.

Test Case Derivation: the fit criterion of NFR-MAINT-004 names two different violations.

How test will be performed: the DevOps Lead opens both PRs once the pipeline is set up, and repeats this whenever the lint rules change.

4.2.5 Security

These tests cover SRS Section 15 (NFR-SEC-001 to NFR-SEC-006). They use test accounts only. A child profile holds only a unique identifier generated by Squigglr and an avatar chosen by the adult or the child (SRS BR-001, NFR-SEC-004).

1. T-SEC-001: An adult can only reach their own children

   Purpose: an adult must only ever reach their own children's profiles and records.

   Type: Dynamic, automatic. Covers NFR-SEC-001, with T-SAF-011.

   Initial State: accounts A and B, each with one child profile and attempts.

   Input: as A, read and modify B's child profile and attempt records through direct server requests, using B's record IDs.

   Output: every request is refused, and A's own data is unchanged.

   Test Case Derivation: the fit criterion of NFR-SEC-001.

   How test will be performed: a script sends the requests with A's session; the access policies in the database are also reviewed in the safety inspection.

2. T-SEC-002: Logout ends the session

   Purpose: after logging out, nobody using the same device can continue the session.

   Type: Dynamic, automatic. Covers NFR-SEC-002.

   Initial State: an adult is logged in and the session token is copied.

Input: the adult logs out; the copied token is then used for an authenticated request, three times.

Output: all three requests are refused.

Test Case Derivation: the fit criterion of NFR-SEC-002.

How test will be performed: Playwright reads the token before logging out and sends the requests with it afterwards.

3. T-SEC-003: Audit event for a blocked change to a saved record

   Purpose: saved attempts are the record the adult relies on; any blocked attempt to change one must be logged.

   Type: Dynamic, automatic. Covers NFR-SEC-003.

   Initial State: a saved attempt record.

   Input: three requests to change the record, sent one after another.

   Output: every request is refused, each creates an audit event, and no event contains the record's contents.

   Example: a request to change a saved attempt's result from fail to pass is refused, and the audit event records the time, the account and the record's identifier, but not the drawing or the result.

   Test Case Derivation: the fit criterion of NFR-SEC-003.

   How test will be performed: the script reads the audit log through the team's database console and checks each event.

4. T-SEC-004: Minimal personal data

   Purpose: collecting less personal data lowers the harm if data is ever exposed.

   Type: Static (inspection), manual. Covers NFR-SEC-004.

   Initial State: the screen for adding a child and the database schema.

   Input: a reviewer lists every field on the screen for adding a child and every column of the child profile in the schema.

   Output: the only thing entered is the choice of avatar, and the only child profile columns are the identifier, the avatar and the adult-set

values; there is no name, birth date, photo or other child-identifying field.

Example: a field asking for the child's first name, even an optional one, fails the test.

Test Case Derivation: the fit criterion of NFR-SEC-004. The schema is included because a field can be stored without being on the screen.

How test will be performed: the review is done with the PIPEDA review in T-COMP-001.

5. T-SEC-005: Login failures and timeouts are logged privately

   Purpose: failed logins and timeouts are logged so problems can be found, without exposing private details in the logs.

   Type: Dynamic, automatic. Covers NFR-SEC-005.

   Initial State: a test adult account.

   Input: ten failed logins and ten logouts by inactivity (using the controlled clock from T-SAF-013); then a request for the audit log as that adult.

   Output: all twenty events are in the server-side log with a time stamp, and the adult's request is refused.

   Test Case Derivation: the fit criterion of NFR-SEC-005.

   How test will be performed: Playwright triggers the events and the script reads the log through the team's database console.

6. T-SEC-006: Text inputs are sanitized

   Purpose: text typed into the app must not be able to run code or reach the database.

   Type: Dynamic, automatic. Covers NFR-SEC-006.

   Initial State: every text field and every server endpoint that accepts text.

   Input: the probe list `injection-probes.txt`, built from the OWASP SQL injection and XSS filter-evasion cheat sheets, for example `' OR '1'='1` and `<script>alert(1)</script>`.

Output: no probe changes a query or runs a script; each is stored or shown as plain text or refused.

Test Case Derivation: the fit criterion of NFR-SEC-006.

How test will be performed: Playwright enters each probe in every field and checks the page for script execution, and a script sends each probe to every text endpoint.

4.2.6 Cultural

These tests cover SRS Section 16 (NFR-CULT-002): cultural sensitivity of symbols and imagery.

1. T-CULT-001: Style review of symbols and imagery

   Purpose: symbols and images should be suitable for families from different cultures.

   Type: Static (review), manual. Covers NFR-CULT-002 (SRS Section 16).

   Initial State: every icon, avatar, image and colour used to mean success or failure, at Rev 0.

   Input: two team members, and the supervisor at the Rev 0 demonstration, review each item against a checklist: hand gestures, animals, colours, religious or national symbols, and food.

   Output: no item has a known negative meaning in communities likely to use Squigglr, or every concern raised is fixed and approved by the reviewers.

   Test Case Derivation: the fit criterion of NFR-CULT-002.

   How test will be performed: concerns are recorded as GitHub issues and the review is repeated for new images before Rev 1.

4.2.7 Compliance

These reviews cover SRS Section 17 (NFR-COMP-001, NFR-COMP-002 and NFR-COMP-005): the Canadian and Ontario privacy laws that apply to Squigglr, and the selected WCAG 2.2 success criteria.

1. T-COMP-001: PIPEDA review

   Purpose: Squigglr collects personal information in Canada, so its handling must meet PIPEDA.

   Type: Static (document and code review), manual. Covers NFR-COMP-001.

   Initial State: a data inventory listing every personal-information field, where it is stored and where it is sent.

   Input: the inventory, checked against each of the ten PIPEDA fair information principles (Government of Canada, 2000).

   Output: each principle is recorded as met or not applicable, with none unmet.

   Test Case Derivation: the fit criterion of NFR-COMP-001.

   How test will be performed: two team members do the review at Rev 0, so any problem can still be fixed, and repeat it at the final demo (PH-3, 15 March 2027), using the network findings from T-SAF-012 and the field list from T-SEC-004.

2. T-COMP-002: Ontario MFIPPA review

   Purpose: Ontario school boards may use Squigglr, so its handling of student data must meet MFIPPA, the privacy act that covers school boards.

   Type: Static (document review), manual. Covers NFR-COMP-002.

   Initial State: the same data inventory, with retention and disclosure added.

   Input: the inventory, checked against MFIPPA Part II, which covers the collection, use, retention and disclosure of personal information.

   Output: no unmet obligation is recorded.

   Test Case Derivation: the fit criterion of NFR-COMP-002.

   How test will be performed: done with T-COMP-001; any obligation the team cannot judge is raised with the supervisor before adoption by a school board.

3. T-COMP-003: Selected WCAG 2.2 criteria review

   Purpose: the WCAG 2.2 criteria the SRS selects make Squigglr usable by children and adults with different abilities, so each one must be met.

   Type: Static (review) and dynamic, manual. Covers NFR-COMP-005.

   Initial State: the release build at the final demo (PH-3), on the iPad.

   Input: two team members check every screen against each WCAG 2.2 success criterion selected through NFR-UX-001, NFR-UX-004, NFR-UX-005 and NFR-UX-018 to NFR-UX-023 (World Wide Web Consortium, 2023).

   Output: each selected criterion is recorded as met on every screen, with none unmet.

   Test Case Derivation: the fit criterion of NFR-COMP-005, which asks for a review team. Full WCAG conformance is a stretch goal in the SRS, so only the selected criteria are checked.

   How test will be performed: the review is first done at Rev 0, so problems can still be fixed, and repeated at PH-3; each unmet criterion is recorded as a GitHub issue.

4.3 Traceability Between Test Cases and Requirements

Every functional and nonfunctional requirement in the SRS has at least one test or named check, and the most important ones have two. The tables below map each requirement to its tests. A letter after a test ID, like `T-LRN-005 (c)`, points to one case of a parameterized test. Waiting Room features (SRS Section 25) are not listed, since they are out of scope.

Table 42 maps each performance, safety-critical, operational, maintainability, security, cultural and compliance requirement to the tests that cover it. "SI" is the safety inspection described with the safety-critical tests. Every safety-critical requirement has at least two kinds of evidence, and several other requirements are also covered by a second test.

Table 42: Traceability for nonfunctional requirements

Requirement (SRS)	**Tests**
NFR-PERF-001	T-PERF-001, T-PERF-006
NFR-PERF-002	T-PERF-002, T-PERF-006
NFR-PERF-003	T-PERF-003, T-PERF-006
NFR-PERF-004	T-PERF-004 (unit and UI)
NFR-PERF-005	T-PERF-005, T-SAF-003
NFR-PERF-008	T-PERF-006
NFR-PERF-009	T-PERF-007, T-OPS-003
SAF-RAY-001	T-SAF-001, SI
SAF-RAY-002	T-SAF-002, SI
SAF-RAY-003	T-SAF-003, T-PERF-005, SI
SAF-RAY-004	T-SAF-004, SI
SAF-RAY-005	T-SAF-005, SI
SAF-RAY-006	T-SAF-006, SI
SAF-RAY-007	T-SAF-007, SI
SAF-RAY-008	T-SAF-008, SI
SAF-RAY-009	T-SAF-017, SI
SAF-AHM-001	T-SAF-009, SI
SAF-AHM-002	T-SAF-010, SI
SAF-SHA-001	T-SAF-011, T-SEC-001, SI
SAF-SHA-002	T-SAF-012, T-COMP-001, SI
SAF-SHA-003	T-SAF-003, SI
SAF-SHA-006	T-SAF-013, T-SEC-005, SI
SAF-MOU-001	T-SAF-014, SI
SAF-MOU-002	T-SAF-015, SI
SAF-MOU-003	T-SAF-016, SI
NFR-OPS-001	T-OPS-001
NFR-OPS-002	T-OPS-002
NFR-OPS-003	T-OPS-003, T-PERF-007
NFR-MAINT-001	T-MAINT-001
NFR-MAINT-002	T-MAINT-002
NFR-MAINT-003	T-MAINT-003
NFR-MAINT-004	T-MAINT-004
NFR-SEC-001	T-SEC-001, T-SAF-011
NFR-SEC-002	T-SEC-002
NFR-SEC-003	T-SEC-003
NFR-SEC-004	T-SEC-004, T-COMP-001
NFR-SEC-005	T-SEC-005

Requirement (SRS)	Tests
NFR-SEC-006	T-SEC-006
NFR-CULT-002	T-CULT-001
NFR-COMP-001	T-COMP-001
NFR-COMP-002	T-COMP-002
NFR-COMP-005	T-COMP-003

Table 43 maps each functional requirement to the tests that cover it. INS-LRN, INS-EVAL, INS-REC and INS-CRT are the code inspections planned under Implementation Verification. Tests from other sections that also cover a requirement are listed as a second approach.

Table 43: Traceability for functional requirements

Requirement (SRS)	Tests
FR-LRN-001	T-LRN-001, T-MAINT-001
FR-LRN-002	T-LRN-001, T-LRN-002, T-LRN-020
FR-LRN-003	T-LRN-003, T-REC-006 (a)
FR-LRN-004	T-LRN-004 (a), T-LRN-010 (a)
FR-LRN-005	T-LRN-004 (b), T-LRN-010 (b)
FR-LRN-006	T-LRN-004 (c), T-LRN-010 (c)
FR-LRN-007	T-LRN-005, T-REC-002, T-SAF-009, T-SAF-010, INS-EVAL
FR-LRN-008	T-LRN-006, T-SAF-009, INS-EVAL
FR-LRN-009	T-LRN-007, T-LRN-021, T-REC-005, INS-EVAL, INS-LRN
FR-LRN-010	T-LRN-008, T-LRN-021, T-SAF-010, INS-EVAL, INS-LRN
FR-LRN-011	T-LRN-009, T-LRN-011 (d), INS-EVAL
FR-LRN-012	T-LRN-010, T-SAF-015, INS-EVAL
FR-LRN-013	T-LRN-011, T-LRN-021, T-PERF-004, INS-EVAL
FR-LRN-014	T-LRN-012, T-SAF-014, INS-EVAL
FR-LRN-015	T-LRN-013, T-LRN-018, T-LRN-021, INS-LRN
FR-LRN-016	T-LRN-002, T-SAF-001, INS-LRN
FR-LRN-017	T-LRN-014, T-SAF-008
FR-LRN-018	T-LRN-015, T-LRN-021, T-SAF-001, INS-LRN
FR-LRN-019	T-LRN-016, T-LRN-021, INS-LRN

Requirement (SRS)	Tests
FR-LRN-020	T-LRN-016 (g), T-LRN-016 (h), T-LRN-017, T-LRN-021, T-SAF-005, INS-LRN
FR-LRN-021	T-LRN-018, T-LRN-021, T-SAF-005, INS-LRN
FR-LRN-022	T-LRN-019 (inspection and automatic), INS-EVAL
FR-LRN-023	T-LRN-020, T-LRN-021, INS-LRN
FR-CRT-001	T-CRT-001
FR-CRT-002	T-CRT-002
FR-CRT-003	T-CRT-003
FR-CRT-004	T-CRT-004
FR-CRT-005	T-CRT-005
FR-CRT-006	T-CRT-006
FR-CRT-007	T-CRT-007, T-CRT-006, T-CRT-016 (f), T-CRT-020 (b, e), T-CRT-022
FR-CRT-008	T-CRT-007 (b, c), T-CRT-020 (e), T-CRT-022
FR-CRT-009	T-CRT-008
FR-CRT-010	T-CRT-013, T-CRT-026
FR-CRT-011	T-CRT-014, T-CRT-015, T-CRT-023 (d, g)
FR-CRT-012	T-CRT-014, T-CRT-015, T-CRT-023 (e)
FR-CRT-013	T-CRT-016, T-CRT-027
FR-CRT-014	T-CRT-017, T-CRT-022
FR-CRT-015	T-CRT-018
FR-CRT-016	T-CRT-019, T-CRT-027, T-CRT-018, T-CRT-023
FR-CRT-017	T-CRT-020, T-CRT-022, T-CRT-021, T-CRT-027
FR-CRT-018	T-CRT-020 (a)
FR-CRT-019	T-CRT-021
FR-CRT-020	T-CRT-023, T-CRT-015
FR-CRT-021	T-CRT-024, T-CRT-025 (b)
FR-CRT-022	T-CRT-024 (b)
FR-CRT-023	T-CRT-009, T-CRT-008 (b), T-CRT-010, T-CRT-027
FR-CRT-024	T-CRT-009, T-CRT-008, T-CRT-027
FR-CRT-025	T-CRT-011 (a, b, d)
FR-CRT-026	T-CRT-010
FR-CRT-027	T-CRT-025, T-REC-002
FR-CRT-028	T-CRT-012
FR-CRT-029	T-CRT-011
FR-CRT-030	T-CRT-026
FR-REC-001	T-REC-001, INS-REC
FR-REC-002	T-REC-002, T-LRN-005, INS-EVAL

Requirement (SRS)	Tests
FR-REC-003	T-REC-003, INS-REC
FR-REC-004	T-REC-004, T-LRN-014 (l), INS-REC
FR-REC-005	T-REC-005, T-LRN-007 (b), T-LRN-007 (c), INS-REC
FR-REC-006	T-REC-006, T-SAF-006, INS-REC
FR-REC-007	T-REC-007, T-LRN-015 (f), INS-REC
FR-REC-008	T-REC-008, T-PERF-005, T-SAF-003, INS-REC
FR-REC-009	T-REC-009, T-SEC-003, INS-REC
FR-REC-010	T-REC-010, T-SAF-007, INS-REC
FR-PRG-001	T-PRG-001, T-PRG-005
FR-PRG-002	T-PRG-002, T-SAF-006
FR-PRG-003	T-PRG-003, T-PRG-013 (R1–R3), T-PRG-007
FR-PRG-004	T-PRG-004, T-PRG-013 (R4)
FR-PRG-005	T-PRG-005, T-PRG-013 (R4–R6), T-PRG-007 (f)
FR-PRG-006	T-PRG-006
FR-PRG-007	T-PRG-007, T-PRG-008 (b)
FR-PRG-008	T-PRG-008
FR-PRG-009	T-PRG-009, T-PRG-006 (c)
FR-PRG-010	T-PRG-010, T-PRG-002
FR-PRG-011	T-PRG-011
FR-PRG-012	T-PRG-012 (inspection and automatic), T-SAF-004

5 Unit Test Description

The Module Interface Specification (MIS) is due on 18 January 2027, so this section gives the rules for choosing unit tests rather than the tests themselves. The rules are written so that, once the MIS exists, each module's tests follow from its specification, and a reviewer can check that none is missing. The module tests and their traceability table are added in the VnV Plan revision after the MIS.

How unit tests are chosen. Each module's tests are designed first from its MIS (black box), then completed from the code (white box):

1. **Normal case.** Each access program gets at least one test of its usual behaviour, with the expected output or state change worked out from its semantics in the MIS.

2. **Boundaries.** Each input with a range is tested at its minimum and maximum and just outside them. Inputs that are SRS constants, such as `STAGE_PASS` or `ATTEMPT_LIMIT`, are tested at the ends of their ranges in SRS Section 4.2, as the system tests are.

3. **Errors.** Each exception in the MIS ($exc :=$) gets a test that raises it, and a test that a caller handles it.

4. **State.** For modules with state (abstract objects and templates), the state invariant is checked after every call in every test. For the lesson rules, each transition in the table of SRS Section 9.2 gets a test, and T-LRN-021 compares the module with a model of that section.

5. **Random inputs.** For modules that take shapes or sequences, a fast-check property test runs many generated inputs from a fixed seed (`MODEL_SEED`), as T-LRN-021, T-CRT-015 and T-CRT-022 already do.

6. **Coverage.** The coverage report (Section 3.6) is then read for branches that no test reaches, and a test is added for each one, or the reason it cannot be reached is written in the test file.

How tests are written and named. Tests are written in Vitest, as named in Section 3.6, in a file next to the module (`<module>.test.ts`). Each test's name gives the access program, the case and the expected result, for example "`evaluate -- empty stroke list -- returns incomplete`", and a comment cites the MIS access program and the SRS requirement it supports. The tests are given IDs `U-<MODULE>-###` for the traceability table. The test code is the detailed record of inputs and outputs, so they are not copied into this document. Outside services (the database, sign-in and the refinement service) are reached only through Squigglr's own wrapper modules, so unit tests replace them with fakes.

Who does what. The author of a module writes its unit tests in the same PR as the module. The PR reviewer checks the tests against the rules above as part of the PR code review in the Implementation Verification section. The Testing Lead (Huzaifah) checks that each expected output can be worked out from the MIS, as planned in Design Verification (Section 3.3).

5.1 Unit Testing Scope

Out of scope. We do not unit test code we did not write, which its makers verify (Section 2.2):

- the drawing library (Paper.js) and the browser's Pointer Events;
- the web framework (Next.js and React);
- the hosting and database services (Vercel and Supabase, including Supabase Auth);
- the refinement model.

Squigglr reaches each of these through a wrapper module of its own, and the wrappers are in scope: their unit tests use a fake of the service and check what Squigglr sends and how it handles each answer, including errors and timeouts. How the real services behave with Squigglr is checked by the system tests, for example T-CRT-014 for the real refinement service and T-OPS-002 for an outage.

Priority. Modules are tested in the order below, which follows the harm a defect could do (Hazard Analysis) and the TA's advice on coverage (Section 3.4). The coverage goals are the ones in Section 3.6; this list only sets the order and does not add a second policy:

1. **Modules tied to a safety-critical requirement:** any module that a SAF- requirement depends on or that a safety-critical test (Section 4.2.2) calls directly. These include the lesson rules, stroke evaluation and the input handler (T-SAF-010), attempt and lesson record saving, the adult check and access checks, settings validation (T-SAF-008) and the progress summary computation (T-SAF-015). A defect here can mislead an adult or lose a child's work. Goal, as in Section 3.6: every branch covered.
2. **Creative drawing logic:** the refinement result check and the drawing history (undo, redo and restore). A defect here can change or lose a drawing (H-MUH-001, H-MUH-002). Any of these that a SAF- requirement depends on is in the first group instead.

3. **Other logic modules:** for example, the parts of the progress summary that no SAF- requirement depends on.

4. **User interface components:** component tests with React Testing Library for each screen's states (empty, loading, error, normal). Most of their behaviour is covered by the end-to-end and usability tests.

Outside the first group, Section 3.6 sets no fixed target: reviewers check that new or changed logic has tests. The coverage reached for each module is reported in the VnV Report, as Section 3.6 describes.

5.2 Tests for Functional Requirements

The module tests for the functional requirements are added here once the MIS is done, with one subsection per MIS module. Each subsection will name the module's access programs and list its tests by `U-` ID, with the rule above that each one covers. Some unit-level tests are already planned as part of the system tests in Section 4.1, because they check one module directly:

- T-LRN-021, the lesson rules against a model of SRS Section 9.2;
- T-CRT-015, the refinement result check against random results;
- T-CRT-022, the drawing history against a model of undo, redo and restore;
- the Vitest parts of the guided lesson tests, which call the evaluation function and the input handler directly.

These tests stay in Section 4.1 with their IDs, because several of them run in both Vitest and Playwright under one ID, and the traceability table points to them there. Each module's subsection lists the ones that check it as cross-references, so the module's tests can be read in one place. If a test is later split, its unit part gets a new `U-` ID, and Section 4.1 and the traceability table are updated in the same PR.

5.3 Tests for Nonfunctional Requirements

Most nonfunctional requirements are about the whole product and are tested in Section 4.2. At module level, two kinds of test will be added once the MIS is done:

- **Timing of hot code.** A Vitest benchmark of the evaluation function and the refinement result check on the prepared stroke files, so a slow module can be found before T-PERF-001 and T-PERF-002 fail on the whole system.
- **Wrapper behaviour under failure.** Tests that each wrapper module times out, retries or reports an error as SAF-RAY-003 and the operational requirements expect, with a fake service that fails on purpose.

5.4 Traceability Between Test Cases and Modules

Once the MIS is done, a table here maps each MIS module to its unit tests and to the requirements they support, so every module can be seen to be considered. The table is complete when every module has a test file, or a written reason it has none (for example, a module that only re-exports types), and every access program has at least a normal case. The traceability check in Design Verification (Section 3.3) is extended to list any module with no tests, and it runs at the final check before each design revision.

References

Government of Canada. Personal information protection and electronic documents act (PIPEDA), S.C. 2000, c. 5. https://laws-lois.justice.gc.ca/eng/acts/P-8.6/, 2000. Last amended 2015.

Squigglr Team. Development Plan for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/DevelopmentPlan/DevelopmentPlan.pdf, 2026a.

Squigglr Team. Hazard Analysis for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/HazardAnalysis/HazardAnalysis.pdf, 2026b.

Squigglr Team. Problem Statement and Goals for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/ProblemStatementAndGoals/ProblemStatement.pdf, 2026c.

Squigglr Team. Software Requirements Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/SRS/SRS.pdf, 2026d.

Squigglr Team. Module Guide for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftArchitecture/MG.pdf, 2027a. In preparation, due January 2027.

Squigglr Team. Module Interface Specification for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/Design/SoftDetailedDes/MIS.pdf, 2027b. In preparation, due January 2027.

Squigglr Team. Verification and Validation Report for Squigglr. https://github.com/AHMEDELZARIA/squigglr/blob/main/pdfs/VnVReport/VnVReport.pdf, 2027c. Due March 8, 2027.

World Wide Web Consortium. Web content accessibility guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/, 2023. W3C Recommendation.

6 Appendix

This is where you can place additional information.

6.1 Symbolic Parameters

The definition of the test cases will call for SYMBOLIC_CONSTANTS. Their values are defined in this section for easy maintenance.

The constants below are used by the nonfunctional tests. The SRS leaves the timing thresholds to this plan; their values are set by measurement rather than chosen in advance, so each entry says how and when its value is set. Milestones use the SRS phase names: PH-1 is the proof-of-concept demo (23 November 2026), PH-2 is Rev 0 (1 February 2027) and PH-3 is the final demo (15 March 2027). Each constant includes its unit, so the tests name the constant alone (for example, "within `FEEDBACK_LATENCY_MAX`").

Table 44: Symbolic parameters for nonfunctional tests

Constant	Value or how it is set
`FEEDBACK_LATENCY_MAX`	Milliseconds. Set at the end of PH-1 from the target-tablet experiment in the Development Plan, run on the test iPad: the team measures the stroke latency of the first working build, checks whether drawing feels immediate, and sets the limit as the highest latency it accepts plus a margin, so that ordinary strokes of an accepted build do not fail. After that the limit is fixed and does not follow later builds. Recorded in the SRS constants table (NFR-PERF-001).
`REFINE_LATENCY_MAX`	Seconds. Set in PH-2 from the measured response time of the chosen refinement service (NFR-PERF-002).
`SUMMARY_LOAD_MAX`	Seconds. Set in PH-2 from the first measurement of T-PERF-003 (NFR-PERF-003).
`CONCURRENT_CAPACITY`	Sessions. Set in PH-2 from the number of schools and households expected during the capstone (NFR-PERF-008).

Constant	Value or how it is set
LOCATE_TIME_MAX	Minutes. Set by the team before the first T-MAINT-002 walkthrough (NFR-MAINT-002).
IDLE_TIMEOUT, FAILED_SAVES, SUPPORTED_DEVICES, ADULT_CHECK, STAGE_PASS, ATTEMPT_LIMIT, ATTEMPT_LIMIT_MAX, ATTEMPT_EXTENSION, DIFFICULTY_STREAK, LESSON_IDLE_PROMPT, LESSON_IDLE_END	As defined in the SRS constants table.
PERF_RUNS	Number of repeated runs per timing test on the test iPad. Proposed value: 30, enough to report a stable median and 95th percentile.
HISTORY_SIZE_LARGE	Number of attempts in the large prepared history. Proposed value: 500, well above a term of daily practice.

Prepared test data. The nonfunctional tests use stroke files that will be recorded once on the test iPad, before the first test run in PH-1, and stored in the repository under `tests/fixtures/`: `line.json`, `circle.json`, `square-4-strokes.json`, `circle-near-miss.json`, `circle-fail.json`, `square-missing-side.json`, `overlap-2-strokes.json`, `house.json` and `sun.json`, plus the probe list `injection-probes.txt`. Each file holds the pointer events (position, pressure, time) for one attempt.

The functional tests use the lesson constants at the ends of their SRS ranges, and add the test-only constants in Table 45.

Table 45: Symbolic parameters for functional tests

Constant	Value or how it is set
STAGE_PASS, DIFFICULTY_STREAK, ATTEMPT_LIMIT, IMPROVE_WINDOW, ATTEMPT_EXTENSION, ATTEMPT_LIMIT_MAX, IMPROVE_MARGIN	Ranges and defaults from SRS Section 4.2, restated because the tests use the ends of each range: 70 to 90 (default 80); 3 to 5 (4); 6 to 10 (8); 1 to 3 (2); 1 to 3 (2); 8 to 12 (10); IMPROVE_MARGIN is fixed at 5. If the SRS changes them, the test cells change with them.
TEST_CANVAS	1024 × 768 CSS pixels: the drawing area size for every replay, so stored coordinates can be compared exactly.
STROKE_PAUSE	2 seconds between strokes or before a check. Three times STROKE_PAUSE (used in T-LRN-006 (c)) must stay below the minimum of LESSON_IDLE_PROMPT once its range is set; if that minimum is 6 seconds or less, a quarter of it is used instead.
CAL_FAIL_MAX	60 points: `circle-fail.json` must score at most this at every drawing stage of the circle lesson.
CAL_NEAR_MISS	70 to 79 points: `circle-near-miss.json` must score in this range, and be classified complete, at every drawing stage.
CAL_PASS_MIN	90 points: `line.json`, `circle.json` and `square-4-strokes.json` must each score at least this, and be classified complete, at every drawing stage of their lesson.
CAL_INCOMPLETE_MIN	70 points: `square-missing-side.json` must score at least this, and be classified incomplete, at every drawing stage of the square lesson.
MODEL_RUNS	1,000 random sequences per run of T-LRN-021.
SEQ_LEN_MAX	60 events, enough to come close to the limit at T and E, then reach ATTEMPT_LIMIT_MAX (12) at I, with every suggestion answered.

Constant	Value or how it is set
MODEL_SEED	A fixed integer stored in the test file and printed with every result; replaced once per milestone, and recorded in the VnV Report.
CLOCK_START	16 November 2026, 10:00:00 America/Toronto: the controlled-clock start for the attempt record tests.
ATTEMPT_GAP	60 seconds between finished attempts in T-REC-001.
RETRY_DELAY	2 seconds of lost connection before the network is restored in T-REC-008.
COORD_TOL	Largest difference, in CSS pixels, at which two coordinates still count as equal when a test compares a saved drawing with the expected one. Value: 0.01 CSS pixels. This allows for floating-point rounding when coordinates are saved and reloaded, and is far smaller than one pixel on the test iPad's screen, so a stroke that has really moved still fails.
SCALE_TOL	Largest relative error allowed between a point's scale factor and the part's scale factor in a resize (T-CRT-004). Value: 0.001 (0.1%). Over a part 1000 CSS pixels across, this is at most one CSS pixel, so a visibly distorted shape still fails.

Constant	Value or how it is set
`REFINE_TOLERANCE`	The SRS constant (TBD in SRS Section 4.2): the largest distance, both ways, between a refined stroke and its original. *Unit:* CSS pixels on `TEST_CANVAS`. *How and when it is set:* by the team in PH-1, before the first run of T-CRT-014, from adult-made drawings only (SRS BR-009). A script makes copies of each drawing of the supported set in which every point is moved by a smooth random displacement of at most d, for $d = 2$, 4, 6, 8, 12 and 16 CSS pixels; the copies do not come from the refinement model. Two team members who did not make the drawings each view every original beside its copies on the test iPad, and say whether each copy is still clearly the same drawing, with no shape replaced. `REFINE_TOLERANCE` is the largest d that both accept for every drawing. This first value is needed early, because the refinement tests depend on it. *Confirmed:* in the adult usability sessions run by Shamil and Ahmed, as the Hazard Analysis asks for H-MUH-006 (a). Supporting adults who are not on the team view each adult-made drawing beside its copies for every d up to `REFINE_TOLERANCE` and beside its refined drawing from the model, and say whether each is still the same drawing. If any adult rejects a copy, the value is lowered to the largest d that every adult accepts for every drawing, and the refinement tests are run again. A rejected refined drawing that is within the value is recorded as a finding against the model, not the value. The value, the copies and every reviewer's answers are recorded in the VnV Report, and the SRS constants table is updated in the same PR. If OI-03 changes the supported set, the new drawings are judged the same way, and the value can only go down. Setting it without the model means the tests cannot be tuned to what the model happens to return.

Constant	Value or how it is set
REFINE_SHIFT	Half of REFINE_TOLERANCE: how far the shift stub moves every point. It is a valid refinement, far enough inside the limit that rounding cannot make it invalid, and far larger than COORD_TOL, so a stroke that was or was not shifted is easy to tell apart.
REFINE_DELAY	How long the stub holds its answer when the test does not release it: half of REFINE_LATENCY_MAX once that is set, and 2 seconds until then, so a held answer never counts as a timeout.
REFINE_RUNS	10 runs of each supported drawing in T-CRT-014, since the model may answer differently each time; 50 calls to the real service per run of the test.
CHECK_RUNS	2,000 random drawing and result pairs per run of T-CRT-015.
CRT_MODEL_RUNS, CRT_SEQ_LEN_MAX	500 random sequences of up to 40 events per run of T-CRT-022: enough for two refinements, edits, restores and a reopen in one sequence. The seed is MODEL_SEED.
RECUR_MIN	The SRS constant (SRS Section 4.2, proposed there and confirmed as below): how many of a learner's attempts at one stage of one lesson must show the same predefined difficulty for the summary to list it as recurring. *Proposed value:* 3 attempts. Two could be chance, and three at the same stage is the smallest count that shows a pattern. *How and when it is set:* the team shows the specialists summaries of learner B's history built with 2, 3 and 4 at the first specialist validation session after the summary is built (PH-2), and keeps 3 unless they advise otherwise. The value is recorded in the SRS constants table and the VnV Report. T-PRG-004 uses the symbol and generates its history from it, so the test does not change.

Constant	Value or how it is set
ADULT_REVIEWERS	The SRS constant (SRS Section 4.2): the fewest adult reviewers in T-PRG-013. *Value:* 5, set in this plan. Five adults outside the team (caregivers, educators or students) is what the team can recruit for one round of sessions, and it gives 15 tries of each requirement's three tasks. If fewer than five take part, the round is extended until five have; the number is not lowered.
REVIEW_PASS_RATE	The SRS constant (SRS Section 4.2): the smallest share of reviewer tasks in T-PRG-013 that must succeed, counted separately for each requirement. *Value:* 0.8, set in this plan, before any session, and not changed after the results are seen. With five reviewers, at least 12 of the 15 tries must succeed: one reviewer may fail every task, or several may each miss one, but a summary that misleads most readers on one task cannot pass.

Prepared test data for the functional tests. The functional tests use the files above plus the following, added to `tests/fixtures/`: `square-2-strokes.json`, `circle-gap.json`, `line-short.json`, `line-wrong-end.json`, `line-twice.json`, `circle-fast.json`, `empty.json`, `nan-points.json` and `order-unknown.json`, plus the term list `forbidden-terms.txt` (the terms of T-LRN-019). An adult team member records each stroke file once on the test iPad with the stylus, before the first run of the tests that use it (PH-1 or PH-2); no child takes part. The Testing Lead checks the calibration (the `CAL_*` constants) after recording, and again in CI whenever the evaluation function changes. A file that falls outside its band is recorded again; this is not a test failure. Each lesson added to the catalogue also needs a good-drawing fixture, recorded by an adult when the lesson is added and calibrated like the others: it must score at least `CAL_PASS_MIN` and be classified complete at every drawing stage. T-LRN-002 (a) to (d) and T-LRN-004 use it. The attempt record tests also use `square-missing-side-corrected.json` (`square-missing-side.json`, a pause until the missing-side feedback appears, then the fourth side), `square-endpoint-and-side.json` (the square at E with one side missing and one side ending away from its corner, a pause until the endpoint feedback appears, then

that side redrawn to the corner) and `history-seed.json` (the T-REC-009 history: three lesson records and 19 attempt records, loaded into the test database).

The creative drawing tests use the files below, made by adults like the files above and stored in the same folder. None is drawn by a child (SRS CON-005).

- `crt-draw-5.json`: the pointer events for five strokes drawn with the stylus: a straight line, a curve, a single tap, a closed loop, and a line that crosses the first. The fourth and fifth strokes are drawn with the stylus lifted only briefly between them.
- `crt-three.json`: a saved creative drawing of three strokes that do not touch, each in a different colour, using two stroke widths.
- `crt-cross.json`: a saved creative drawing of two strokes that cross at one point.
- `crt-dot.json`: a saved creative drawing of a stroke with one point and a straight line.
- `history-basic.json`: a learner's attempt and lesson records: two lesson records and eight attempt records, at known stages and with known results.
- `ref-circle.json`, `ref-square.json` and `ref-triangle.json`: free-hand drawings of a circle, a square and a triangle, drawn in the creative space without an outline. With `house.json` and `sun.json`, these stand in for the supported set in the refinement tests until OI-03 is closed (Section 4.1.3).
- `ref-out-scribble.json` and `ref-out-fish.json`: a scribble and a fish, drawings of items outside the supported set.

The progress summary tests load these histories into the test database, as in T-REC-009. Each is a JSON file of attempt and lesson records written by the team, not recorded from anyone, so no person's data is used:

- `prg-a.json`: learner A, the five lesson records and 32 attempts of Table 37, stored out of start order, with the latest attempt at 10:42:00 on 16 November 2026.

- `prg-b.json`: learner B, square lesson attempts with the difficulties of Table 39, generated by a script from `RECUR_MIN`.
- `prg-c.json`: learner C, three interrupted attempts at T of the circle lesson.
- `prg-d.json`: learner D, a circle lesson started at I with a failed attempt and then a passed one, which completed it (SRS BR-002), and a square lesson with three failed attempts at T, unfinished.
- `prg-f.json`: learner F, in the second account, a copy of A's history with new record IDs.
- `prg-h.json`: learner H, one circle lesson started at T: a failed attempt with `STAGE_PASS` at 80, then the adult changes it to 70 with the adult check, then a passed attempt with `STAGE_PASS` at 70, as in T-REC-004 (i). Then one passed attempt each at E and I completes the lesson.

Learner E has no records, and learner G has only the creative drawings of `crt-three.json`, saved three times.

6.2 Usability Survey Questions?

[This is a section that would be appropriate for some projects. —SS]

7 Appendix — Generative AI Use Disclosure

[ChatGPT (version 5) was used to brainstorm ideas for the template for this disclose section. —SS]

7.1 AI Tools Used

[Give the name of the AI tool(s) used and the version. —SS]

7.2 How and Where Used

[Explain what the tool(s) were used for. Examples include: brainstorming, explaining a technical concept, reviewing the document, generating or modifying text, generating or modifying diagrams, identifying errors or omissions or inconsistencies. —SS]

[For each of the uses, identify which parts of the document were affected. —SS]

[You do not need to report every interaction, but you should disclose the uses that materially contributed to your work —SS]

7.3 Verification of AI-Generated Content

[

- Describe how the team checked AI-generated or AI-assisted material before including it in the document.
- In particular, verify factual claims, technical assertions, requirements, calculations, references, and other information for which an AI system may produce incorrect or fabricated results.
- **The team is responsible for the accuracy, completeness, consistency, and appropriateness of the submitted document, regardless of whether material was generated by a human or an AI system.**

—SS]

Individual Disclosures

Rayan: I used Anthropic's Claude Opus 5.5 at medium effort, accessed through Claude Code, while preparing my sections of this document (the automated testing and verification tools, the performance, safety-critical, operational, maintainability, security, cultural and compliance tests, the traceability table, the symbolic parameters and my reflection). I made the testing decisions in my sections myself. A major use of AI was researching testing tools and approaches: I used the model to find tools that fit our planned technology stack, compare the options for each kind of test, and work out which suited our project best, then chose the tools myself. I also used it to identify which privacy and compliance standards apply to the data Squigglr collects, such as PIPEDA and the Ontario act that covers school boards (MFIPPA), and to work out strategies for the more technical tests, such as checking that all traffic uses HTTPS, trying to read or change another account's data, and probing text inputs for injection; I checked each standard and decided which tests to include myself. I also used it to check my test cases against the SRS requirements, the Hazard Analysis and the notes from our supervisor and TA meetings, to turn my notes and decisions into draft wording, to improve sentence structure, clarity and flow, and to help with LaTeX formatting. Through Claude Code, it also helped manage the repository (branches, commits and merges). I reviewed every AI-assisted passage before including it.

Huzaifah: I used Anthropic's Claude Opus 5.5 at high and extra-high effort through the omp coding agent, along with OpenAI's GPT-6.1 Sol (medium) helper agents that read the SRS, the Hazard Analysis and the Development Plan and returned summaries with line numbers, and a Claude Opus 5.5 (high) helper agent that reviewed my branches. I used AI in four ways. For planning, it helped me split my assignment (issue #196) into four smaller sub-issues and build a plan for them from the issue, our TA's rule for parameterized tests, the rubric, the VnV, Code and Writing checklists, the SRS, the Hazard Analysis and the Development Plan. For writing, I decided what each test checks and which cases to leave out where the SRS does not yet agree with itself; the model turned these decisions into draft test cases and LaTeX, and worked out the expected output of each attempt sequence from SRS Section 9.2. For review, the review agent checked my sections in three ways: against the sections my teammates had already merged, against the SRS and our requirements, and for correctness, recomputing every at-

tempt sequence on its own. Two scripts, written with AI help, also checked my work: one applies the rules of SRS Section 9.2 to the sequence tables, and one confirms that every FR-LRN and FR-REC requirement is in the traceability table. For Git, it managed my branches and commits, rebased my local branches onto `main` before I first pushed them and only merged after that, fixed merge conflicts, and wrote and opened my pull requests. In this document, this covered Section 3.5, the guided lesson and attempt record tests in Section 4.1, the functional traceability table in Section 4.3, the functional symbolic parameters and test data in Section 6.1, and my reflection answers. I reviewed every AI-assisted passage before including it. All decisions in my sections are my own. **Ahmed:** I used Anthropic's Claude Opus 5.5, accessed through Claude Code, while preparing my sections of this document: the creative drawing and progress summary tests in Section 4.1, their rows in the traceability table, their symbolic parameters and test data, Design Verification (Section 3.3), the Unit Test Description (Section 5) and my reflection answers. All the ideas behind these sections are my own: what each test checks, which cases try to break Squigglr, how the constants are set, and how the design is reviewed. I used the model to turn my notes and decisions into draft wording, and to clean up grammar, sentence structure and clarity, along with LaTeX formatting. I reviewed every AI-assisted passage against the SRS, the Hazard Analysis and the parts of this plan my teammates wrote, and checked the expected outputs of my tests myself, before including it. The final content is my responsibility.

8 Appendix — Reflection

The information in this section will be used to evaluate the team members on the graduate attribute of Lifelong Learning.

The purpose of reflection questions is to give you a chance to assess your own learning and that of your group as a whole, and to find ways to improve in the future. Reflection is an important part of the learning process. Reflection is also an essential component of a successful software development process.

Reflections are most interesting and useful when they're honest, even if the stories they tell are imperfect. You will be marked based on your depth of thought and analysis, and not based on the content of the reflections themselves. Thus, for full marks we encourage you to answer openly and honestly and to avoid simply writing "what you think the evaluator wants to hear."

Please answer the following questions. Some questions can be answered on the team level, but where appropriate, each team member should write their own response:

1. What went well while writing this deliverable?

   Team: Something that really went well was talking to Chris, our TA, and Dr. Anand, our supervisor, before the plan was finished. In our meeting with Chris on October 28, 2026, we asked about the things we were unsure of: whether thresholds we cannot set yet can be decided by measurement, how many tests a safety requirement needs, whether we need a fixed code coverage target, and whether one parameterized test can cover several boundary cases. His answers helped us narrow down which testing strategies to use and how to run certain tests, and showed us what in our first drafts was not clear yet, for example that every TBD value in the SRS should point to the method in this plan, and that we needed to justify why one test plus a safety inspection gives enough confidence. Chris also told us that validation and usability matter more than verification for this project, which is why they come right after safety in our objectives. Dr. Anand told us the same day to go back to the specialists we interviewed, or people with similar qualifications, to validate the app.

   Our process also went better than last time. In the SRS and Hazard Analysis we had many merge conflicts, mostly because everyone added

rows to the same revision history table. This time Rayan suggested that everyone send their revision history entries to Moustafa, who is doing the final quality check and adds them all at the end. We also started from one set of shared notes in all five VnV issues, so everyone knew which sections and test IDs they owned from the start. Rayan, Huzaifah and Moustafa split their parts into small sub-issues so each piece could be reviewed on its own, and Rayan's five branches merged together without conflicts when tested.

2. What pain points did you experience during this deliverable, and how did you resolve them?

   Team: The main pain point was that writing the plan kept sending us back to the SRS. Every test has to match a requirement, so each time a meeting or a test changed how something should work, we had to go back through the SRS, change it, and then check that our tests still matched it. The main changes came from two meetings:

   - *Our supervisor meeting on October 28, 2026.* Dr. Anand advised that we should not collect children's personal details, so a child's profile now holds only a generated identifier and an avatar instead of a first name and username. This changed the business rules, the data dictionary, a use case, a security requirement and the Hazard Analysis, as well as the security test.
   - *Our TA meeting on the same day.* Chris advised that a value left as TBD should say how it will be decided, so every TBD performance value in the SRS now points to the measurement method in this plan, and the other TBD constants get a method in the Symbolic Parameters appendix.

   We resolved this by keeping all of these SRS changes in one issue and one pull request, separate from the VnV branches, and by re-reading each affected test against the updated requirement before marking a branch ready for review.

   Another pain point was that all five of us were writing the same `.tex` file at the same time. Sections on separate branches could not see each other until they merged, and some parts depended on others. For example, Moustafa's section introductions had to be stacked on top of

the front matter pull request. We resolved this by writing only in our own sections and rebasing onto `main` as other parts merged.

Our shared notes also told us to use the template's Control field, but Rayan's tests, the first ones merged, used Purpose and Type instead. Chris had told us we are not bound by the template's format, so we kept Purpose and Type and asked everyone to use the same fields.

3. What knowledge and skills will the team collectively need to acquire to successfully complete the verification and validation of your project? Examples of possible knowledge and skills include dynamic testing knowledge, static testing knowledge, specific tool usage, Valgrind etc. You should look to identify at least one item for each team member.

   Rayan: For my part of the plan (performance, safety-critical, security and compliance tests), I need three things I do not yet have:

   - *End-to-end test automation with Playwright*, in particular simulating network loss, replaying recorded stylus input and controlling the clock. Most of my safety tests depend on these, for example T-SAF-003, T-SAF-013 and T-SAF-017.
   - *Security testing of a web back end*: checking access policies in the database, reusing session tokens and sending injection probes. As Backend Lead, I am responsible for the code these tests target.
   - *Performance measurement on the iPad*: taking repeatable timings in the browser and summarizing them fairly, so that the thresholds the SRS leaves open can be set from evidence.

   Moustafa: I need to learn how to actually check a screen against WCAG. So far I have only used WCAG to write requirements, like NFR-UX-018 to NFR-UX-023. Writing "text needs a 4.5:1 contrast ratio" is easy, but T-COMP-003 has two of us check every screen against those criteria, and I review the frontend code, so I should be catching these problems before that review. This matters even more for us because we only evaluate with adults (SRS CON-005), so no child will be there to show us that a button is too small or that an icon makes no sense. The skill has two parts:

   - *Using accessibility tools*: a contrast checker for NFR-UX-019 and an automated checker for NFR-UX-020, and knowing what these

tools miss.

- *Checking by hand*: measuring touch targets for NFR-UX-018, using VoiceOver on the iPad for NFR-UX-020, and finishing the adult tasks with only a keyboard for NFR-UX-023.

Huzaifah: For my part of the plan (the guided lesson and attempt record tests and the code inspections), I need two skills I do not yet have:

- *Model-based testing.* T-LRN-021 needs a reference model of the lesson rules in SRS Section 9.2, and properties that fast-check can check over 1,000 random attempt sequences in each run. I have written example-based unit tests, but never a model precise enough to serve as the expected output.
- *Running code inspections.* As moderator of INS-LRN, INS-EVAL, INS-REC and INS-CRT, I have to plan each inspection, keep the meeting to an hour, sort findings into major and minor, and follow up on fixes. So far I have only reviewed pull requests on my own.

Ahmed: For my part of the plan (the creative drawing and progress summary tests, design verification, and the usability sessions with Shamil), I need three skills I do not yet have:

- *Property-based testing with fast-check.* T-CRT-015 and T-CRT-022 only work if the generators produce the inputs that matter, such as a refinement result just past `REFINE_TOLERANCE` or an undo right after a reopen, and if the reference model is simple enough to trust as the expected output.
- *Faking an outside service in Playwright.* Most refinement tests depend on intercepting Squigglr's calls to the refinement service and answering with a stub that can shift strokes, hold its answer, fail, or send a broken or partial result (T-CRT-013, T-CRT-023 and T-CRT-024). If the stub is wrong, every one of those tests is wrong with it.
- *Running reviews with other people.* I moderate the design reviews in Section 3.3. With a teammate I set the first value of

`REFINE_TOLERANCE` in PH-1, and with Shamil I run the adult sessions where T-PRG-013 is done and where supporting adults confirm that value. This means keeping a review focused on findings, and giving reviewers a task without hinting at the answer, so a success really means the summary is clear.

4. For each of the knowledge areas and skills identified in the previous question, what are at least two approaches to acquiring the knowledge or mastering the skill? Of the identified approaches, which will each team member pursue, and why did they make this choice?

   Rayan:

   - *Playwright.* Two approaches: work through the official Playwright documentation and examples on network emulation and clocks, or learn from the first tests the Testing Lead writes. I will start with the documentation and write T-PERF-005 as a first test as soon as the scaffold exists. This is because the network and clock features are exactly what my tests need, and writing a real test early shows quickly what I am missing.
   - *Security testing.* Two approaches: follow the OWASP Web Security Testing Guide for the checks that apply to us, or rely on the hosting provider's security advice and default settings. I will use the OWASP guide, because it gives a checklist we can repeat each release, while default settings alone would not show us that our own access rules are correct.
   - *Performance measurement.* Two approaches: learn the browser developer tools (Safari Web Inspector for the iPad, Chrome DevTools on a laptop), or write timing code with the browser Performance API. I will do both, starting with the Performance API in Playwright scripts so the measurements can be repeated, and using Safari Web Inspector to understand any slow result on the iPad.

   Moustafa: Two approaches: work through the W3C's WCAG 2.2 pages for the criteria we selected and try each check with the tools, or learn by doing hands-on passes of our own screens, on the iPad with VoiceOver and on a laptop with only the keyboard. I will start with the W3C pages, because T-COMP-003 has to record each criterion as

met or not, so I need to know exactly what passing means. Full WCAG conformance is not required, so the list is short enough to get through before the first review at Rev 0. After that I will do hands-on passes on the first frontend screens I review, because a checker can tell me a control has a label, but not whether the label makes sense to someone who can't see the icon.

Huzaifah:

- *Model-based testing.* There are two approaches: work through the fast-check documentation on model-based testing, or read the chapters on state-machine models in Utting and Legeard's *Practical Model-Based Testing*. I will start with the documentation, because it covers fast-check's own model-based testing API, which T-LRN-021 uses directly. I will model the suggestion rule as soon as the first version of the lesson rules exists. Modelling one rule early, against our first lesson rules code, shows quickly where the code and my reading of the SRS differ. Ahmed is learning property-based testing with fast-check for his creative drawing tests, so we will share what we learn. I will turn to the book if modelling all of Section 9.2 gets hard.
- *Code inspections.* There are two approaches: read the inspection procedure in IEEE Std 1028, the standard for software reviews and audits, or learn by doing a trial inspection. I will do both, in that order. I will read the roles and steps in IEEE 1028, then run a 30-minute trial inspection of one small PoC file with one teammate, using the checklist in Implementation Verification, before the PH-1 inspections, which must be completed by 20 November 2026. The trial shows whether the checklist and the one-hour limit work before real inspections depend on them.

Ahmed:

- *Property-based testing.* Two approaches: work through the fast-check documentation and its examples of model-based testing, or start from small properties of my own code and grow them. I will start with the documentation, then write T-CRT-022's history model for undo and redo first, before restore is added. The

documentation shows how shrinking and seeds work, which T-CRT-015 and T-CRT-022 rely on, and a small model that already passes is easier to trust when I add restore and refinement to it.

- *Faking a service in Playwright.* Two approaches: read the Playwright guide on network interception (`page.route`), or run a local mock server that pretends to be the refinement service. I will use `page.route`, because it lives inside each test, so a test can hold, release or break the answer at exactly the step it needs. A mock server would need its own set-up and could fall out of step with the tests. I will build the shift stub first, as soon as the creative space can send a request.
- *Running reviews.* Two approaches: read about inspection roles and usability test facilitation (for example the Nielsen Norman Group's guides on running usability sessions), or practise on a small scale first. I will do both: read about facilitation before the first session, and run a 20-minute practice of the S1 walkthrough with the team at the first internal MIS review, and a practice of the T-PRG-013 tasks with one teammate before any outside reviewer sees them. A practice run shows which tasks or questions give the answer away while it is still cheap to fix them.